

머신러닝 이해와 활용

김 화 종 (강원대학교)

hjkim3@gmail.com

목차

DX/AI/ML의 이해

파이썬 핵심

데이터 탐색과 전처리

머신러닝 모델

클러스터링

딥러닝

이미지 분석

객체검출

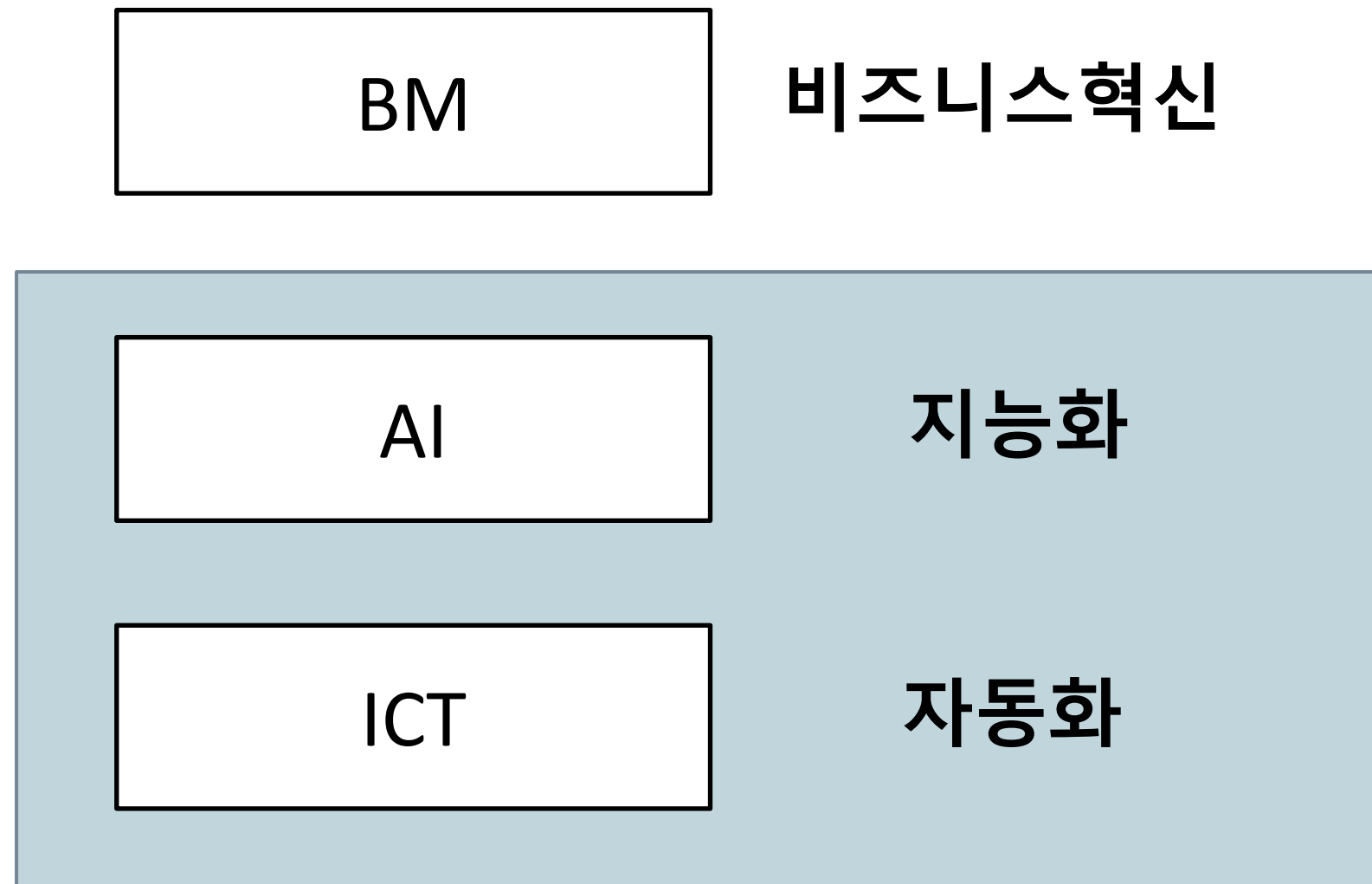
시계열 분석

예지보수

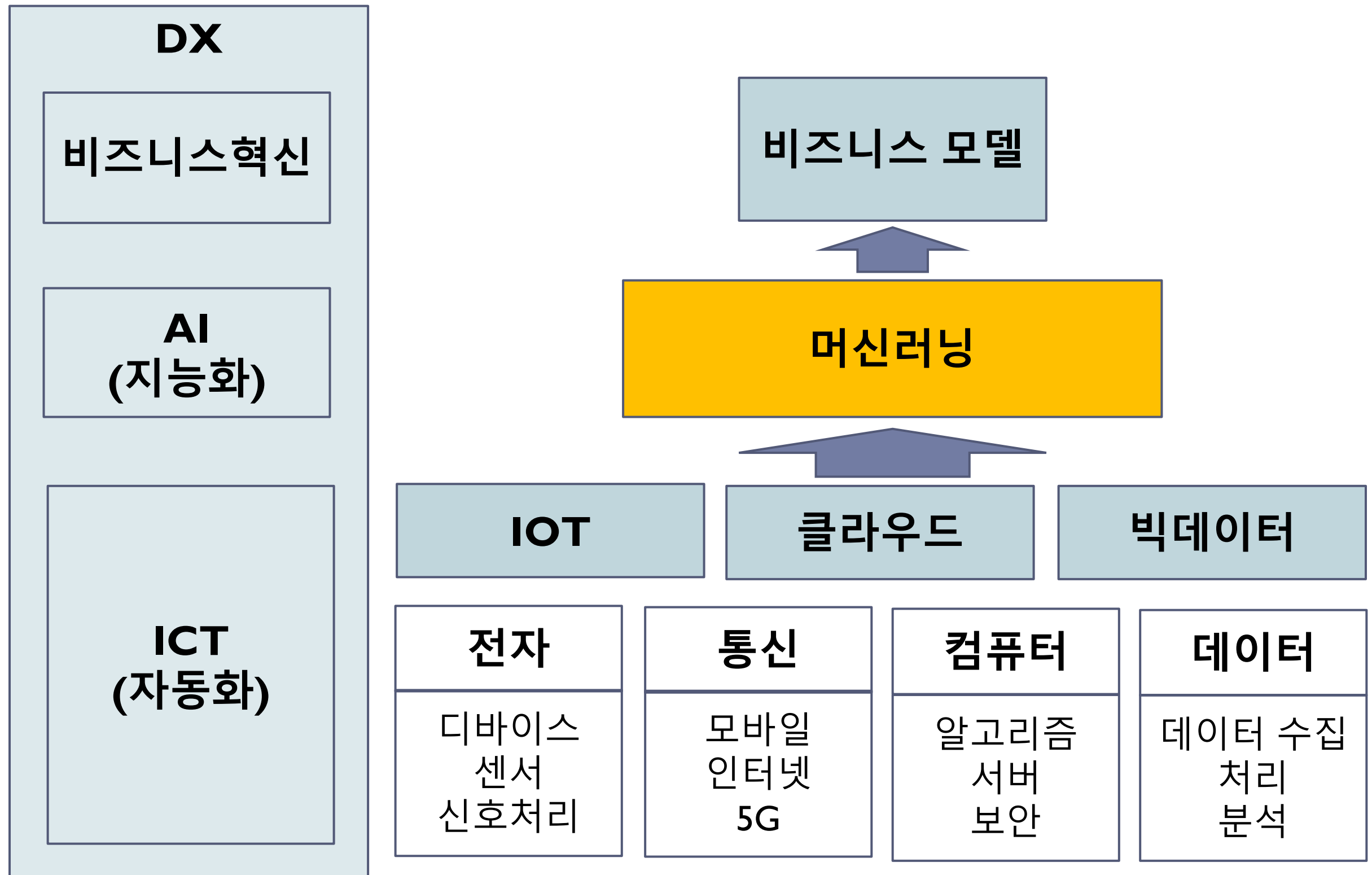
텍스트 분석

DX/AI/ML의 이해

DX의 범위



DX의 구성



자동화와 지능화

▶ 자동화

- ▶ 프로그래머가 코딩한 로직대로 동작한다
- ▶ 정해진 알고리즘(플로우 차트)대로 업무를 빠르고 정확하게 수행
- ▶ 개발자(사람)의 지식과 경험이 성능을 좌우

▶ 지능화

- ▶ 데이터를 보면서 모델(소프트웨어)의 성능이 점차 개선된다
- ▶ 학습에 사용하는 데이터의 양과 다양성에 따라서 성능이 향상
- ▶ 개발자 역할은 모델을 잘 만들고 좋은 데이터를 공급하는 것

AI의 정의

▶ AI란

- ▶ “컴퓨터가 마치 지능이 있는 것처럼 똑똑하게 동작하는 것”을 말한다
- ▶ 내비게이터, 음악, 영화, 상품 추천, 예지정비 등
- ▶ AI가 특정한 기술을 사용해야만 하는 것이 아니다. 예를 들어 신경망 (딥러닝) 을 사용해야만 하는 것은 아니다

▶ AI를 구현하는 기술

- ▶ 사람처럼 “생각하는 능력”을 구현하려고 시도했고 언어학, 기호학 기반의 접근을 했으나 성공하지 못했다
- ▶ 전문가의 지식을 코딩하는 “전문가 시스템”도 성공하지 못했다

▶ 머신러닝 기반 AI

- ▶ 현재 동작하는 AI는, 데이터를 보고 스스로 학습하여 성능이 점차 개선되는 “머신러닝” 방식의 AI이다

AI의 유형

▶ 예측 모델

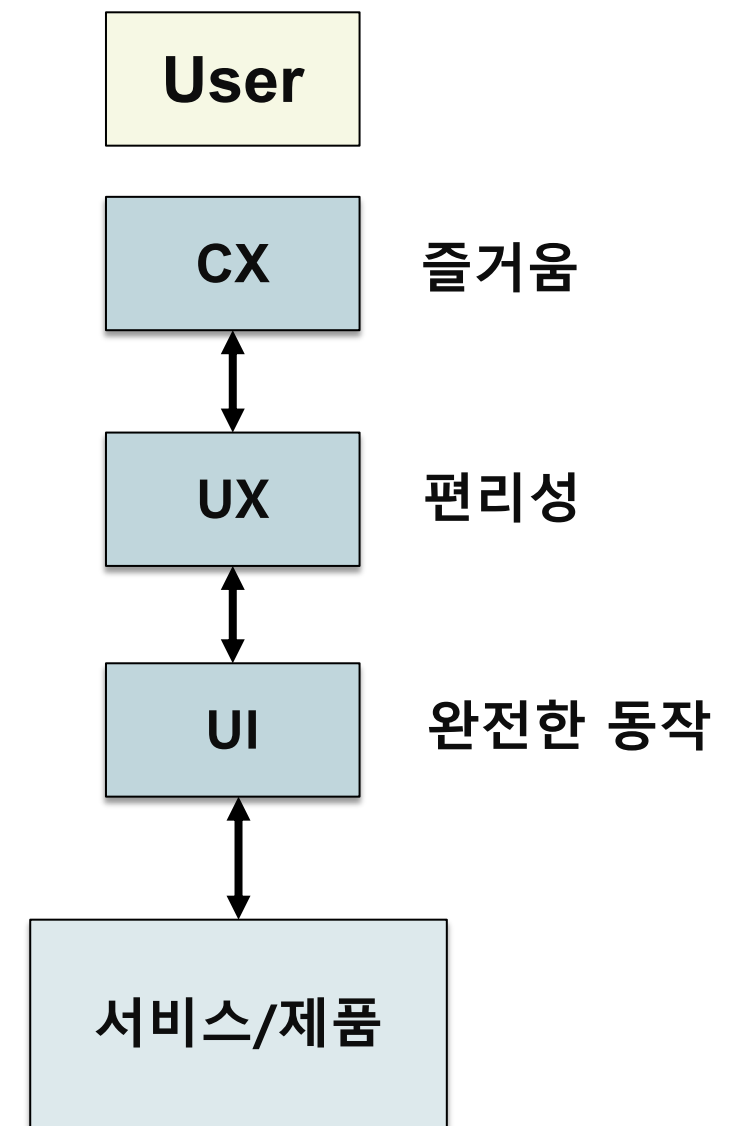
- ▶ 회귀 예측, 분류 예측
- ▶ 수요예측, 재고예측, 판매예측, 장애예측, 양불판정, 성능예측 등
- ▶ 딥러닝 (신경망)의 발전으로 이미지, 음성, 텍스트, 생물정보 등 비정형 데이터에 대해서도 성능이 급속히 향상

▶ 생성 모델

- ▶ 텍스트, 이미지, 음악, 프로그램 코드, 약물후보 등을 생성하는 모델
- ▶ chatGPT
 - ▶ 2022.11 OpenAI 사가 개발한 대화형 AI 서비스
 - ▶ 사람과의 자연스러운 대화가 가능한 AI 채팅 서비스
 - ▶ 대용량 데이터로 사전학습 후 다양한 후속(downstream) 작업에 적용
 - ▶ 감성분석, 키워드 추출, 문서요약, 질의 응답, 기계번역, 스토리 생성 등

DX 관련 기술 3

- ▶ AI가 도입되면서 UI/UX/CX도 변화중
 - ▶ 점차 지능화된 UI/UX/CX 를 기대한다
- ▶ 사용자는 설명서를 읽지 않는다.
 - ▶ Make it Simple!
 - ▶ 단순하게 만들어야 성공한다
- ▶ 사용성(usability) 테스트
 - ▶ 이용자의 사용성/만족도 테스트에도 AI를 도입
 - ▶ 개발자가 설계한대로 사용자가 사용할 것이라고 기대하면 안된다



DX 이용자별 니즈

- ▶ 외부 고객의 니즈
 - ▶ 비용과 시간 절감
 - ▶ 편리성 (UX)과 즐거움 (CX) 개선
 - ▶ 안전성 개선
- ▶ 내부 고객의 니즈
 - ▶ 솔루션(모델) 도입과 운영의 편리성
 - ▶ 데이터 입력, 가공의 편리성
 - ▶ 도입시 실제적인 효과
 - ▶ 투자비용 (인건비, 장비, 시설 등)

빅데이터 분석

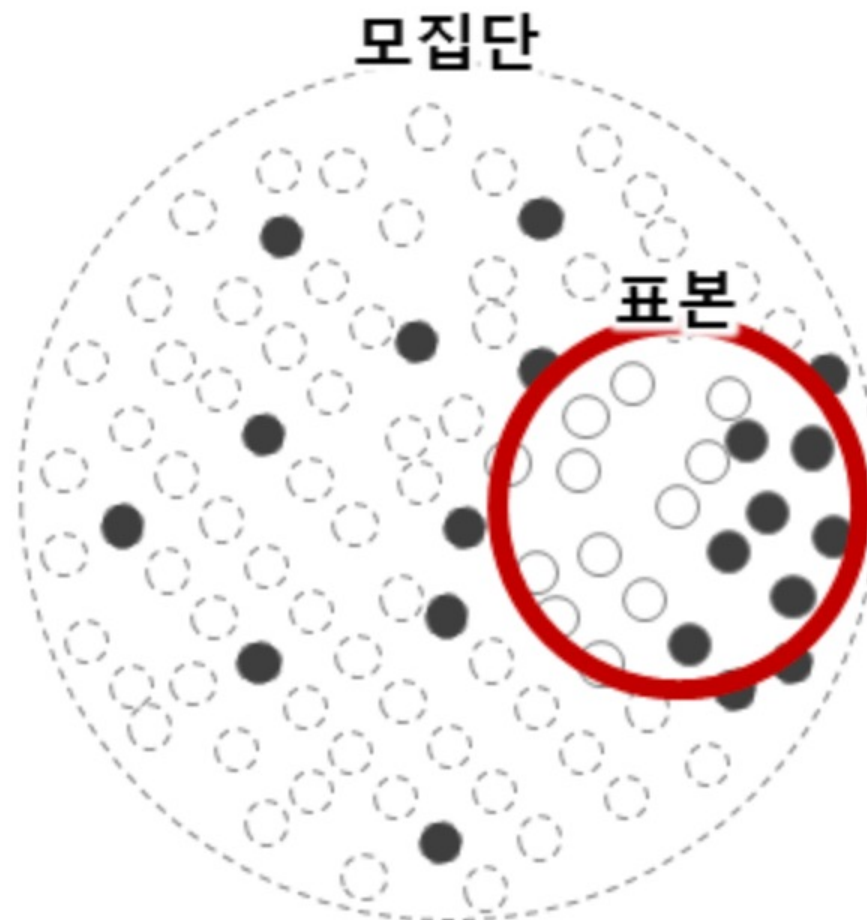


AI 활용 유형

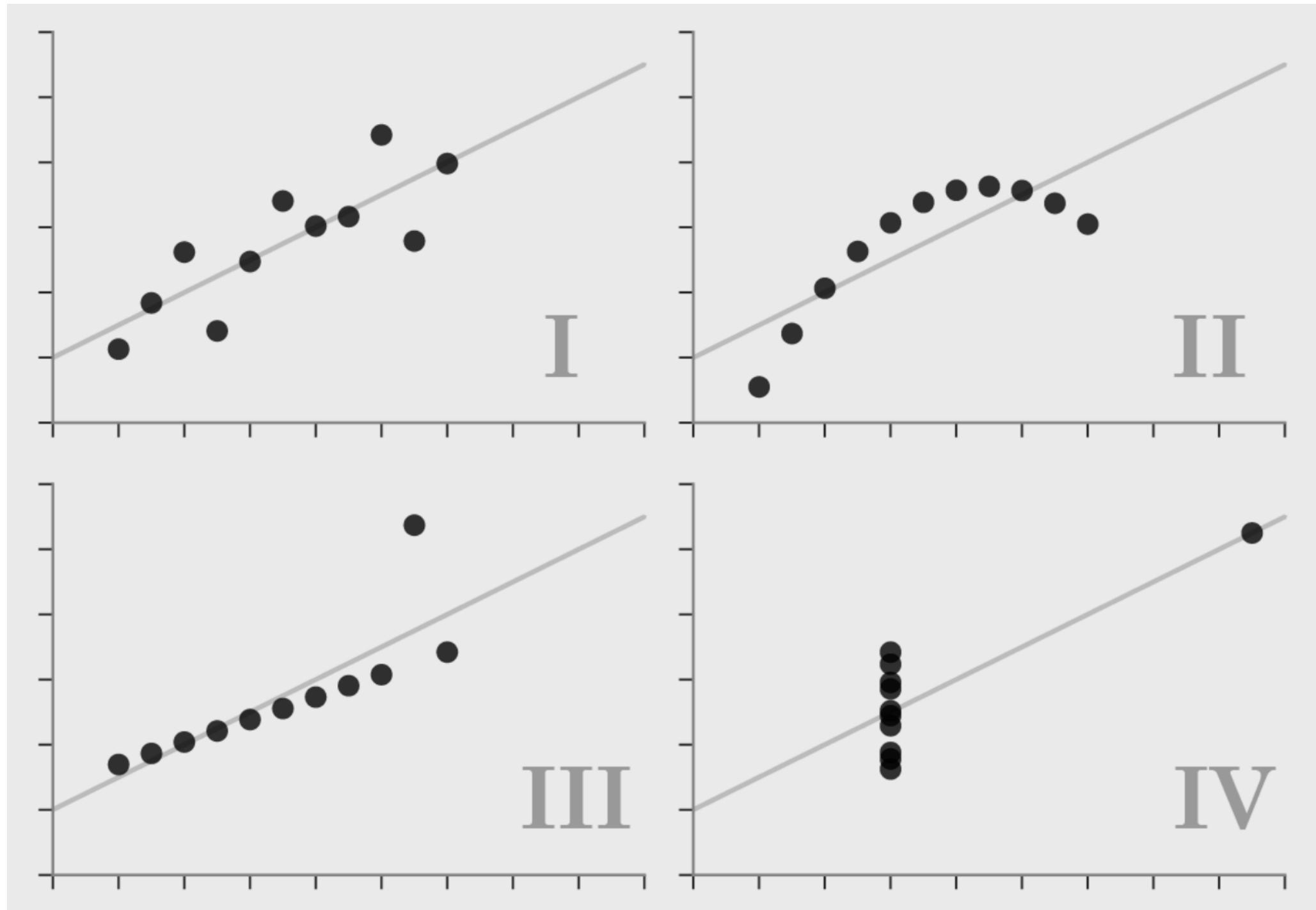
- ▶ 예측 모델 (predictive model)
 - ▶ 회귀 예측 (regression)
 - ▶ 분류 예측 (classification)
- ▶ 생성 모델 (generative model)
 - ▶ 텍스트, 이미지, 동영상, 음악, 프로그램 코드, 신약후보 물질 등
- ▶ 최적화 (optimization)
 - ▶ 최적의 입력조건, 제어변수 선택, 운영환경 조건 선택
- ▶ 추천 (recommendation)
 - ▶ 최적의 선택지 추천, 의사결정 지원
 - ▶ 상품추천, 영화 추천, 최적 설계 등

통계 분석

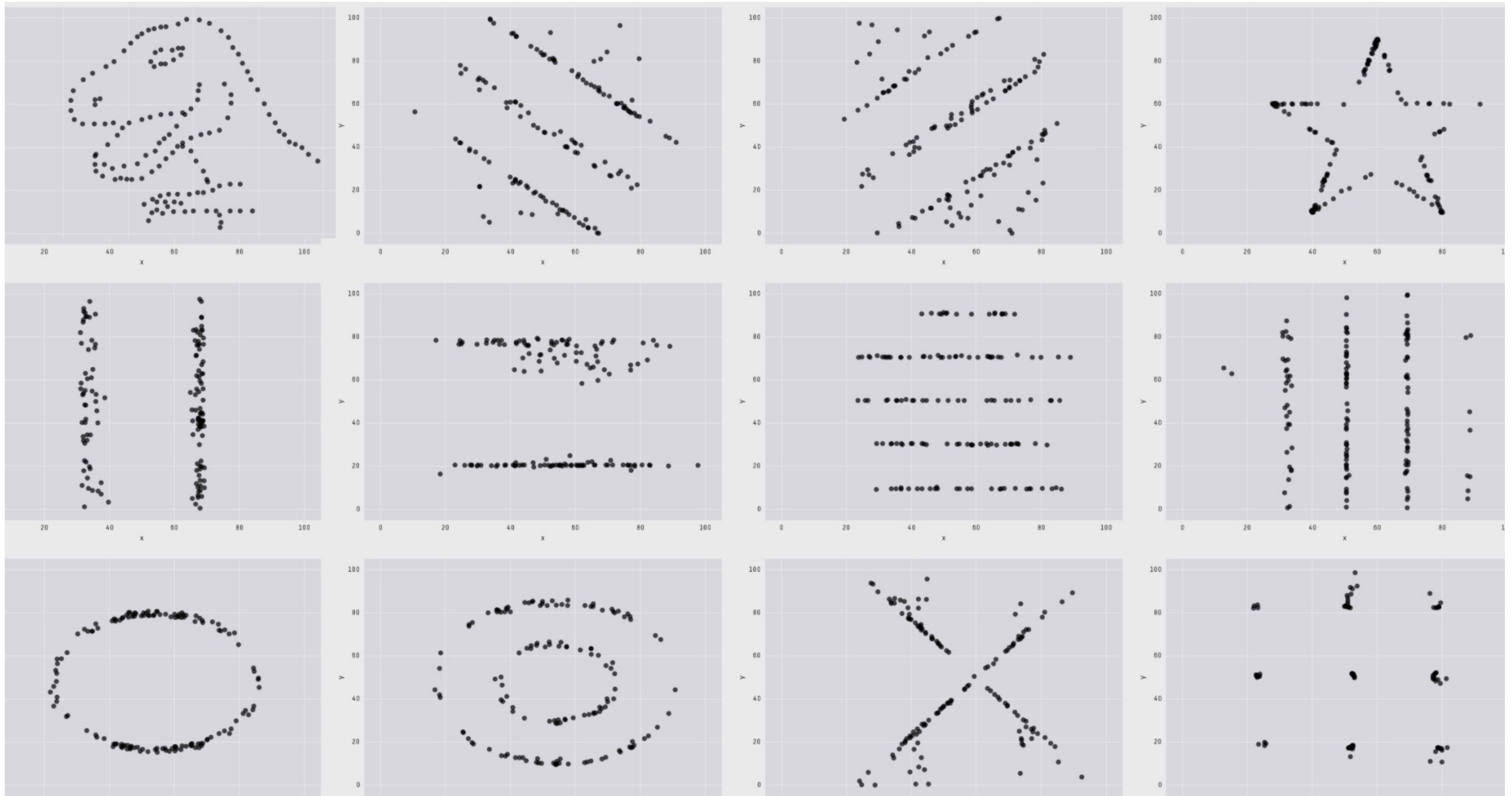
- ▶ 표본(sample)으로부터 모집단(population)의 특성을 설명하는 것
- ▶ 수학적 설명을 위해서 오차범위, 신뢰구간, 가설, 검증 등을 필요로 한다



동일한 평균, 표준편차, 상관계수



동일한 평균, 표준편차, 상관계수

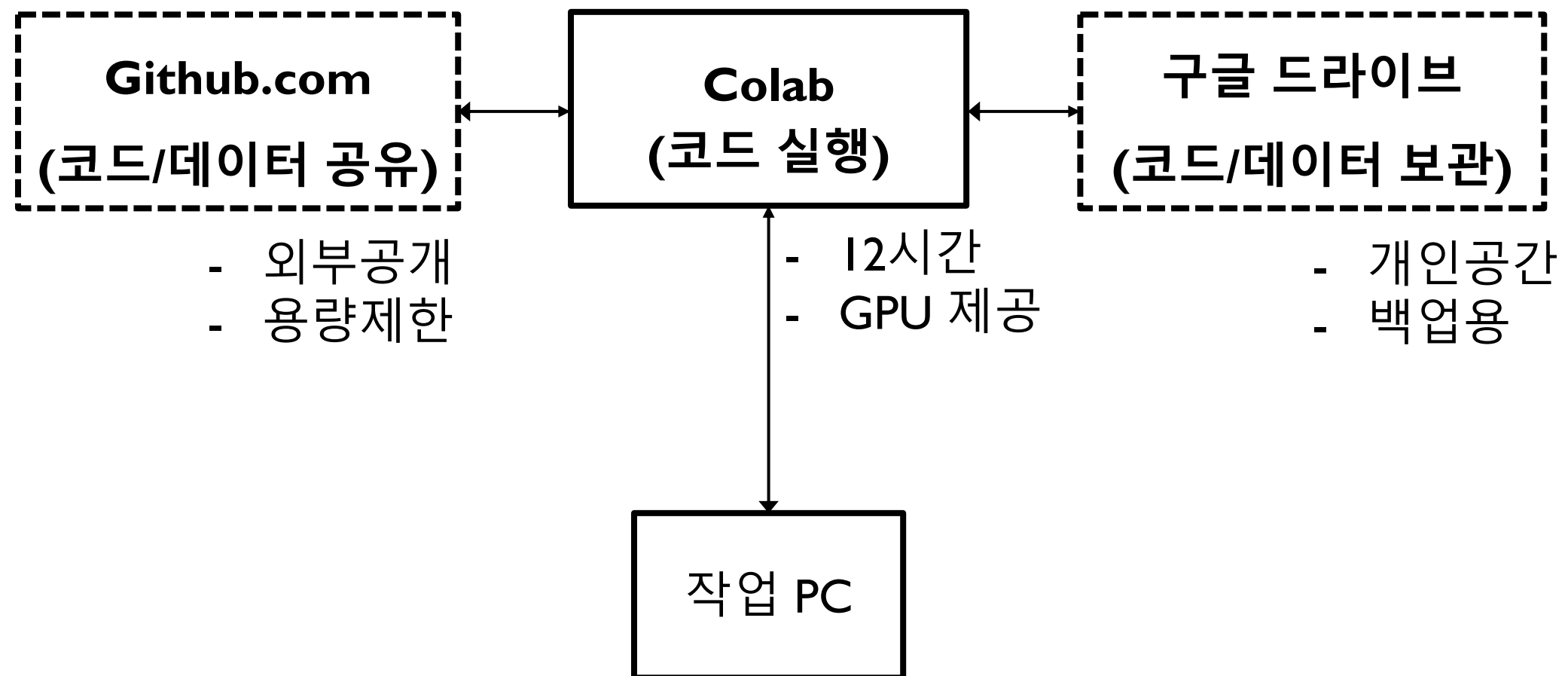


파이썬 핵심

파이썬 실행 환경

- ▶ **쥬피터 노트북 (Jupyter notebook)**
 - ▶ PC에서는 Anaconda 설치
 - ▶ 구글 코랩 사용 colab
 - ▶ Chrome으로 접속
 - ▶ 구글 계정 가입(gmail) – 구글 드라이브 사용
 - ▶ github.com 가입
- ▶ **쥬피터 노트북 특징**
 - ▶ 모든 작업을 셀 단위로 수행
 - ▶ 코드 셀 – 프로그램 영역
 - ▶ 마크다운 셀 – 문서 영역

코랩 실행 환경



주피터 팁

- ▶ 각 셀은 프로그램 코드 또는 문서(마크업 문서 지원) 둘 중 하나로 사용된다.
 - ▶ 셀을 실행하려면 Shift + Enter 를 입력한다 (셀 실행후 커서가 다음 셀로 이동)
 - ▶ 셀을 Ctrl + Enter로 실행하면 셀 실행 후에 현재의 셀에 남아 있다.
- ▶ colab에서 셀에 대한 명령어 (ctrl + m + ...)
 - h : help
 - a : 위에 셀 추가 (above)
 - b : 아래에 셀 추가 (below)
 - d : 셀 삭제 (delete)
 - m : 문서 모드로 전환 (mark up)
 - y : code 모드로 전환
 - o : 출력 보이기/안보이기 (output)
 - '': 셀 둘로 나누기
 - 'shift' 두 개 이상의 셀 선택 후에 (ctrl + 클릭으로) : 셀 합치기

파이썬 기초 문법

- ▶ 기본 변수
 - ▶ 정수, 소수, 문자열, 논리값을 표현한다
- ▶ 리스트
 - ▶ 임의의 데이터를 목록을 만들어 담을 수 있다. [] 로 표현된다
- ▶ 튜플
 - ▶ 상수화된 리스트이다. 값의 변경이 불가하다. () 로 표현된다
- ▶ 사전(dictionary)
 - ▶ 모든 항목이 항상 "키(key)"와 "값(value)" 짝으로 구성. {}로 표현된다
- ▶ 논리 흐름
 - ▶ 조건의 만족을 논리적으로 판단한다 if, else, elif 사용
 - ▶ 반복을 위하여 for, while을 사용한다
- ▶ 함수
 - ▶ 사용자가 임의의 기능을 정의할 수 있다. def를 사용

기호 사용

- ▶ `[]`
 - ▶ 리스트를 표현, `x = [1,2,3]`
 - ▶ 인덱싱을 표현, `z = x[0]`
- ▶ `()`
 - ▶ 튜플을 표현
 - ▶ 함수 호출시 인자를 입력, `x = function(a,b,c)`
- ▶ `{ }`
 - ▶ 사전(dictionary)을 표현
 - ▶ 집합(set) 을 표시

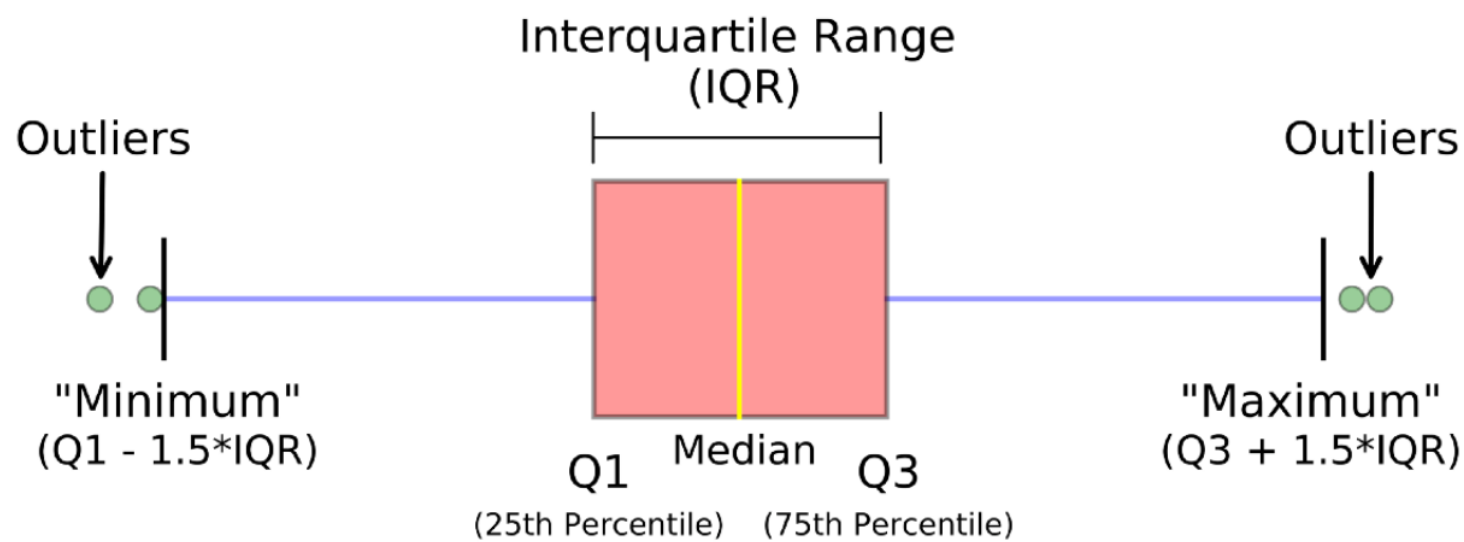
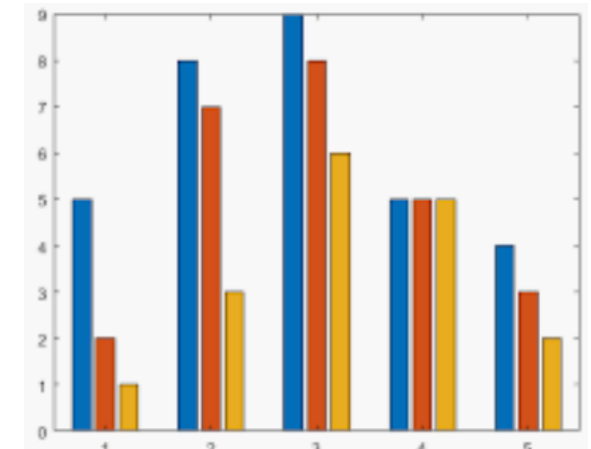
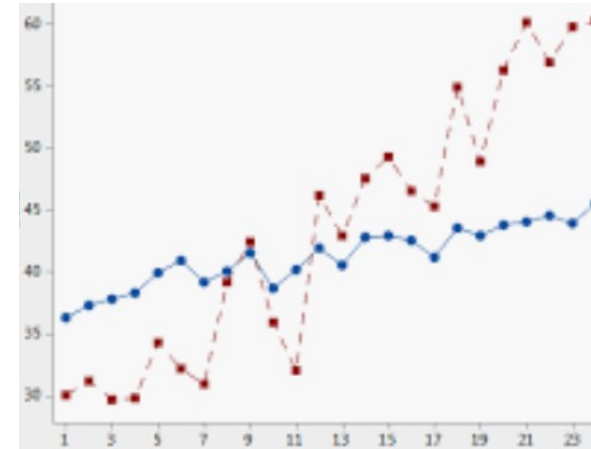
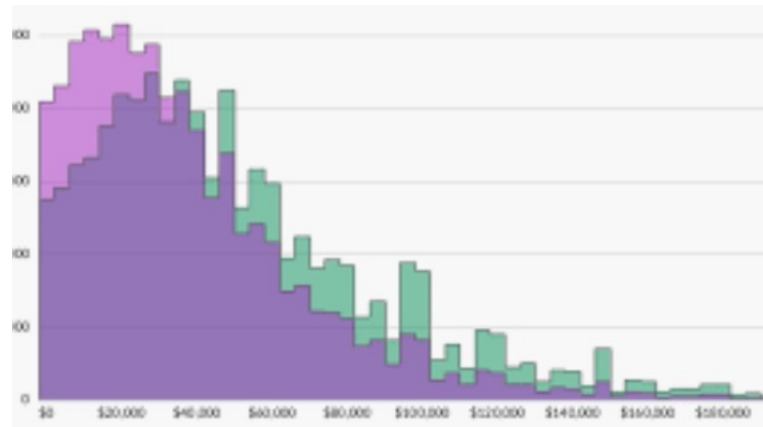
데이터 탐색과 전처리

데이터 탐색

- ▶ **탐색적 분석: exploratory data analysis (EDA)**
- ▶ 본격적인 데이터 분석이나 머신러닝을 수행하기에 앞서 데이터의 전체적인 특성을 살펴보는 것
 - ▶ 분포, 패턴, 동향 파악
- ▶ 수집한 데이터가 분석에 적절한지 알아보는 과정
 - ▶ 오류가 얼마나 많은지
 - ▶ 데이터가 통계적 가정(계층적 샘플링, stratified sampling)을 만족하고 있는지
- ▶ 시각화 기술을 사용

시각화 함수

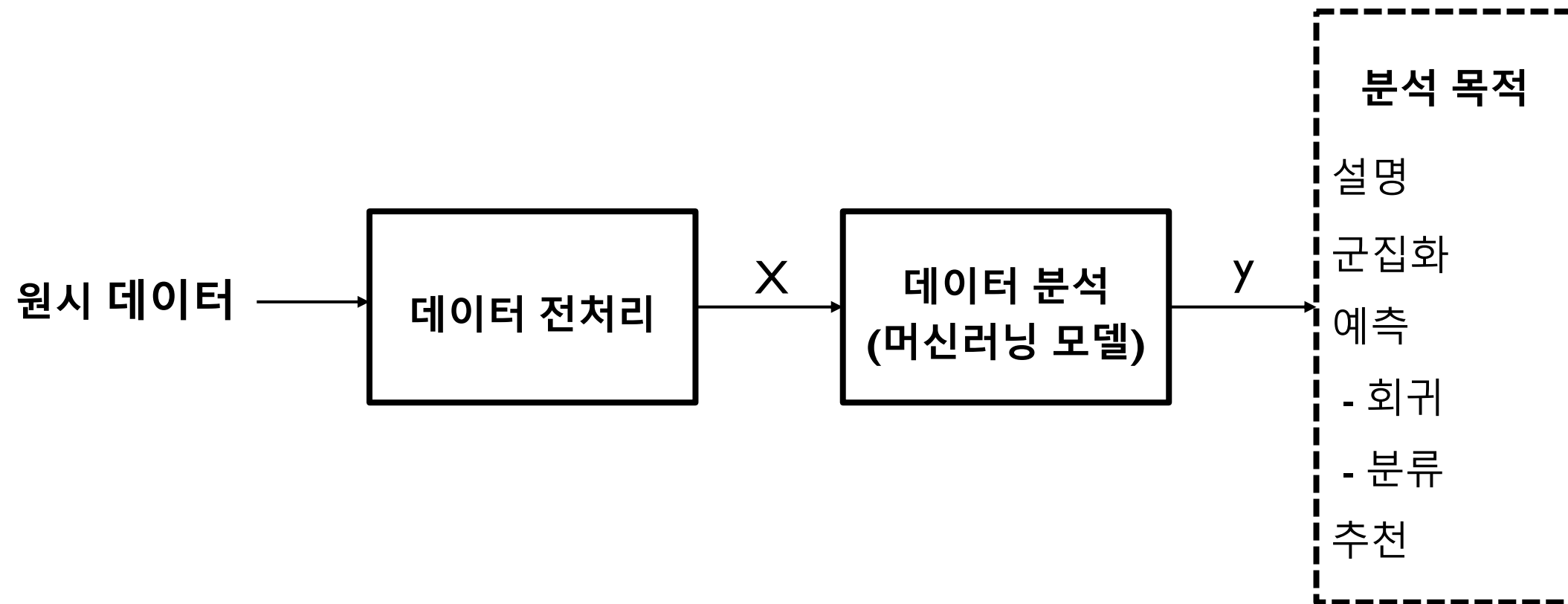
- ▶ 히스토그램
- ▶ 타임 플롯
- ▶ 바플롯
- ▶ 박스플롯
- ▶ 산포도



데이터 전처리

- ▶ 데이터를 분석에 사용할 때 성능이 더 좋게 나오도록 데이터를 수정하거나 형태를 변형하는 것
- ▶ 실제로 수집한 데이터를 머신러닝에 바로 사용하는 경우는 거의 없다
- ▶ 데이터가 비정형이라면 이를 정형 데이터로 바꾸어야 한다.
 - ▶ 이미지나 텍스트와 같은 비정형 데이터를 컴퓨터가 바로 다룰 수는 없다
 - ▶ 컴퓨터는 숫자로 된 데이터만 입력으로 받을 수 있다

데이터 전처리



데이터 전처리 유형

- ▶ 데이터 정제 (cleaning)
 - ▶ 빠진 값을 처리하거나 오류를 수정하는 것
- ▶ 이상치 처리
 - ▶ 비정상적으로 튀는 값(이상치)을 찾거나 제거하는 것
- ▶ 데이터 변형 (transformation)
 - ▶ 로그를 적용하거나, 역수를 취하거나, 카테고리 변수를 적용하는 것
- ▶ 스케일링 (scaling)
 - ▶ 여러 데이터의 특성 값의 범위를 동일한 조건으로 조정하는 것
- ▶ 특성 선택과 차원 축소
 - ▶ 특성 중 일부를 선택하거나 특성을 조합하여 새로운 특성을 만드는 것
 - ▶ 주성분 분석(PCA): 머신러닝에 사용할 특성의 수를 줄이는 것
 - ▶ Principal components analysis

데이터 정제

▶ 결측치 처리

- ▶ 빠진 값 즉, 결측치(missing value)를 처리하는 것
- ▶ 결측치를 처리하는 방법은 크게 세 가지가 있다
 - ▶ 결측치가 포함된 샘플 항목을 모두 버리는 방법
 - ▶ 결측치를 적절한 값으로 대체하는 방법
 - ▶ 분석 단계로 결측치 처리를 넘김

▶ 틀린값 처리

- ▶ 틀린값을 처리하는 방법도 결측치를 처리하는 방법과 같이 세가지이다.
 - ▶ 틀린 값이 포함된 항목을 모두 버리는 방법
 - ▶ 틀린 값을 적절한 값으로 대체
 - ▶ 분석 단계로 틀린 값 처리를 넘김

이상치 처리

- ▶ 이상치(outlier)란 값의 범위가 일반적인 범위를 벗어나 특별한 값을 갖는 것을 말한다
- ▶ 이상치 처리에는 두가지 목적이 있다
- ▶ 이상치 제거
 - ▶ 머신러닝 모델의 성능을 높이기 위해서 이상치를 학습 데이터에서 제거하는 경우 (애매한 사진 판독 등)
 - ▶ 데이터 전처리에 해당한다
- ▶ 이상치 검출
 - ▶ 이상치를 찾는 것 자체가 목적인 경우 (도난카드 사용, 기기 이상 등)
 - ▶ 머신러닝 모델을 사용한다

데이터 변환

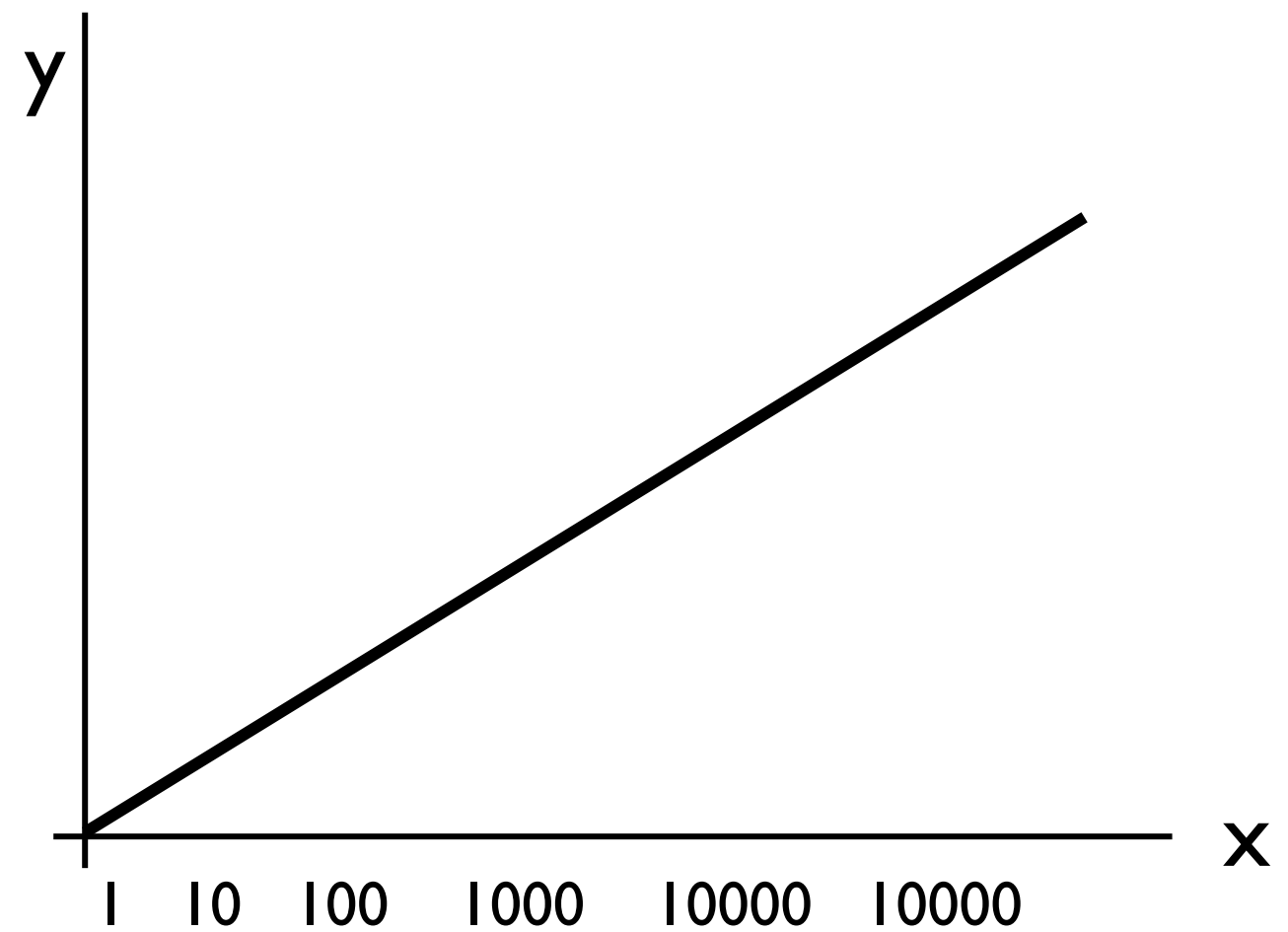
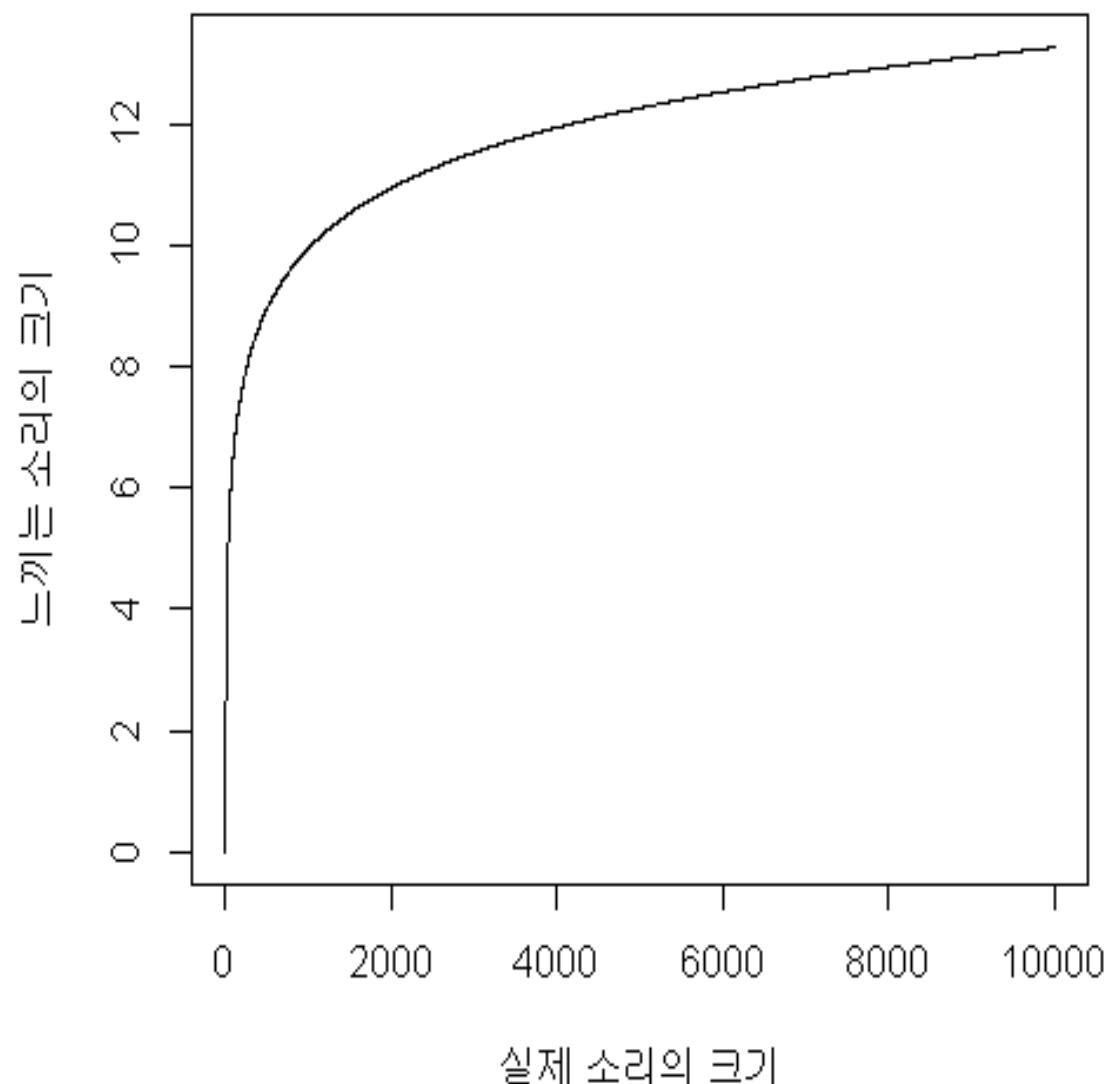
- ▶ 로그변환
 - ▶ 생물학적인 자극과 반응의 관계 (소리, 빛, 냄새, 압력, 금전 등의 자극)
 - ▶ 큰 변화를 약화시키는 효과
- ▶ 역수변환
 - ▶ 어떤 수치를 그대로 사용하지 않고 역수를 사용하는 것 (반비례)
- ▶ 승수변환
 - ▶ 자승값 등 지수승을 사용하는 경우
 - ▶ 큰 값의 변화를 더 강조하는 경우
- ▶ 범주형 변환
 - ▶ 수치 데이터를 범주형으로 변환하여 사용하는 것
 - ▶ 나이, 연간 소득, 성적 등
 - ▶ one hot encoding을 주로 사용한다(`get_dummies()` 사용)
 - ▶ (0,0,0,1,0,0,0,0)

로그 변환

- ▶ 체감형 수치를 선형적으로 표현할 때 사용
 - ▶ 사람이 자연적으로 느끼는 느낌의 양을 표현할 때 사용
 - ▶ 같은 자극을 느끼려면 현재 입력 양(크기)에 비례한 더 강한 자극이 필요하다는 것을 반영하는 것
- ▶ 이를 수학적으로 표현하면 로그 함수가 됨
 - ▶ 현재 보유한 양이 x 이고 이의 변화량, 즉 미분값이 $1/x$ 인 경우, 이러한 함수가 로그 함수임
 - ▶ 로그를 취한 이후의 값에 대해서 사람들이 변화량을 느끼는 것이 선형적으로 바뀌게 된다

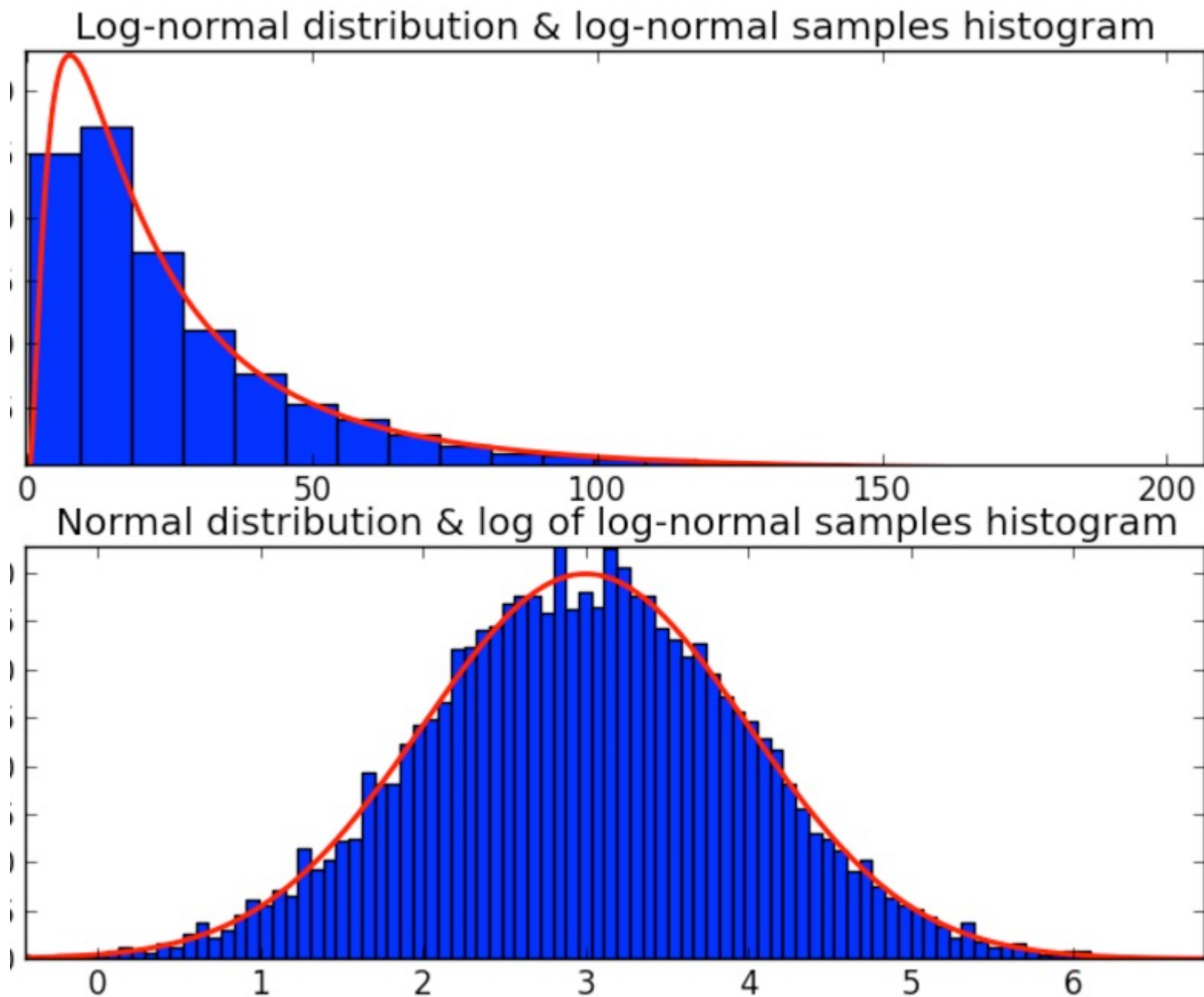
로그 변환

- ▶ 자극에 대한 반응이 현재 보유한 양 x 에 반비례하면 기울기가 $1/x$ 이 되고 이는 로그 함수에 해당한다
- ▶ 로그를 취한 값에 대해서는 자극-반응 관계가 선형적이 된다

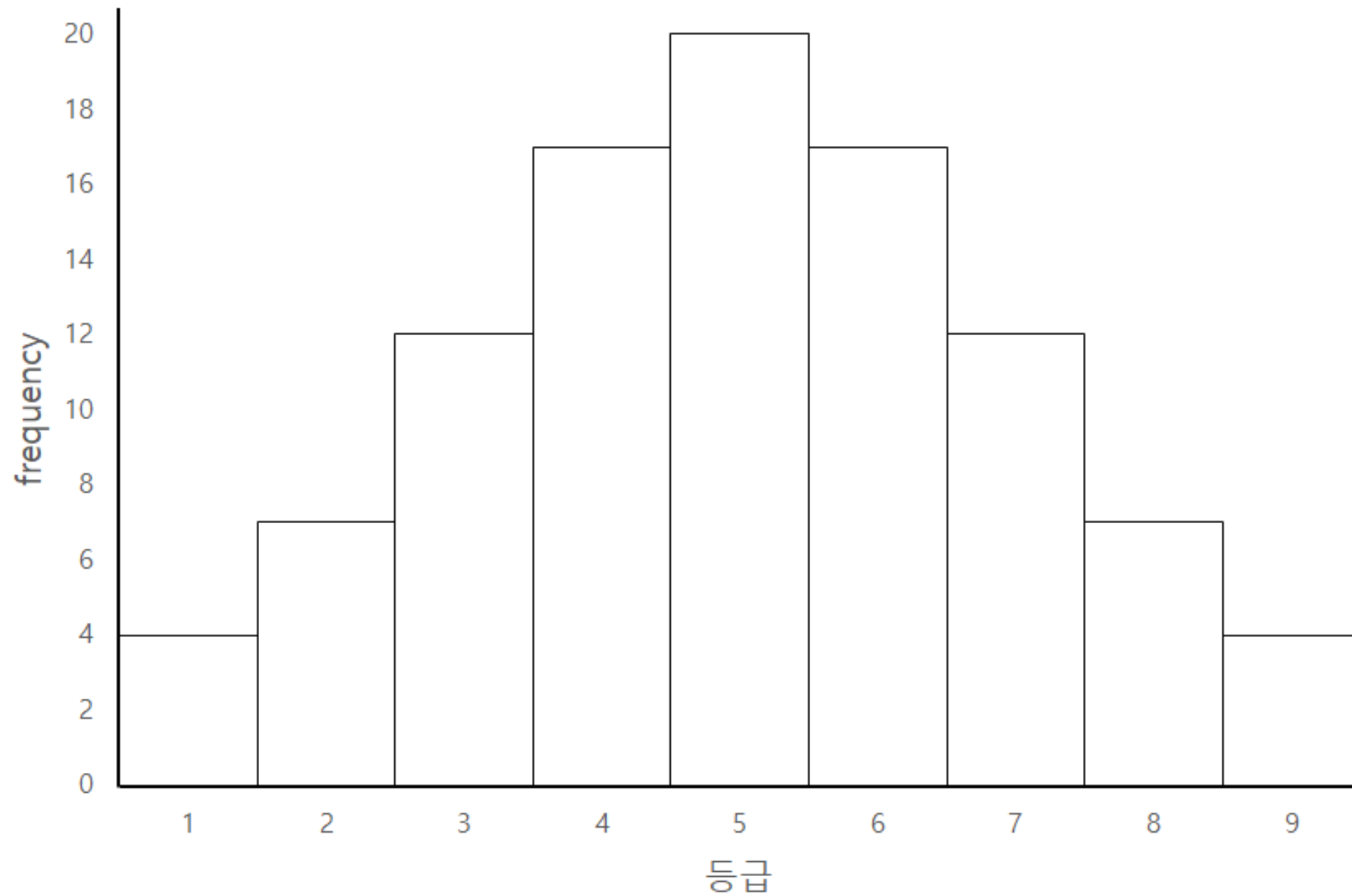


로그-정규 분포

- ▶ 로그를 취하고 나면 정규 분포를 이루는 경우



범주형 변환 예



역수 변환

- ▶ 역수를 취하면 입력-출력 관계가 선형적인 되는 경우
- ▶ (예) 자동차 마일리지
 - ▶ 한국: 연료 1L로 가는 거리 Km
 - ▶ 미국: 100km 주행하는데 필요한 연료 L
 - ▶ 이들은 역수 관계이다
- ▶ 속도와 시간의 관계도 역수이다
 - ▶ 재생속도와 재생시간 등
 - ▶ 어떤 값을 사용할지는 목적에 따라 다르다
- ▶ 분석 목적:
 - ▶ 같은 비용을 얼마나 멀리 갈 수 있는가?
 - ▶ 같은 거리를 여행하는데 비용이 얼마나 드는가?

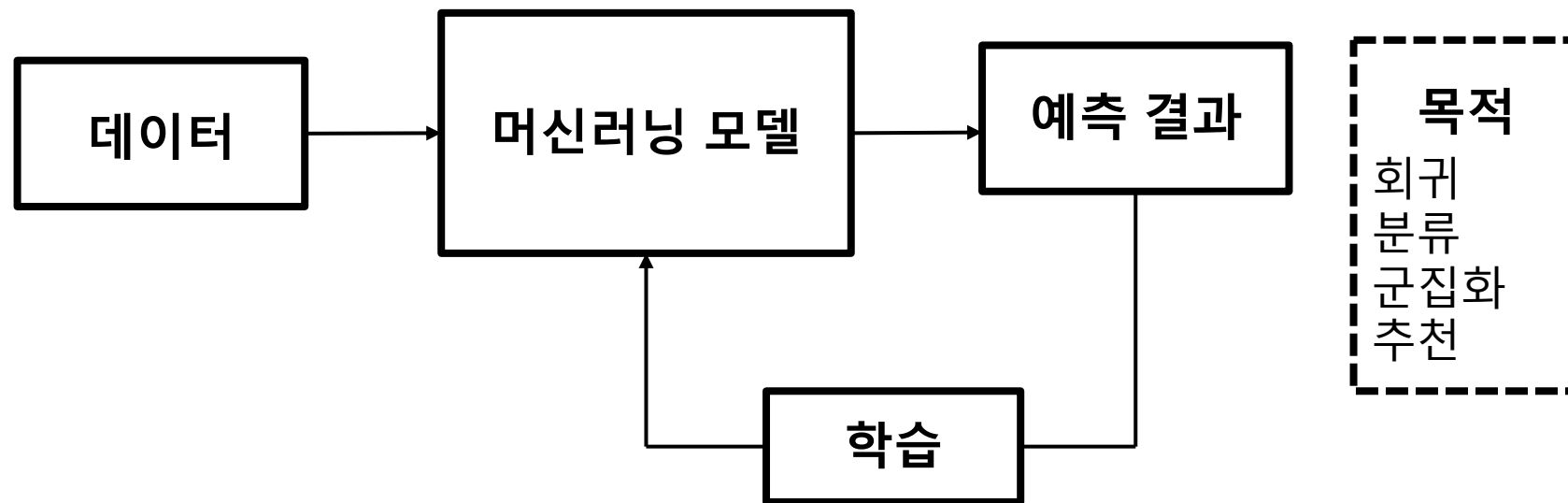
스케일링

- ▶ 원래 데이터가 갖는 값의 범위를 조정하여 모델 성능을 높이는 것
- ▶ 각 입력 특성(feature)의 중요도를 같게 시작하기 위해서 필요
- ▶ 최소-최대 스케일링
 - ▶ 주어진 값의 최소값을 0으로 최대값을 1로 조정하는 것
 - ▶ min-max 스케일링
 - ▶ `MinMaxScaler()` 함수 사용
- ▶ 표준 스케일링
 - ▶ 주어진 샘플의 평균이 0이, 표준편차가 1이 되도록 변환하는 것
 - ▶ z-score 정규화라고도 한다
 - ▶ (주의) 표준 스케일링을 하면 정규 분포를 갖게 되는 것은 아님

머신러닝 모델

머신러닝 모델

- ▶ 데이터를 학습하여 예측 (prediction) 능력이 점차 향상되는 소프트웨어



예측(Predictive) 모델

- ▶ 예측에는 회귀와 분류가 있다.
- ▶ 회귀는 수치를 예측하는 것을 말한다
 - ▶ 수요 예측, 생산 예측, 판매 예측, 재고 예측, 성능 예측 등
- ▶ 분류는 주어진 샘플이 어느 카테고리에 속할지를 예측하는 것
 - ▶ 메일이 스팸인지 아닌지를 구분하는 것
 - ▶ 사진 판독, 번호판 예측 등
 - ▶ 기기고장, 이상진단, 양불 판정, 질병 예측, 약효 예측 등

지도 및 비지도 학습

- ▶ 지도 학습(supervised learning)
 - ▶ 정답이 있고 이를 예측하는 학습
 - ▶ 정답을 목적변수 (target variable) 또는 레이블(label)이라고 부른다.
- ▶ 비지도 학습 (unsupervised learning)
 - ▶ 정답이 없이 데이터에 내포된 의미(insight)를 찾는 것
 - ▶ 탐색적 분석, 시각화, 연관분석, 클러스터링(군집화) 등
 - ▶ 주성분분석(PCA), 차원축소, t-SNE
 - ▶ 언어 모델 등에서 대량의 텍스트를 이용한 사전 학습 등
 - ▶ 표현학습 (representation learning, 임베딩 학습) 등
 - ▶ 생성 모델은 표현학습을 통해서 이미지, 텍스트 등을 표현하는 방법을 미리 사전학습해야 한다

추천(Prescriptive)

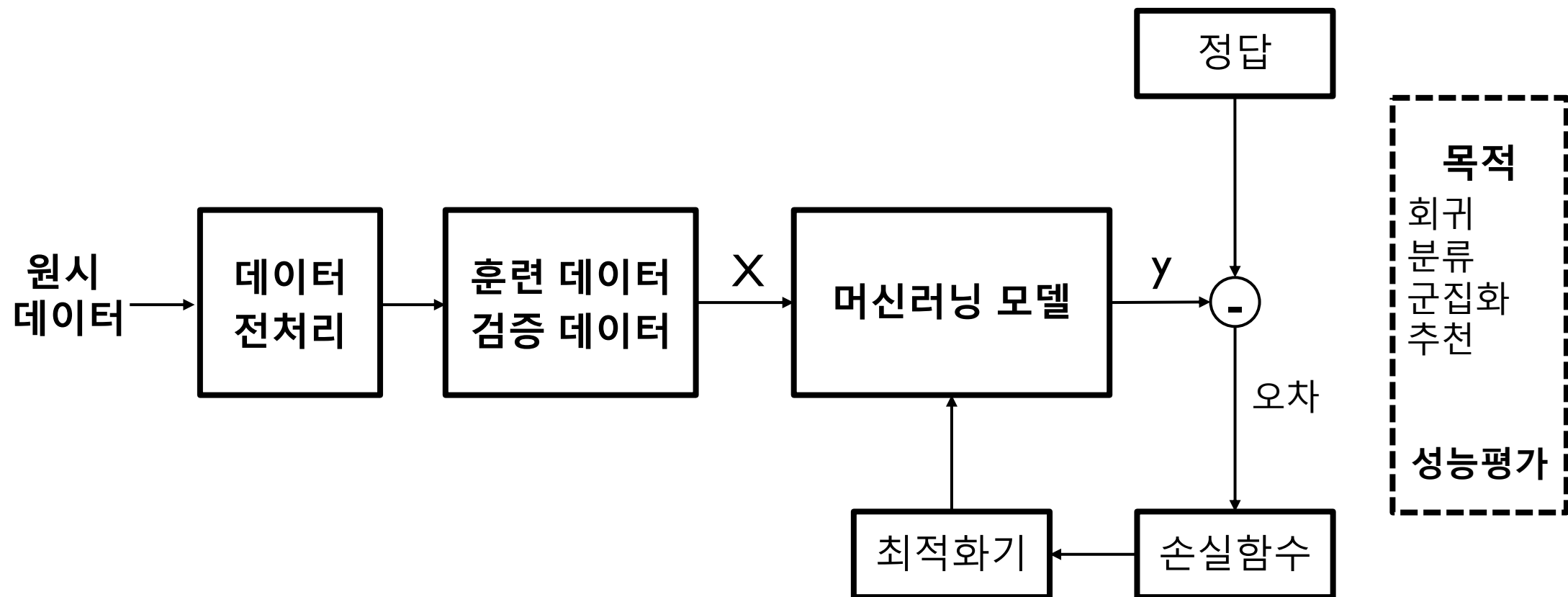
- ▶ 최종적으로 의사결정을 돕는 것
 - ▶ 단순히 정보(인사이트)를 주는 것을 넘어서 최적의 추천
 - ▶ 예측 모델을 종합적으로 활용하는 것 (어떤 사람이 어떤 음악을 좋아하는지 예측을 해야 추천이 가능하게 됨)
- ▶ 예
 - ▶ 약의 처방, 네비게이터, 검색엔진, 상품 추천
 - ▶ 자율차의 운행
 - ▶ 알파고와 같은 게임 플레이어

머신러닝 모델

- ▶ 머신러닝에서는 모델을 만드는 것이 핵심이다.
 - ▶ 스팸 메일을 찾아내는 모델
 - ▶ 수익 예측 모델
 - ▶ 도난 카드 사용 검출 모델
- ▶ 수식은 가장 명확한 모델이다
 - ▶ 그러나 현실의 복잡한 현상은 수식으로 모델링하기 어렵다
 - ▶ 데이터 기반의 모델이 필요하다 – 머신러닝 모델
- ▶ 모델은 구조와 파라미터로 구성된다.
 - ▶ 모델 구조: 모델의 동작 방식 (선형모델, 트리모델, 신경망 모델 등)
 - ▶ 모델 파라미터: 모델이 잘 동작하도록 학습된 가중치(weight)
 - ▶ $y = ax + b$
 - ▶ 위의 입출력 관계는 **모델 구조**이고, a, b 는 **모델 파라미터**임

머신러닝 모델 구현

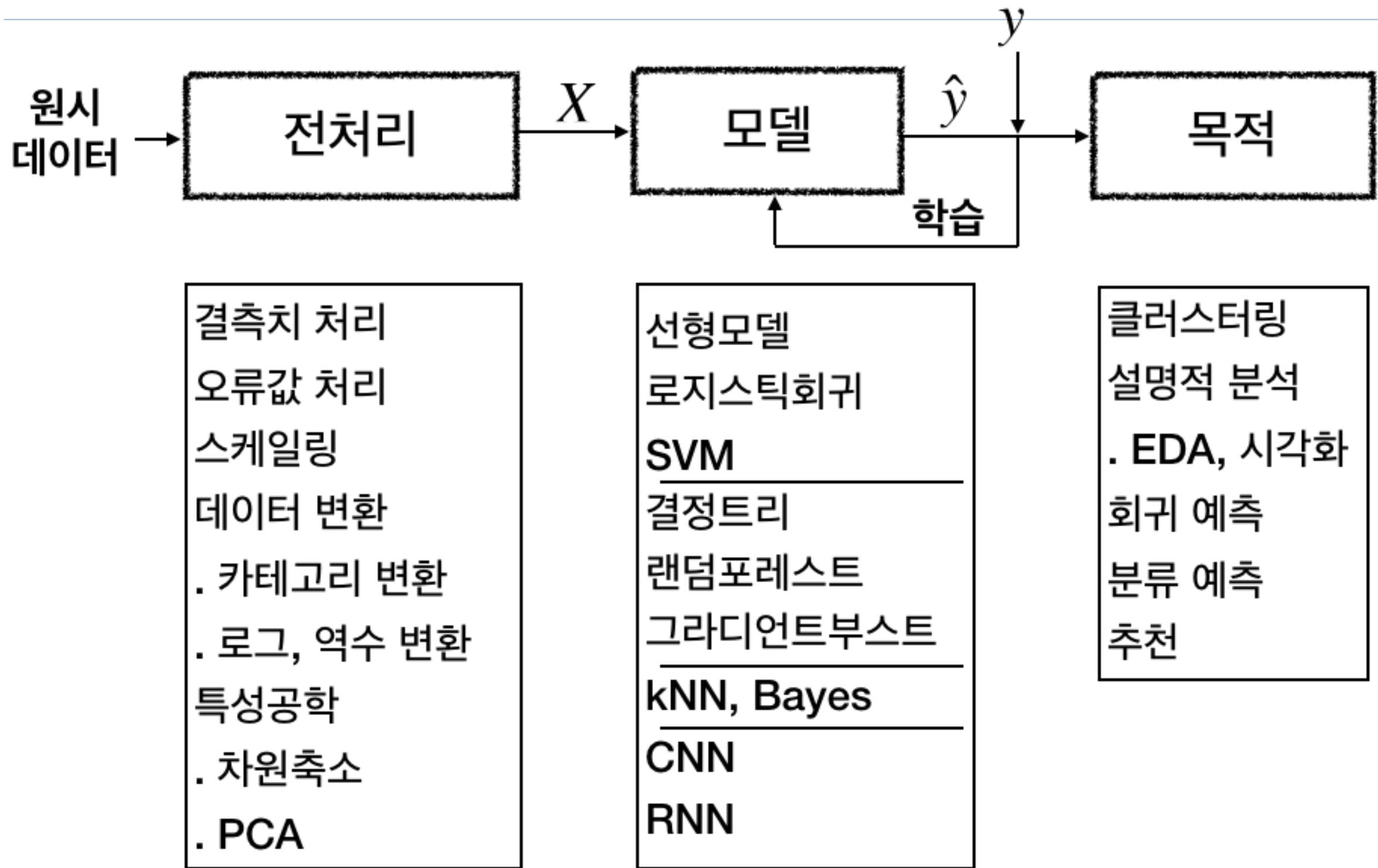
- ▶ 데이터 전처리
 - ▶ 모델의 성능을 높이기 위해서 결측치 처리, 이상치 처리, 스케일링, 카테고리 인코딩 등을 수행하는 것
- ▶ 훈련/검증 데이터
 - ▶ 모델을 만드는 데는 훈련 데이터를, 성능을 검증하는 데는 검증 데이터를 사용한다



훈련과 검증

- ▶ 모델이 데이터를 이용하여 학습하는 과정을 훈련 (training)이라고 한다.
- ▶ 모델 파라미터 값 (예를 들어 선형 회귀에서 가중치의 값)의 초기값은 랜덤한 값을 준다
- ▶ 모델을 훈련시킨 후에는 모델이 제대로 동작하는지 검증하기 위해서 검증데이터를 사용한다

머신러닝 프로세스



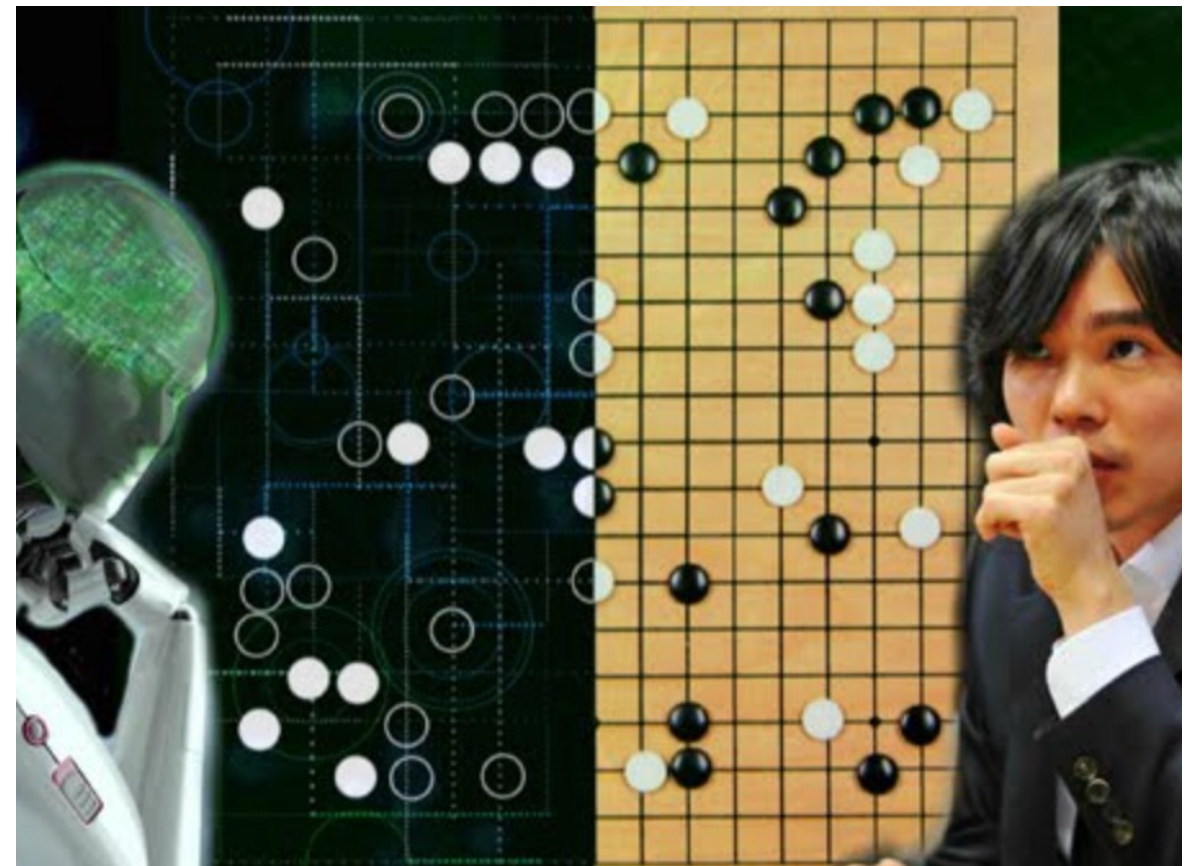
머신러닝 모델 비교

머신러닝 모델 비교

머신러닝 유형		알고리즘	특징
지도학습	선형 계열	선형 모델, SVM 로지스틱회귀	곱셈과 덧셈으로 점수를 구하고 이를 이용하여 회귀와 분류 예측
	신경망	MLP, CNN, RNN, Transformer Graph Network	매트릭스 연산을 기반으로 점수를 계산하며 활성화 함수 도입
	트리 계열	결정 트리, 랜덤포레스트, 그라디언트부스팅	True/False 선택을 반복하여 회귀와 분류 예측 수행. 스케일링이 필요없다
	기타	kNN, 베이지	특성 공간상의 거리를 기준, 또는 조건부 확률을 기준으로 예측
비지도학습	클러스터링	k-means, DBSCAN	특성 공간상 거리 또는 유사도를 기준으로 샘플을 그룹핑
	데이터 변환	스케일링, 로그변환, 카테고리 인코딩	모델의 성능을 높이 위한 데이터 전처리
	차원 축소	PCA, t-SNE	계산량을 줄이고 모델 성능을 향상, 또는 시각화를 위한 차원 축소

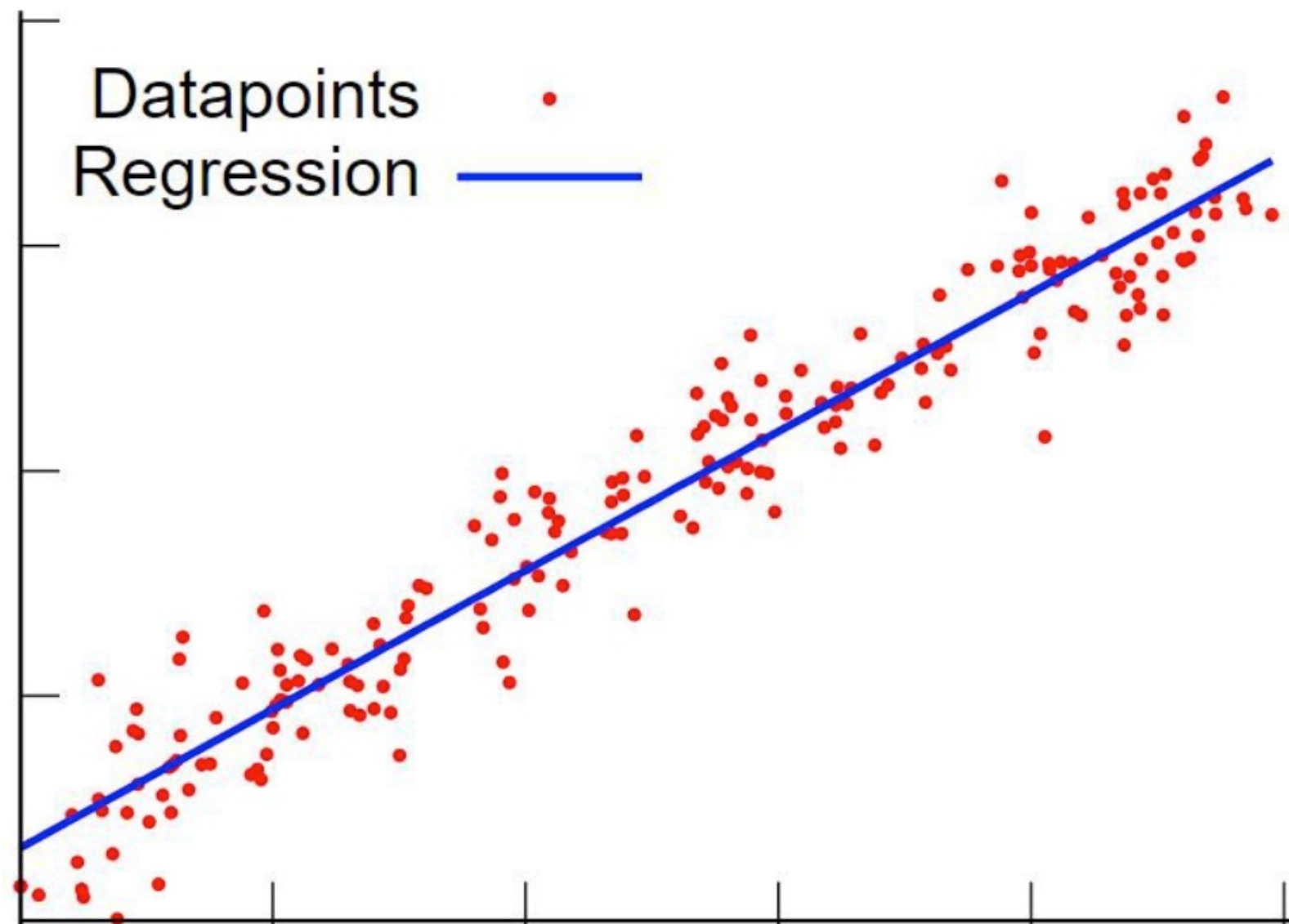
강화 학습

- ▶ Reinforcement Learning
- ▶ 샘플 입력마다 정답(label)이 있지만 최종적으로 달성해야 할 목표를 알려주고 보상을 통해 학습 방법을 학습시킨다
- ▶ 게임과 같이 룰이 있고 시뮬레이션이 가능한 경우에만 동작



선형 회귀

- ▶ 선형 회귀(regression) $y = wX + b$



선형 회귀

- ▶ 입력(x)와 출력(y) 변수간의 관계를 대표하는 회귀직선

$$y = ax + b$$

- ▶ 입력이 여러 특성값으로 구성되어 있으면 다중 선형 회귀 (multiple linear regression)라고 한다.

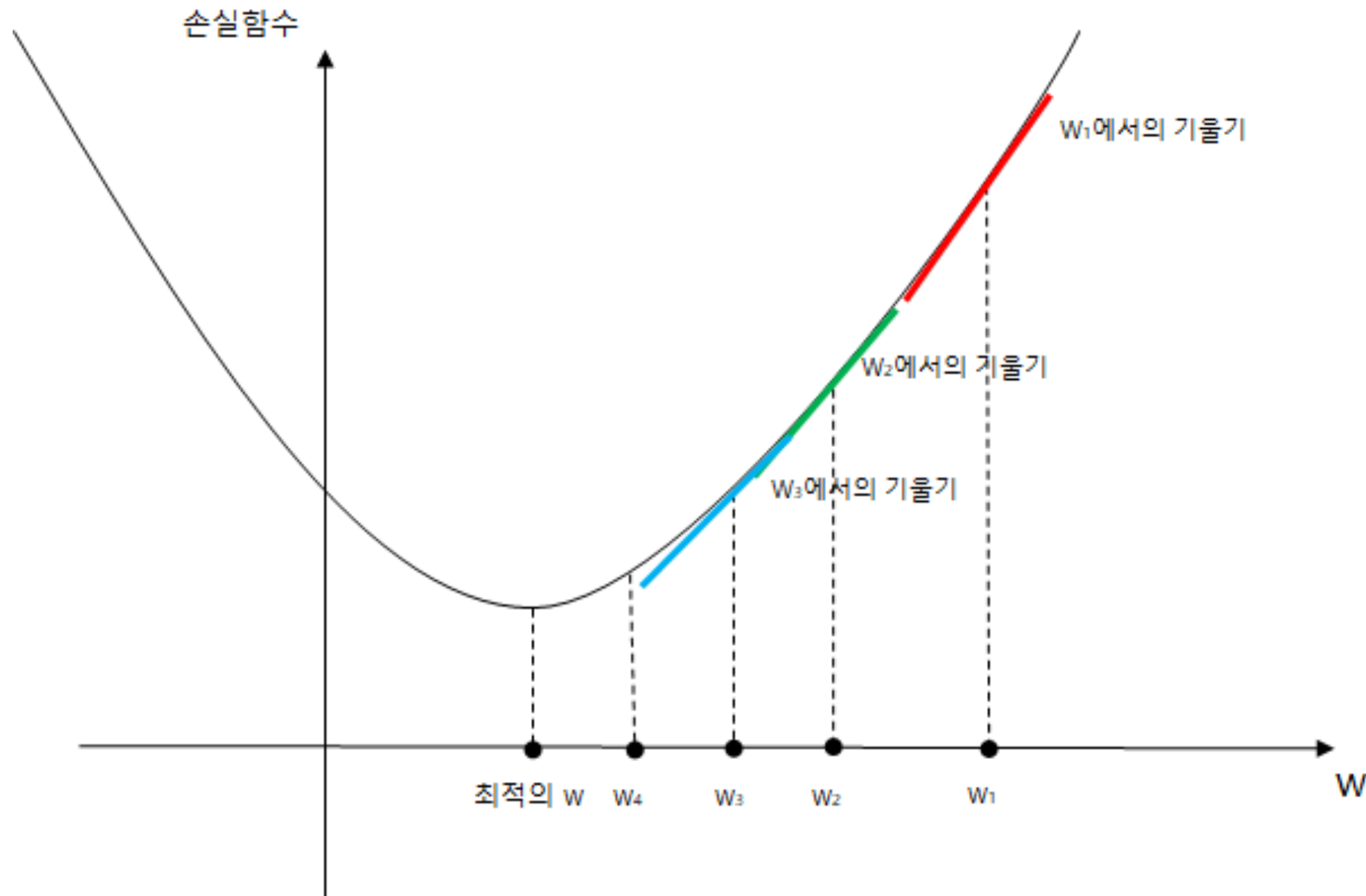
$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_p x_p$$

모델 최적화

- ▶ 모델 최적화(optimizer)란 학습을 통하여 파라미터를 최적의 값으로 수렴시키는 것을 말한다
- ▶ 여기서 최적이란 **손실함수**를 가능한 줄이는 것을 말한다
- ▶ 최적화 알고리즘으로는 경사하강법(GD: Gradient Descent)이 기본적으로 사용된다

경사하강법

- ▶ 경사하강법은 손실함수 공간에서 기울기(gradient, 미분값)에 비례하여, 반대방향으로 계수를 조정하는 방식을 말한다.



경사하강법

- ▶ 이를 수식으로 표현하며 다음과 같다.

$$W_i = W_{i-1} - \eta \text{Grad}(i)$$

- ▶ 위에서 $\text{Grad}(i)$ 는 시점 i 에서 손실함수의 기울기이고, η 는 학습 속도를 조절하는 학습률(learning rate)이다.
- ▶ 경사하강법을 적용하려면 특성 들을 모두 스케일링해야 한다
- ▶ 경사하강법은 크게 미니 배치(Batch)와 통계적(Stochastic) GD (SGD) 두 가지가 있다

손실함수와 성능지표

손실함수와 성능지표

- ▶ **손실함수**(loss function)는 모델을 훈련시킬 때 기준으로 삼기 위해서이다.
 - ▶ 모델은 손실함수를 최소화 하는 방향으로 학습한다
- ▶ **모델의 성능지표**는 이렇게 만든 모델이 궁극적으로 얼마나 잘 동작하는지를 평가하는 척도이다

회귀모델의 손실함수

- ▶ 회귀분석에서는 오차 자승의 합의 평균치(MSE: mean square error)를 손실함수로 주로 사용한다

$$MSE = \sum_{k=1}^N (y - \hat{y})^2$$

- ▶ N: 배치 크기
- ▶ 배치 크기, 학습률과 같은 설정 변수를 하이퍼파라미터라고 한다
 - ▶ **하이퍼파라미터**는 사람이 선택하는 값이며, 머신러닝 모델이 최적화해야 하는 계수는 "**파라미터**"이다

기타 회귀 손실함수

- ▶ 회귀 모델에서 MSE 외에 다음과 같은 손실함수를 사용할 수 있다.
 - ▶ MAE – Mean Absolute Error
 - ▶ MSLE– Mean Absolute Log Error
- ▶ 사용용도
 - ▶ MSE는 큰 오차에 페널티를 많이 부과하는 것
 - ▶ MAE는 모든 크기의 오차를 동일하게 다루는 것
 - ▶ MALE는 작은 크기의 오차에 더 민감하게 반응하는 것

회귀 성능 평가 지표

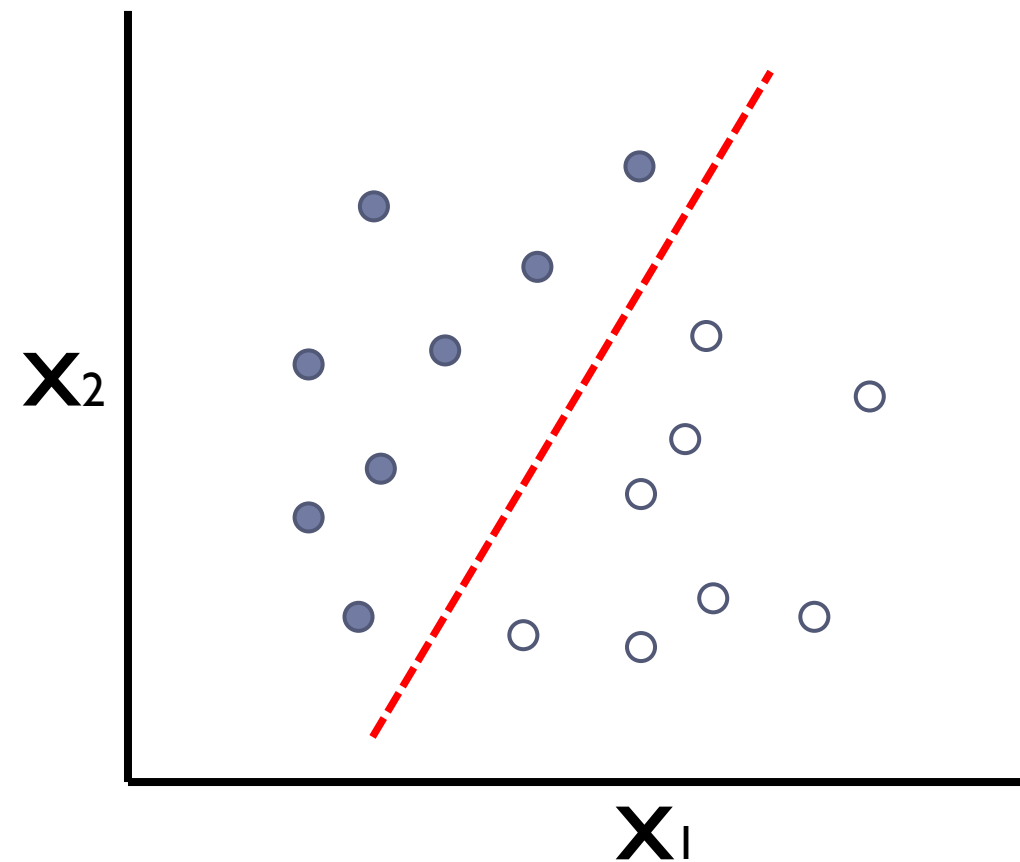
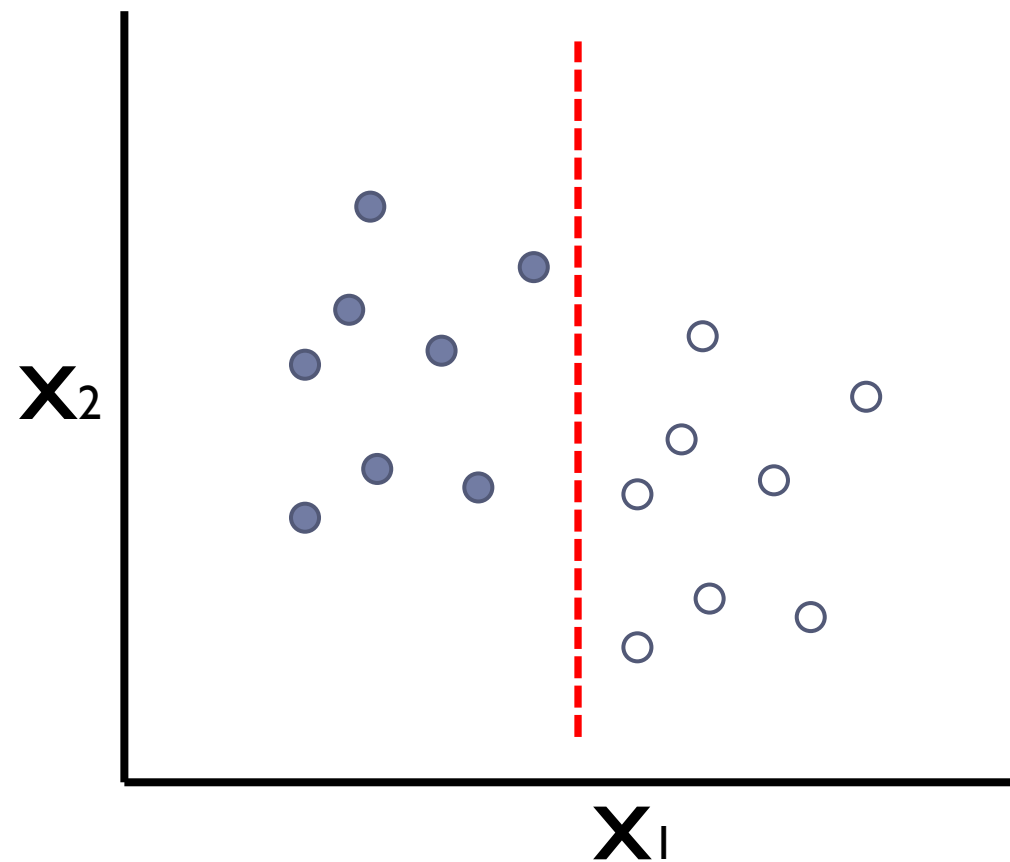
- ▶ 회귀모델의 성능 평가에는 R-squared가 사용된다.
- ▶ R-Squared,

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

- ▶ 위에서 y_i 는 실제 값, $\hat{y}_i$ 는 예측한 값, $\bar{y}$ 는 샘플의 평균 값을 나타낸다
- ▶ 완벽한 회귀 예측 모델은 R^2 값이 1 이 된다

선형 분류

- 하나의 변수만 사용하는 경우 선형분류 결정 경계: $x_1 > b$
- 두 개의 변수를 사용하는 경우 선형분류 결정 경계: $a_1 x_1 + a_2 x_2 + b > 0$



분류의 성능지표

- ▶ 분류에서는 정확도(accuracy)를 기본으로 측정하나 필요에 따라 다음과 같은 값을 추가로 측정해야 한다
- ▶ 컨퓨전 매트릭스를 사용한다
 - ▶ 정밀도 (precision)
 - ▶ 재현률 (recall)
 - ▶ F-1 점수
 - ▶ ROC
 - ▶ AUC

혼돈 매트릭스

- ▶ 혼돈 매트릭스(confusion matrix)란 분류의 결과가 잘 맞았는지를 평가하는 채점표와 같다
- ▶ 이진 분류의 경우 예측한 내용과 실제 타겟 값이 맞는지를 따져보면 총 네 가지 경우의 수가 존재한다.

실제 \ 예측	p로 예측	n으로 예측
실제로는 p	True positive (TP)	False negative (FN)
실제로는 n	False positive (FP)	True negative (TN)

```
print(metrics.confusion_matrix(y_test, y_test_pred))
```

```
[[6 1]
```

```
 [ 2 2]]
```

분류 성능 지표

- ▶ 정확도, 정밀도(precision), 재현률(recall), F1 점수(F1-score) 등을 보기 위해서 `classification_report()`를 사용한다.

	precision	recall	f1-score	support
0	0.75	0.86	0.80	7
1	0.95	0.91	0.93	23
avg / total	0.91	0.90	0.90	30

정확도와 정밀도

▶ 정확도(accuracy)

- ▶ 정확도란 positive 및 negative 두가지에 대해서 전체적으로 정확히 예측한 비율을 말한다.

$$\begin{aligned}\text{정확도} &= (\text{positive 또는 negative를 맞춘 수}) / (\text{전체 샘플 수 } N) \\ &= (TP+TF) / (TP + FP + FN + TN)\end{aligned}$$

▶ 정밀도(precision)

- ▶ 정밀도란 머신러닝 모델이 positive라고 예측한 것 중에 positive인 비율을 말한다.

$$\begin{aligned}\text{정밀도} &= (\text{positive를 맞춘 수}) / (\text{positive라고 예측한 수}) \\ &= TP/(TP+FP)\end{aligned}$$

재현율과 F1 점수

▶ 재현율(recall)

- ▶ 재현율은 전체 positive 샘플 중에 모델이 positive라고 찾아낸 비율을 말한다. 재현율은 관심 있는 대상을 얼마나 찾아내는지를 나타낸다.

$$\begin{aligned}\text{재현율} &= (\text{positive를 맞춘 수}) / (\text{positive 전체 수}) \\ &= TP / (TP + FN)\end{aligned}$$

▶ F1 지표

- ▶ 재현율과 정밀도 두 성능평가 지표를 동시에 측정하는 방법
- ▶ 정밀도와 재현율의 조화평균(harmonic average)으로 정의한다.

$$F1 = 2 \times \text{precision} \times \text{recall} / (\text{precision} + \text{recall})$$

조화 평균

- ▶ 산술 평균과 달리 점수들의 차이가 크면 산술평균보다 더 나은 점수를 얻는 방식
 - ▶ 점수들이 조화롭지 않으면(비슷하지 않으면) 페널티를 더 주는 방식

$$\text{조화 평균: } \frac{1}{c} = \frac{\left(\frac{1}{a} + \frac{1}{b}\right)}{2}$$

$$c = \frac{2ab}{a+b}$$

분류의 손실함수

- ▶ 분류에서 정확도(accuracy)를 손실함수로 사용할 수 없다
- ▶ **카테고리 분포 불균형**시 문제
 - ▶ 정확도를 손실함수로 사용하면 발생 빈도가 낮은 샘플은 잘 찾아내지 못하게 된다 (분포가 큰 샘플만 찾으려고 한다)
 - ▶ 이를 보완하기 위해서 분류 모델에서는 크로스 엔트로피(cross entropy)를 손실함수로 사용한다
 - ▶ (참고) 손실함수란 모델이 학습할 때 줄이는 대상이 되는 값이다

손실함수와 성능지표

- ▶ 대표적인 손실함수와 성능 평가 지표
 - ▶ 분류에서는 정확도 외에 성능 지표로 정밀도(precision), 재현률(recall), F1-지수 등이 추가로 사용된다.

	손실함수	성능평가지표
정의	손실함수를 줄이는 방향으로 모델이 학습을 함	성능을 높이는 것이 머신러닝을 사용하는 목적임
회귀 모델의 대표적인 값	MSE (오차 자승의 편균치)	R^2
분류 모델의 대표적인 값	크로스 엔트로피	정확도, 정밀도, 재현률, F1점수

과대적합

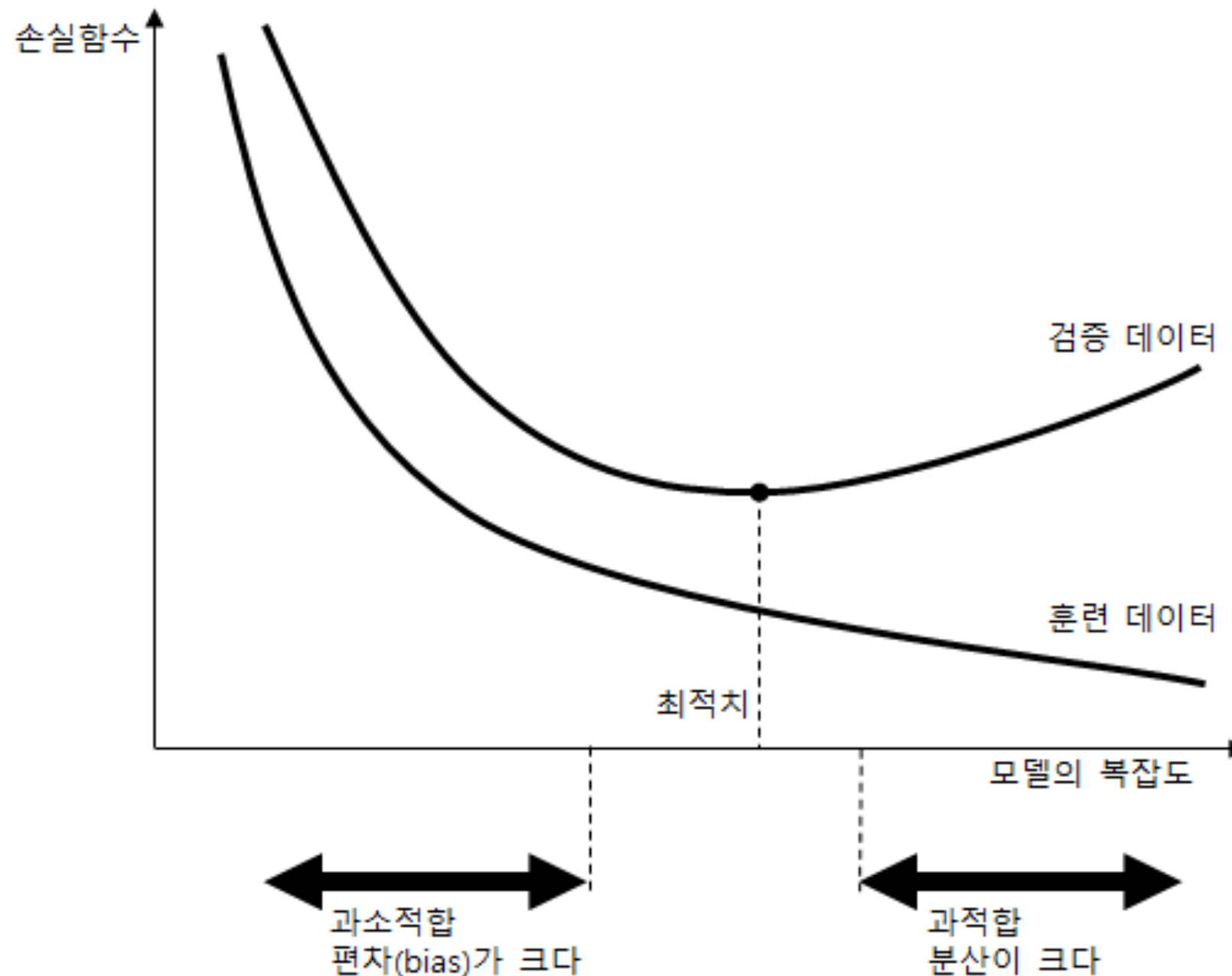
- ▶ 모델이 훈련 데이터에 대해서만 잘 동작하도록 만들어져서 새로운 데이터에 대해서는 오히려 잘 동작하지 못하는 경우를 과대적합(over fitting)되었다고 한다
- ▶ 과대적합은 주어진 훈련 데이터를 너무 세밀하게 학습에 반영하여 발생하는 현상이다
- ▶ 머신러닝에서는 과대적합을 피해서 일반적으로 잘 동작하게 모델을 만드는 것이 매우 중요하다
 - ▶ 이를 모델의 일반화(generalization)라고 한다
- ▶ 일반화 방법
 - ▶ 모델을 단순하게 만든다(이를 규제화(regularization)라고도 한다)
 - ▶ 더 많은 훈련 데이터를 확보한다

과소적합

- ▶ 과대적합과 반대로 모델이 너무 간단하거나 학습이 덜 되어 성능이 미흡한 경우를 과소적합(under fitting)이라고 한다
- ▶ 과소적합을 피하려면
 - ▶ 좀 더 상세한 모델 구조를 사용한다
 - ▶ 학습을 더 오래 수행한다 (같은 데이터를 반복하여 학습에 사용)
- ▶ 머신러닝에서는 과대적합과 과소적합을 모두 피해야 한다

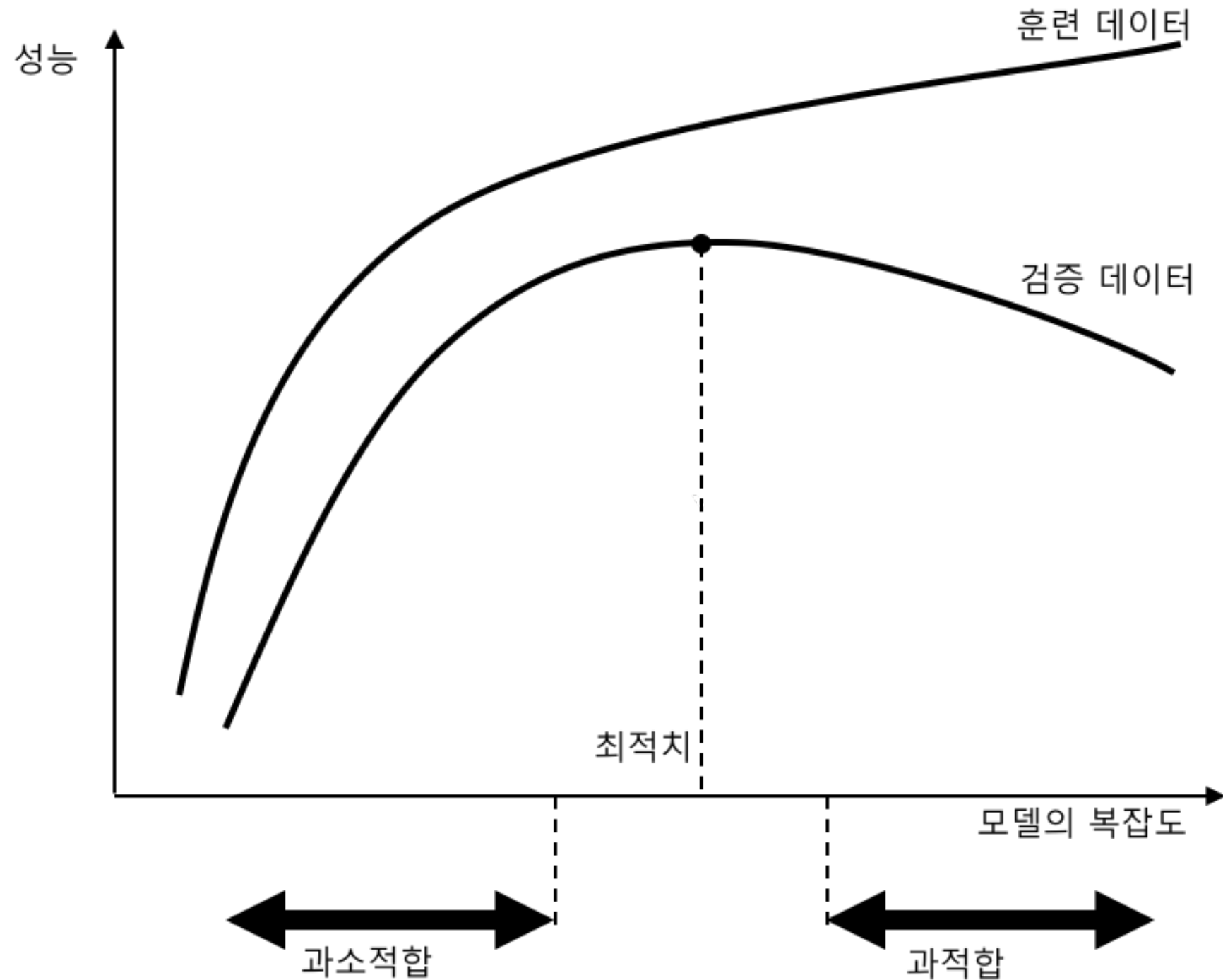
과대적합 검증

- ▶ 훈련 데이터에 대한 성능과 검증 데이터에 대한 손실함수를 관찰한다

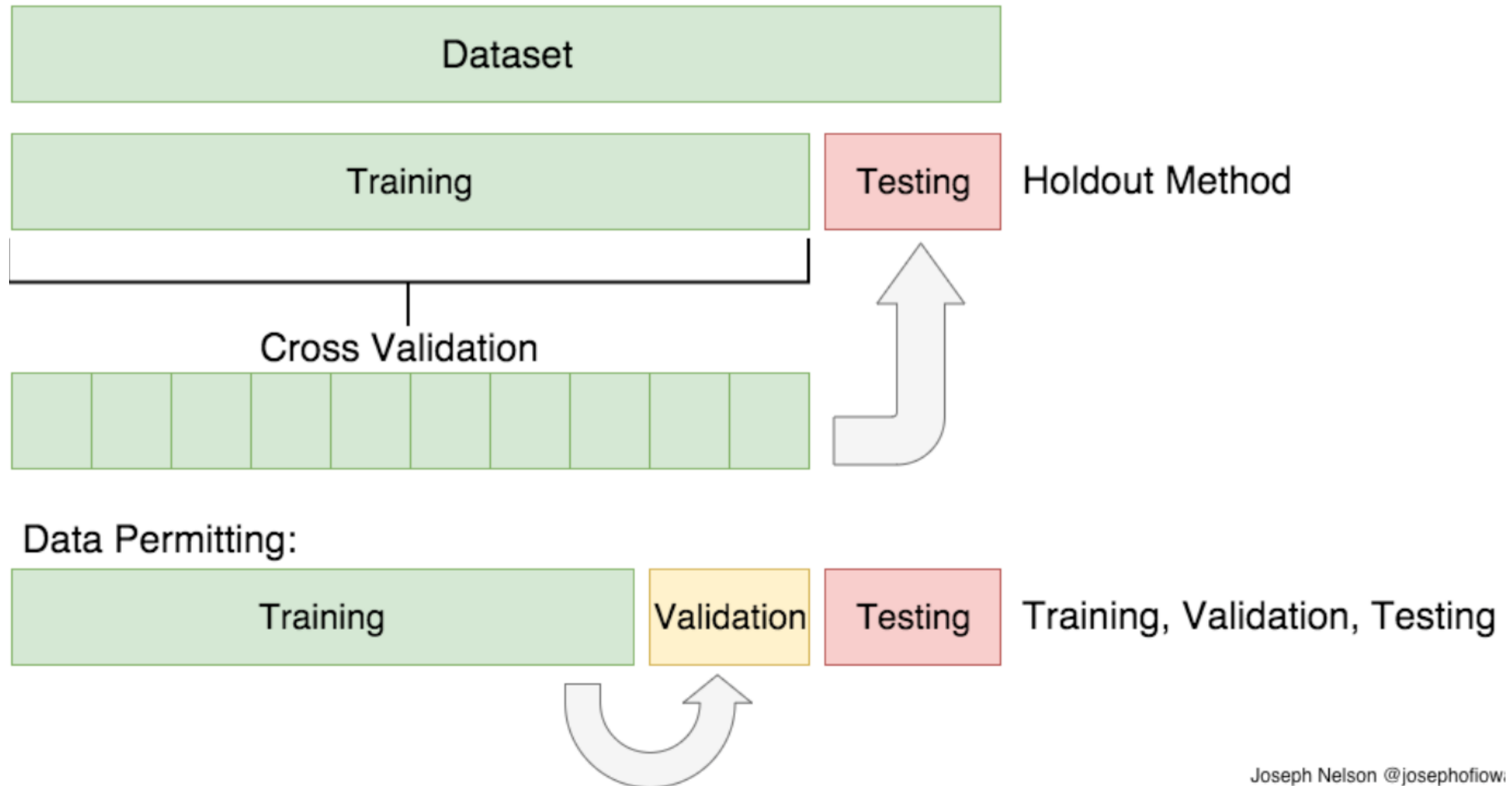


과대적합 검증

▶ 성능 비교



훈련, 검증, 테스트 데이터



Joseph Nelson @josephoflow

교차 검증

- ▶ cross validation
- ▶ 주어진 데이터를 훈련 및 테스트 데이터로 한번만 나누어 사용하지 않고 여러가지 경우로 나누어 사용하는 방식
- ▶ K-fold 교차 검증이 널리 사용된다



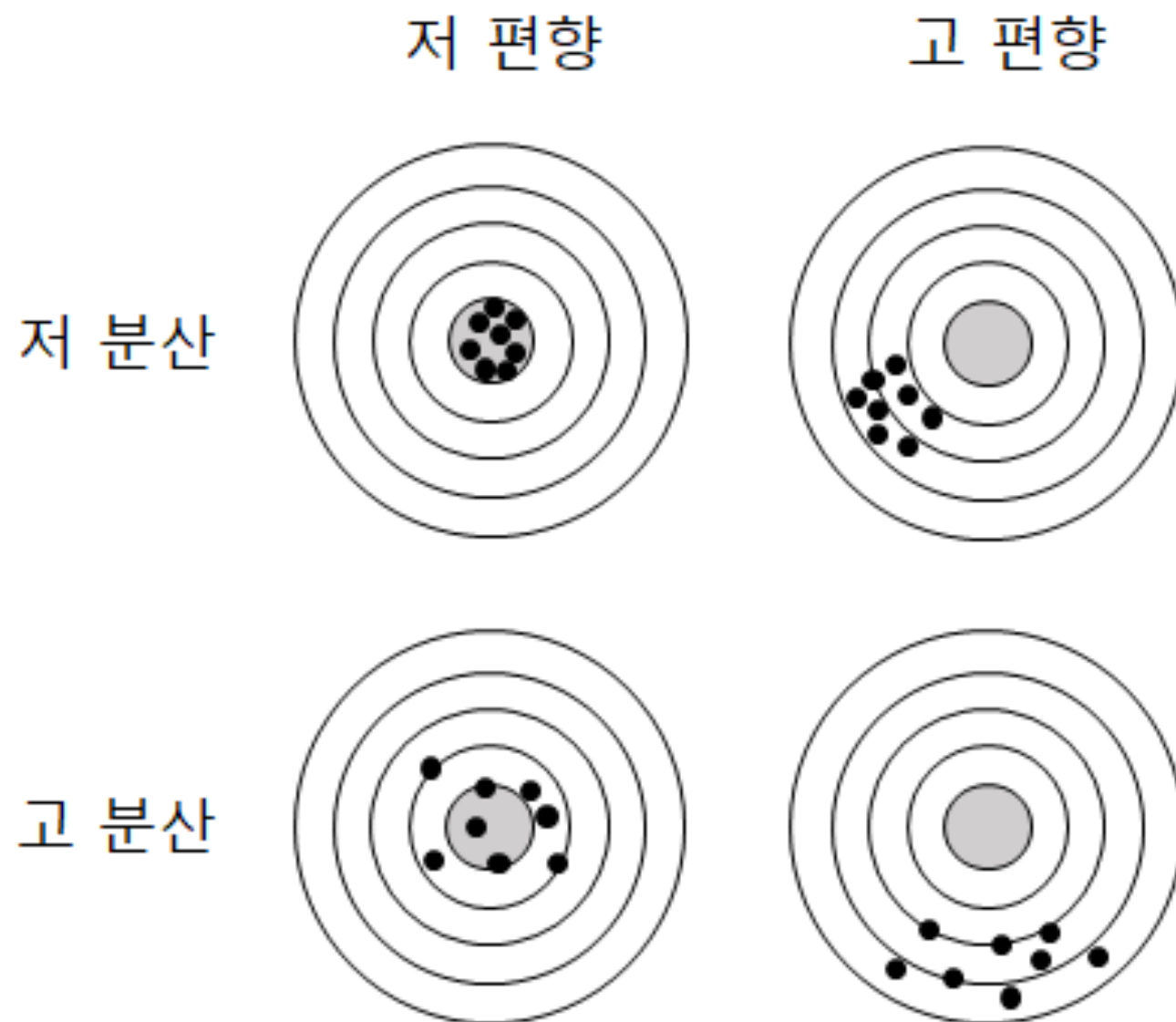
교차검증

- ▶ `cross_val_score()` 함수 사용
- ▶ K개 검증의 점수가 유사하게 나오면 안정적인 모델이지만 만일 K개의 점수의 편차가 크면 모델이 불안정하다고 볼 수 있다.

```
cv = KFold(X.shape[0], 5, shuffle=True)  
print(cross_val_score(clf_all, X, y, cv=cv))
```

편향과 분산

- ▶ 예측 모델 오차는 분산(variance)과 편향(bias) 두 성분이 있다
 - ▶ 분산이란 모델이 너무 복잡하거나 학습데이터 민감하게 반응하여 예측 값이 산발적으로 나타나는 것
 - ▶ 편향이란 모델 자체가 부정확하여 피할 수 없이 발생하는 오차



머신러닝 모델 오류

▶ 분산

- ▶ 모델이 과대적합되어 입력의 작은 변화가 큰 오류로 나타날 수 있다
- ▶ 모델이 일반화되지 못하고 잡음을 신호로 잘못 취급한다

▶ 편향(바이어스)

- ▶ 과소적합 상태로서 잘못된 가정에 의한 오류이다
- ▶ 모델 자체가 단순하여 피할 수 없이 발생하는 오차이다

▶ 잡음 (noise)

- ▶ 측정중에 발생하는 오류로서 모델이 찾아내지 못하는 오류이다
- ▶ 노이즈는 예측이 안되므로 학습 데이터에서 노이즈를 미리 줄여야 한다
- ▶ (참고) 신호란 모델 예측에 도움이 되는 정보를 말한다

데이터의 대표성

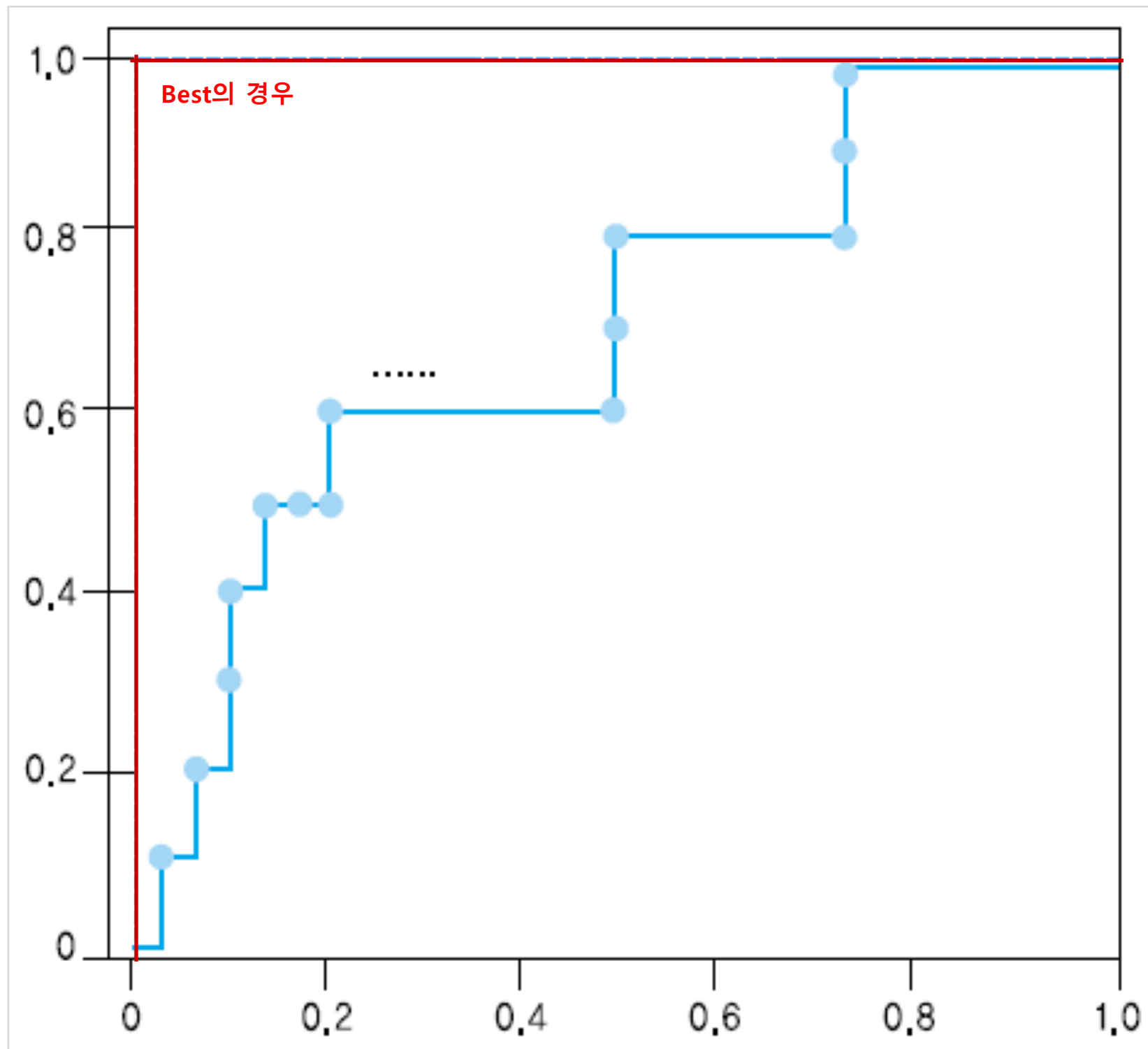
- ▶ 훈련 데이터는, 미래에 실제 적용할 데이터의 분포를 반영하도록 준비되어야 한다
- ▶ 이를 위해 계층적 샘플링(stratified sampling)이 필요하다
 - ▶ 남녀노소 및 지역의 비율 조정 등
- ▶ (참고) 클래스 데이터가 매우 불균형(imbalanced)일 때
 - ▶ 머신러닝의 성능을 높이기 위해서 계층적 샘플링을 무시하고 적게 발생하는 샘플을 적절히 증식(augmentation) 해야 한다

순위 평가

순위 평가

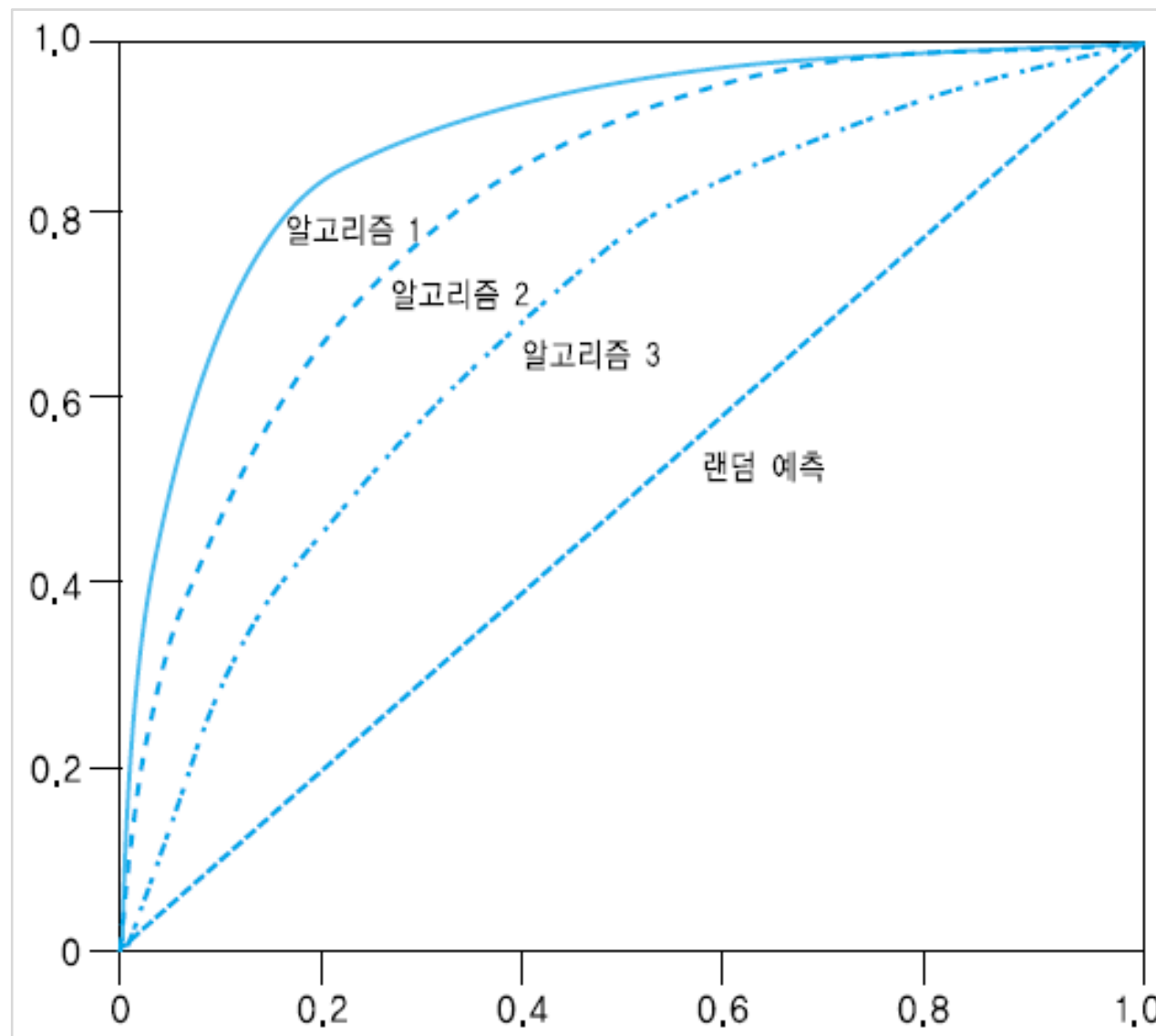
- ▶ 하나의 선택된 혼돈 매트릭스로 계산된 **정적인** 성능 평가가 아니라, 예측한 순서 전체를 점수순으로 평가하는 것
- ▶ ROC 곡선
 - ▶ 예측 결과를 순서대로 제시한 것이 실제 값과 얼마나 순서대로 맞는지는 검증하는 2차원 그래프
 - ▶ (0,0)점에서 시작하여 점수순으로 한 샘플씩 확인하여, 정답을 맞추었으면 y축 위로 한 칸 이동, 정답을 맞추지 못했으면 x축 방향으로 한 칸 이동
- ▶ AUC(Area Under the Curve)
 - ▶ ROC 성능을 간단히 수치로 나타내기 위해서 ROC 그래프의 면적을 계산한 값 (이상적인 분류이면 1점)

ROC 그래프 예시



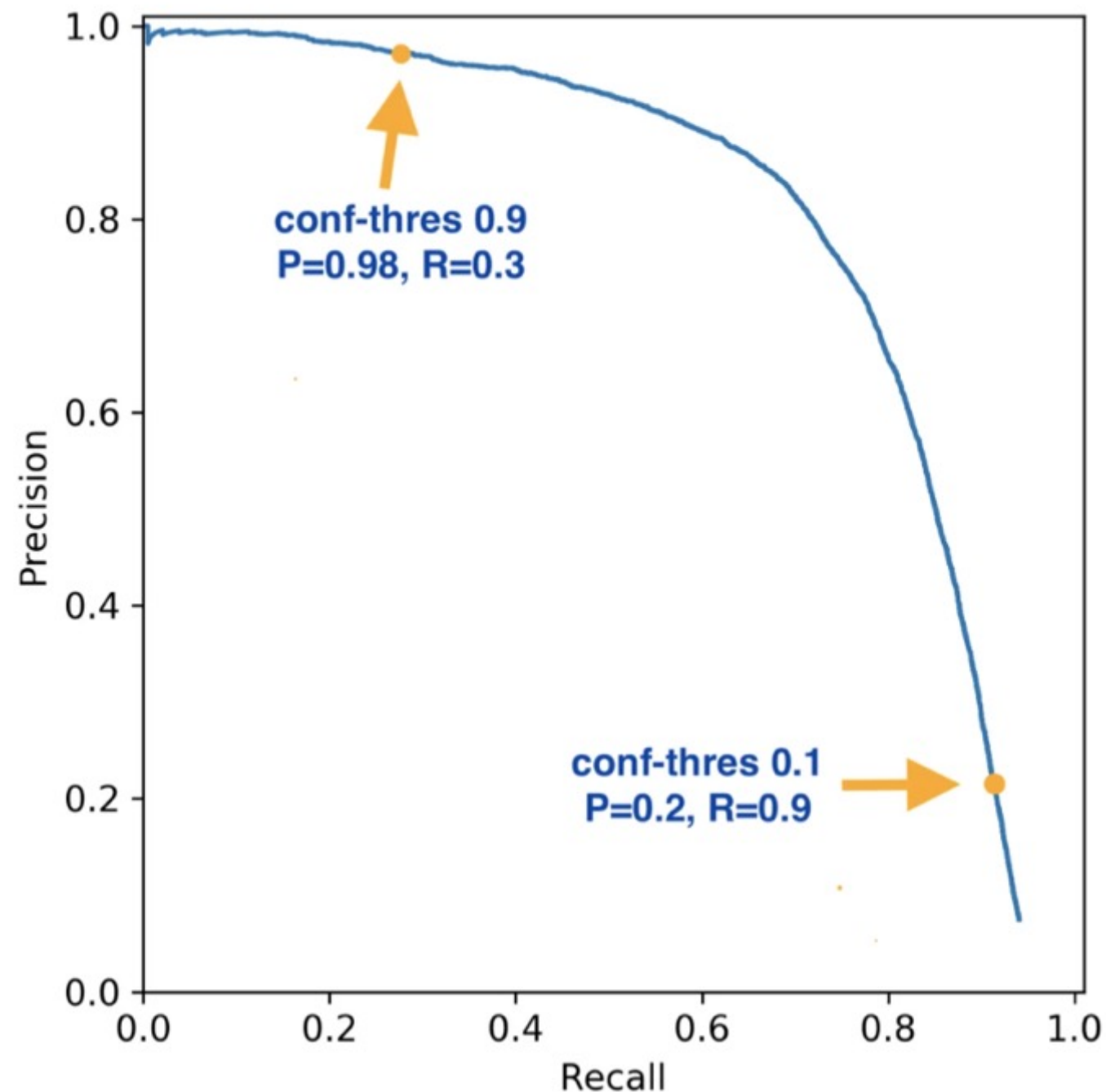
AUC(Area Under Curve)

- ▶ ROC 그래프의 면적을 계산
- ▶ 우수한 알고리즘일수록 초반에 y축 상단 방향으로 이동하므로 ROC 커브의 면적이 넓어짐



Recall-Precision 곡선

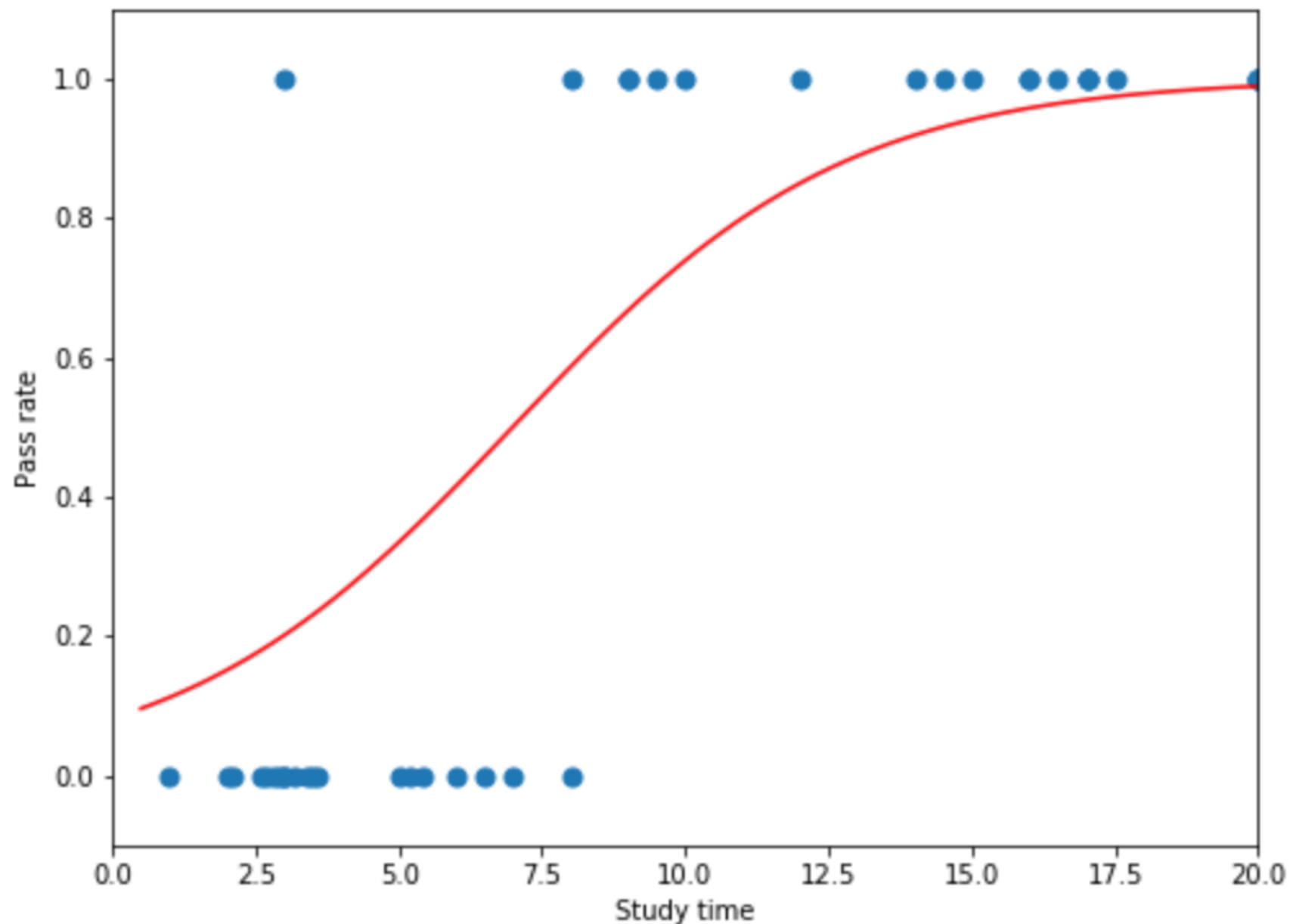
- ▶ 분류 예측 점수가 높은 순으로 예측 값을 하나씩 확인하여 (recall, precision) 좌표에 표현한 것



로지스틱 회귀

로지스틱 회귀

- ▶ 로지스틱 회귀분석(logistic regression)은 임의의 범위를 갖는 선형적 값으로부터 이진 분류의 확률을 예측하는 모델

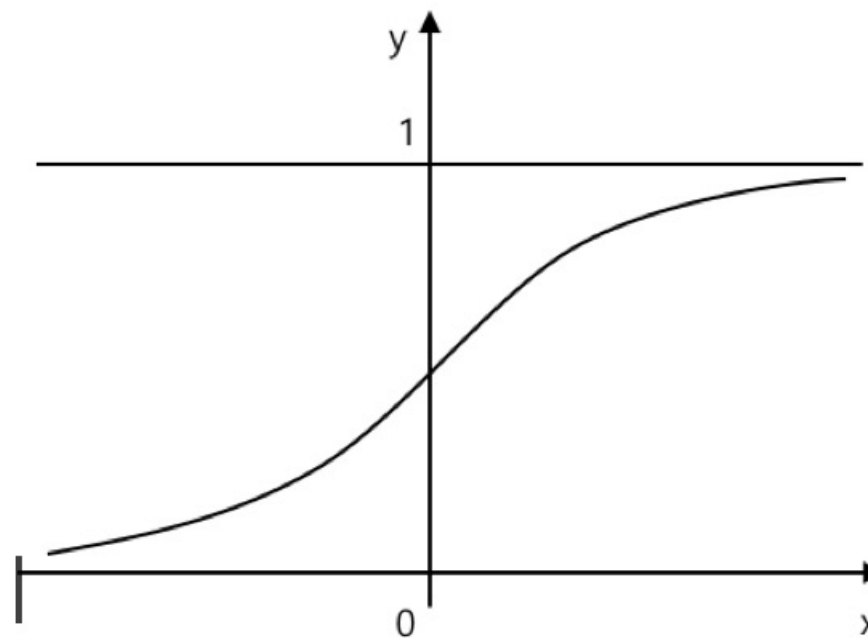


sigmoid

- ▶ 입출력의 관계를 S형 커브를 사용하여 선형값을 확률로 매핑할 때 사용되는 함수
- ▶ S 커브를 시그모이드 함수라고 한다

$$p = \frac{1}{1 + e^{-y}}$$

$$p = \frac{1}{1 + e^{-(ax+b)}}$$

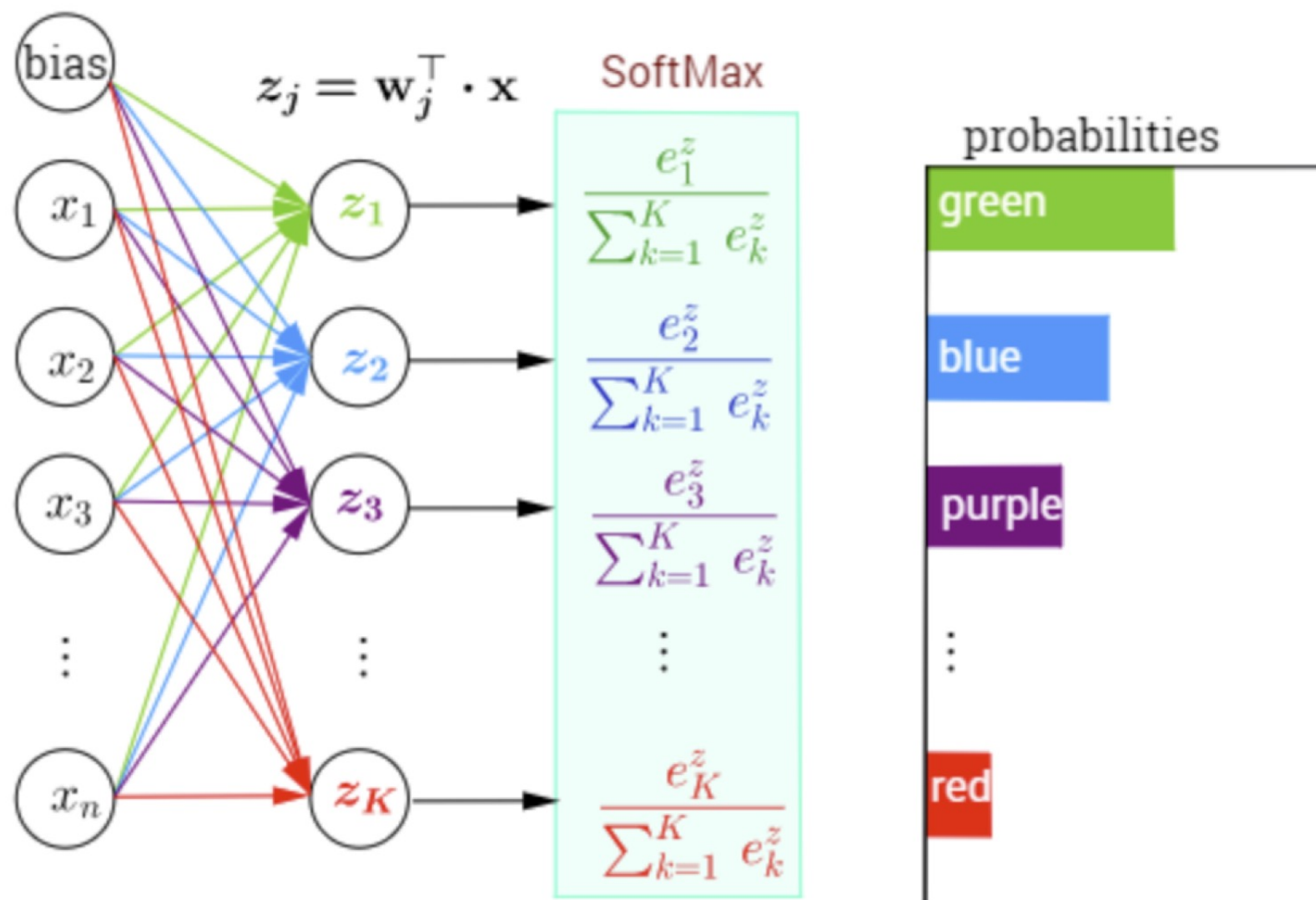


소프트맥스

- ▶ 이진 분류가 아니라 3개 이상의 클래스 중에 하나를 예측해야 하는 경우는 로지스틱 회귀를 사용할 수 없다
- ▶ 이때는 소프트맥스 (softmax) 함수를 이용한다
 - ▶ 각 클래스로 분류될 가능성을 나타내는 점수를 먼저 구하고 이 점수들을 사용하여 상대적인 확률의 비율을 구한다

소프트맥스

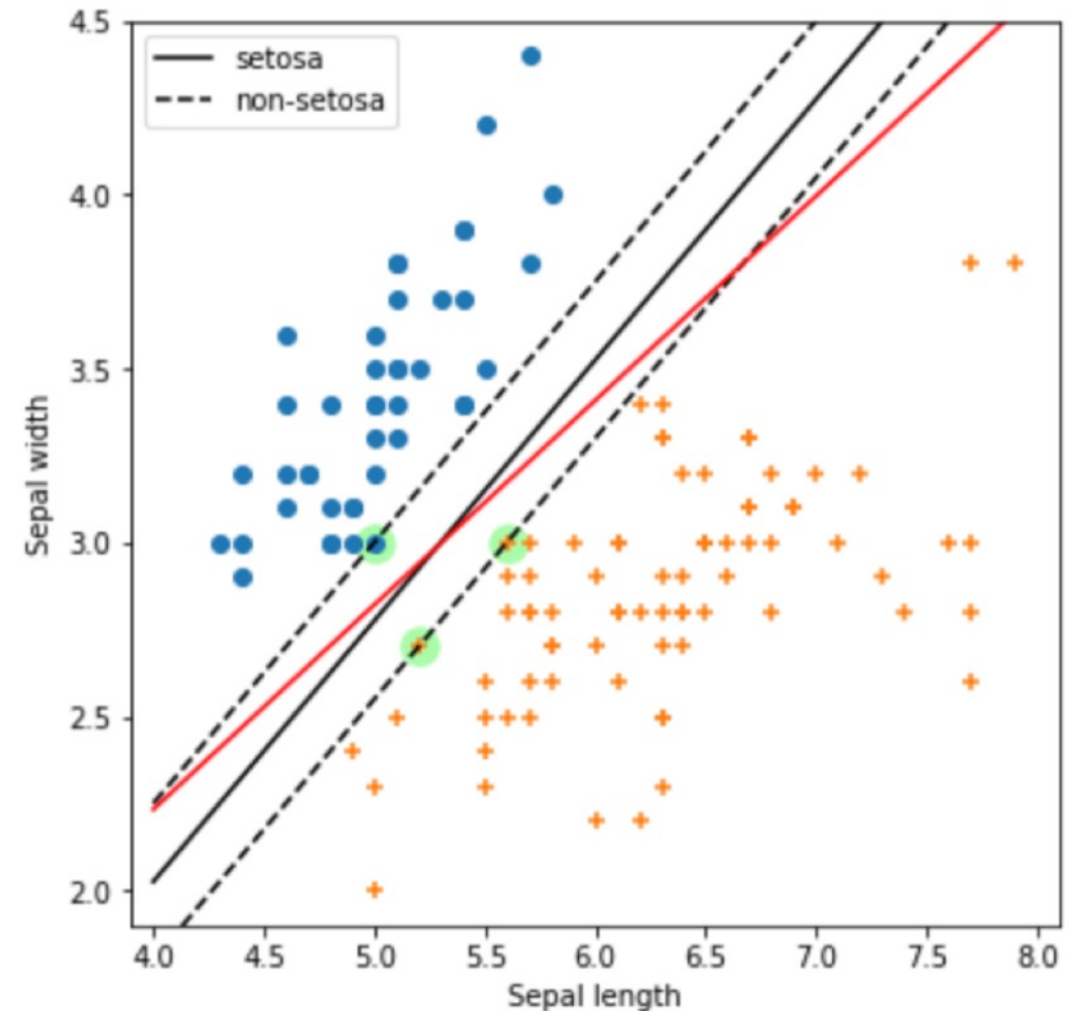
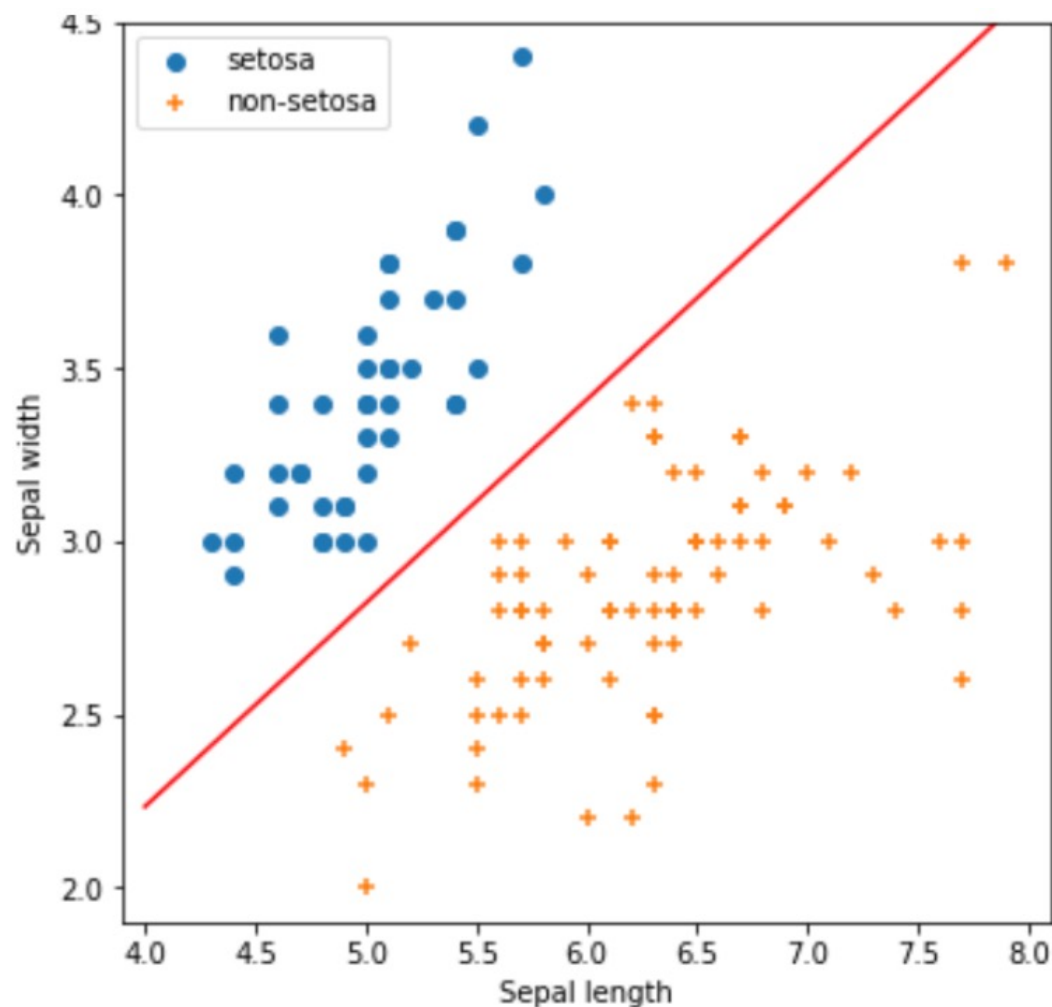
$$\hat{p}_k = \sigma(s(x))_k = \frac{\exp(s_k(x))}{\sum_{j=1}^K \exp(s_j(x))}$$



SVM

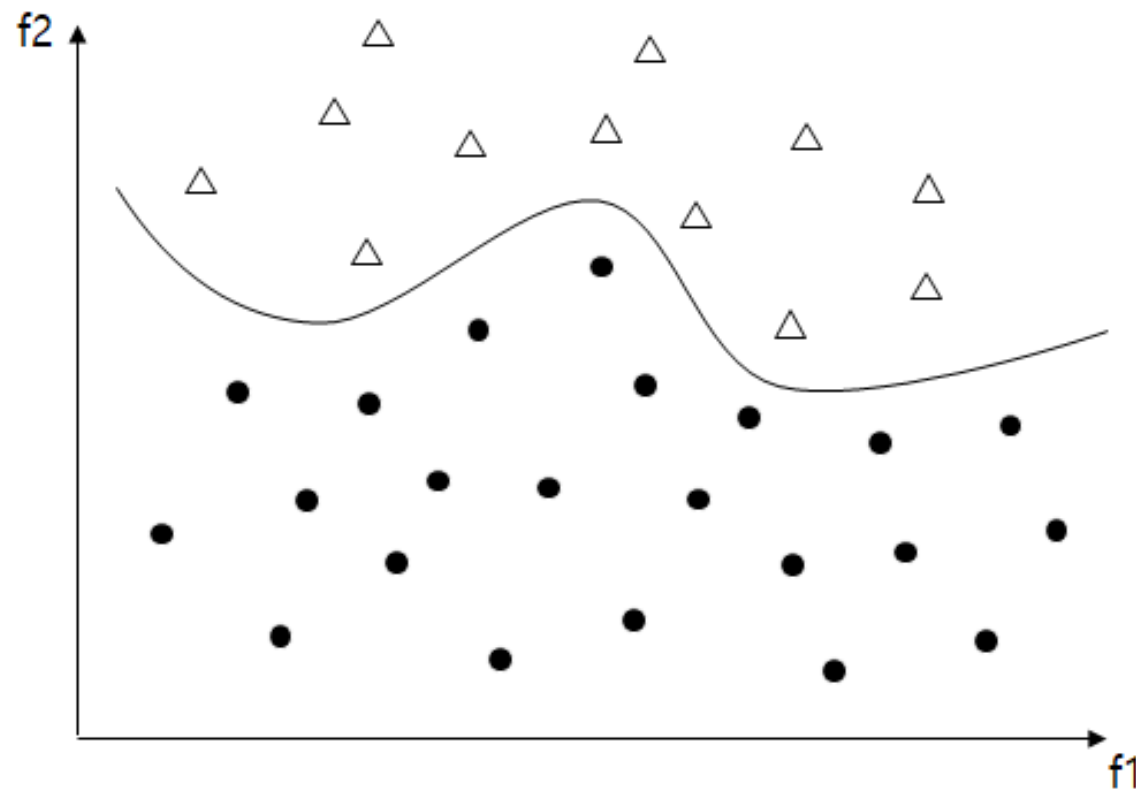
SVM

- ▶ 서포트 벡터 머신(Support Vector Machine, SVM)은 선형 모델을 개선한 것
- ▶ 기본 아이디어는 분류시에 경계면을 가능한 일반화 한 것
 - ▶ 분류시 샘플 집단 간의 거리를 가능한 멀리 나눌 수 있는 경계면을 찾는다 (Margin을 최대화 하는 경계면을 찾는 것)



커널 기법

- ▶ SVM에서는 입력 특성의 2승, 3승, 4승 등 고차원의 속성을 추가로 사용한다
- ▶ 이를 커널 트릭 기법이라고 하며 이를 통해 비선형적인 경계면을 사용하는 효과를 얻는다



SVM 특징

- ▶ SVM은 샘플 수가 적을 때 유용하다
- ▶ SVM은 선형 모델이므로 입력을 스케일링 해야 한다
- ▶ 마진의 크기와, 커널의 차원 하이퍼파라미터의 최적값을 실험으로 찾아야만 한다
 - ▶ 하이퍼파라미터 실험적인 최적화가 반드시 필요하다

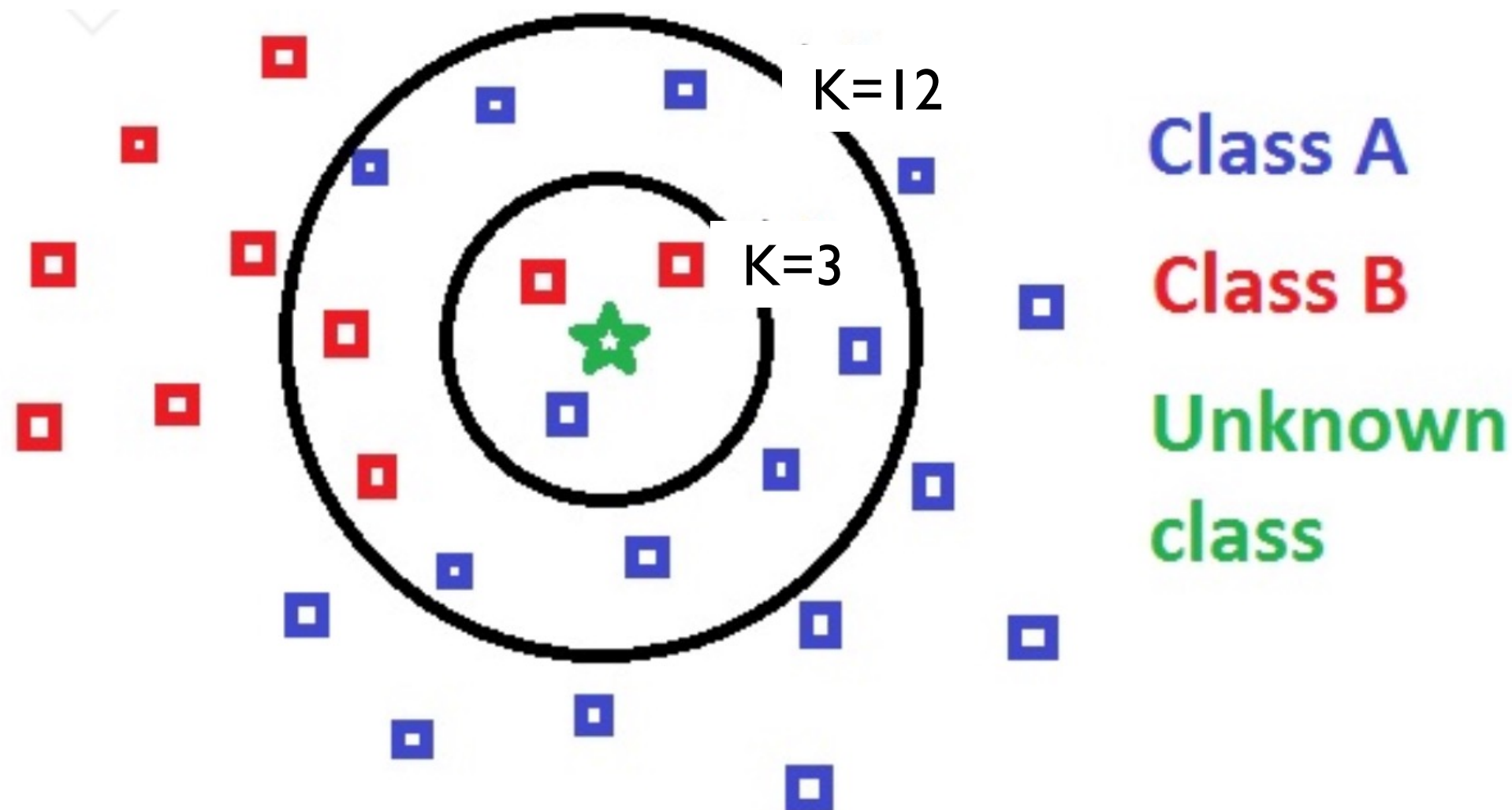
kNN

KNN

- ▶ kNN(k-nearest neighbor) 모델
 - ▶ 샘플의 특성 공간에서 가까운 이웃(neighbor)을 k개 선택하고 이들 레이블의 평균치로 이 샘플이 속할 분류를 예측하는 방식이다.
- ▶ kNN알고리즘을 협업 필터링(collaborative filtering)이라고도 부른다

kNN 하이퍼파라미터

- ▶ k 값이 작을수록 과대적합이 되고 k 가 크면 과소적합이 된다

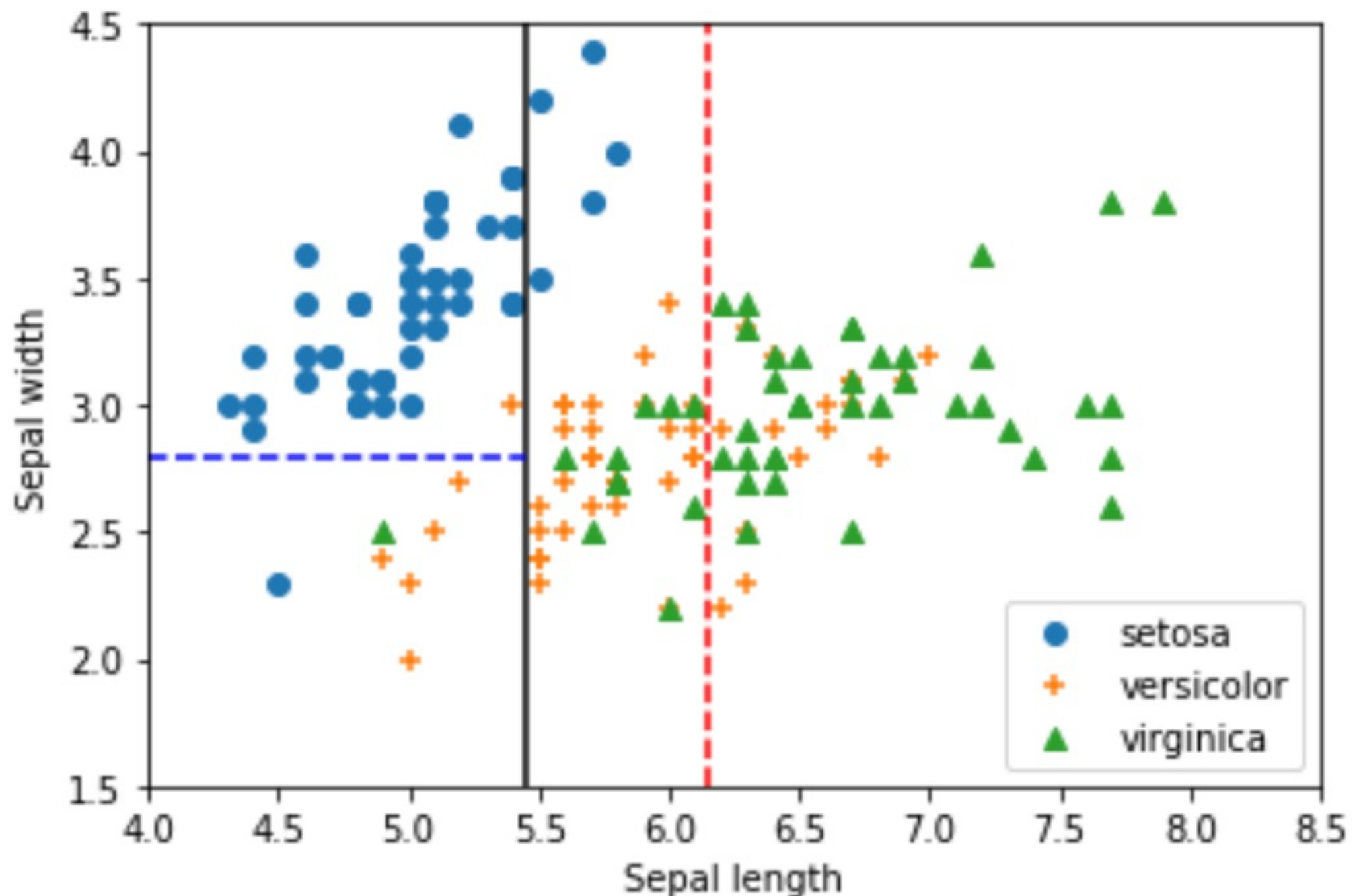


- ▶ 모델 훈련시간이 필요 없지만 새로운 샘플이 추가되면 상호 거리 계산을 다시 해야 한다

결정 트리

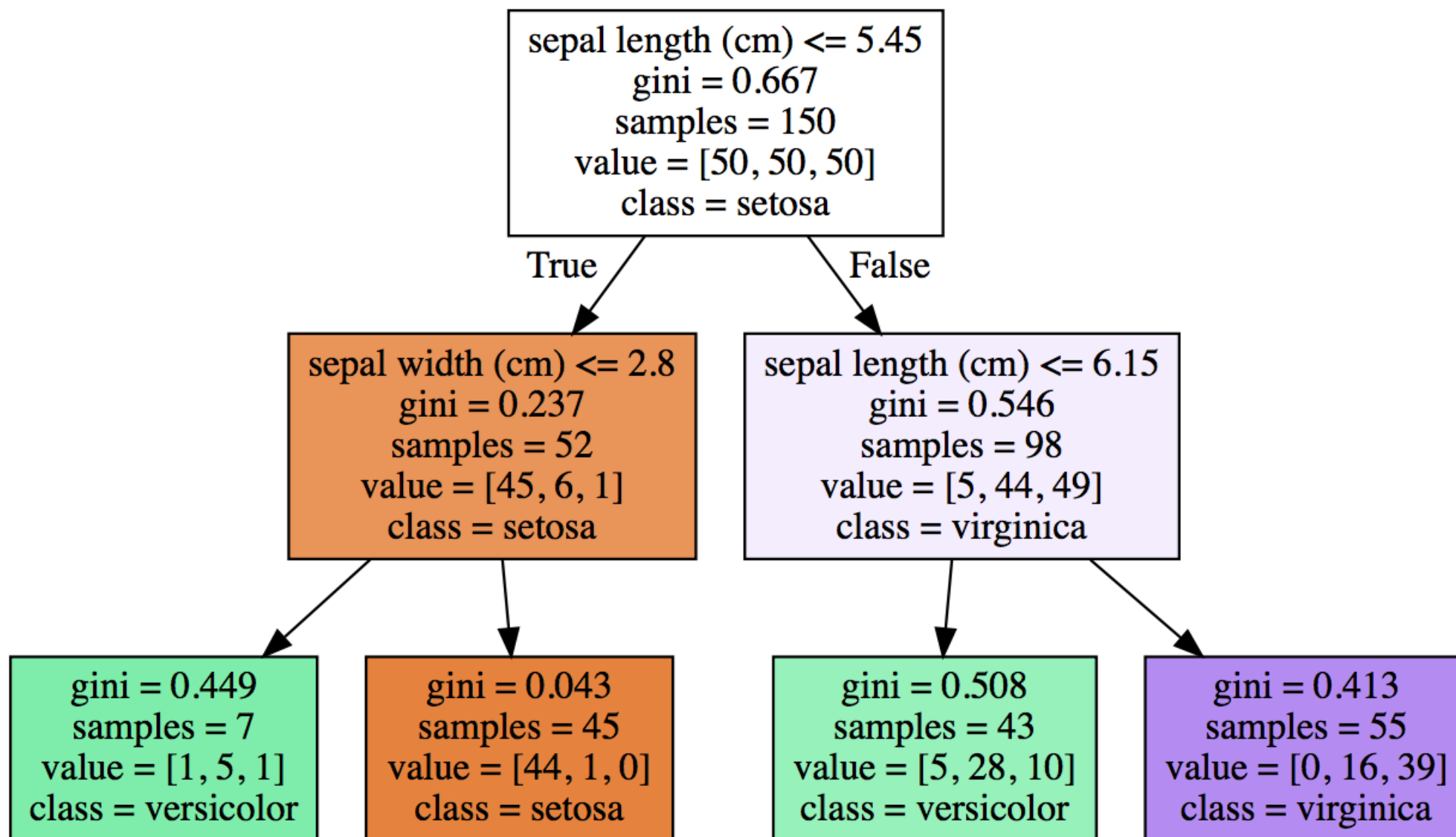
결정트리

- ▶ 한번에 한 특성씩을 대상으로 훈련 샘플을 (두개의) 그룹으로 나누되 각 그룹에 가능한 유사한 샘플들이 모이도록 한다



결정트리 동작

- ▶ 그룹을 효과적으로 “잘 나누는 것”의 기준은 그룹을 나눈 후에 생성되는 하위 그룹들에 가능하면 같은 종류의 아이템들이 모이는지를 기준으로 삼는다
- ▶ 순도(purity)가 높게 나눈다



결정 트리

- ▶ 선형계열 모델과 다른점
 - ▶ 선형 모델은 입력 특성들을 대상으로 곱셈과 덧셈을 하고 그 값을 사용하여 회귀나 분류를 예측했고 스케일링이 필요했다
- ▶ 결정 트리(decision tree) 모델은 이와 달리 각 특성을 독립적으로 하나씩 검토하여 분류 작업을 수행한다
 - ▶ 마치 스무고개 하여 예측을 하듯이 동작 한 번에 한 특성을 따져보는 방법이다
 - ▶ Yes/No 이진 판별만 수행하며 나누어지는 그룹의 순도의 합이 가장 낮아지도록 그룹을 나눈다
- ▶ 결정 트리는 분류와 회귀에 모두 사용된다
 - ▶ 각각 DecisionTreeClassifier, DecisionTreeRegressor 사용
 - ▶ 회귀모델로 동작할 때는, 나누어진 그룹의 샘플의 분산값이 가능하면 적게 되도록 분류를 수행한다 (손실함수로 MSE 사용)

정보량과 엔트로피

- ▶ 정보량은 일어날 확률의 역수에 비례한다고 가정

- ▶ 정보량 = $\log\left(\frac{1}{p}\right)$

- ▶ 엔트로피: 정보량의 기대치로, 어떤 사건이 갖는 가치와 그 사건이 발생할 확률의 곱

$$p \log\left(\frac{1}{p}\right) = -p \log(p)$$

- ▶ 분류시 순도(purity)를 측정하는데 사용된다
 - ▶ 한 노드에 여러 클래스가 골고루 균일하게 섞여 있을 때는 엔트로피가 가장 높고, 동종의 클래스로 모여 있을수록 엔트로피가 낮다

분류의 손실함수

- ▶ 분류에서 예를 들어 정확도(accuracy)를 손실함수로 사용하면 다음과 같은 문제가 있다
 - ▶ **카테고리 분포 불균형**시 문제
 - ▶ 클래스 발생빈도가 다른 경우, 적게 발생하는 클래스를 잘 찾지 못한다
- ▶ 분류 모델에서는 이를 보완하기 위해서 크로스 엔트로피(cross entropy)를 손실함수로 사용한다
 - ▶ m: 클래스 수

$$L = -\frac{1}{m} \sum_{i=1}^m y_i \cdot \log(\hat{y}_i)$$

지니계수

- ▶ 그룹의 순도를 표현하는 데 지니(Gini) 계수도 자주 사용된다

$$Gini = 1 - \sum_{k=1}^m p_k^2$$

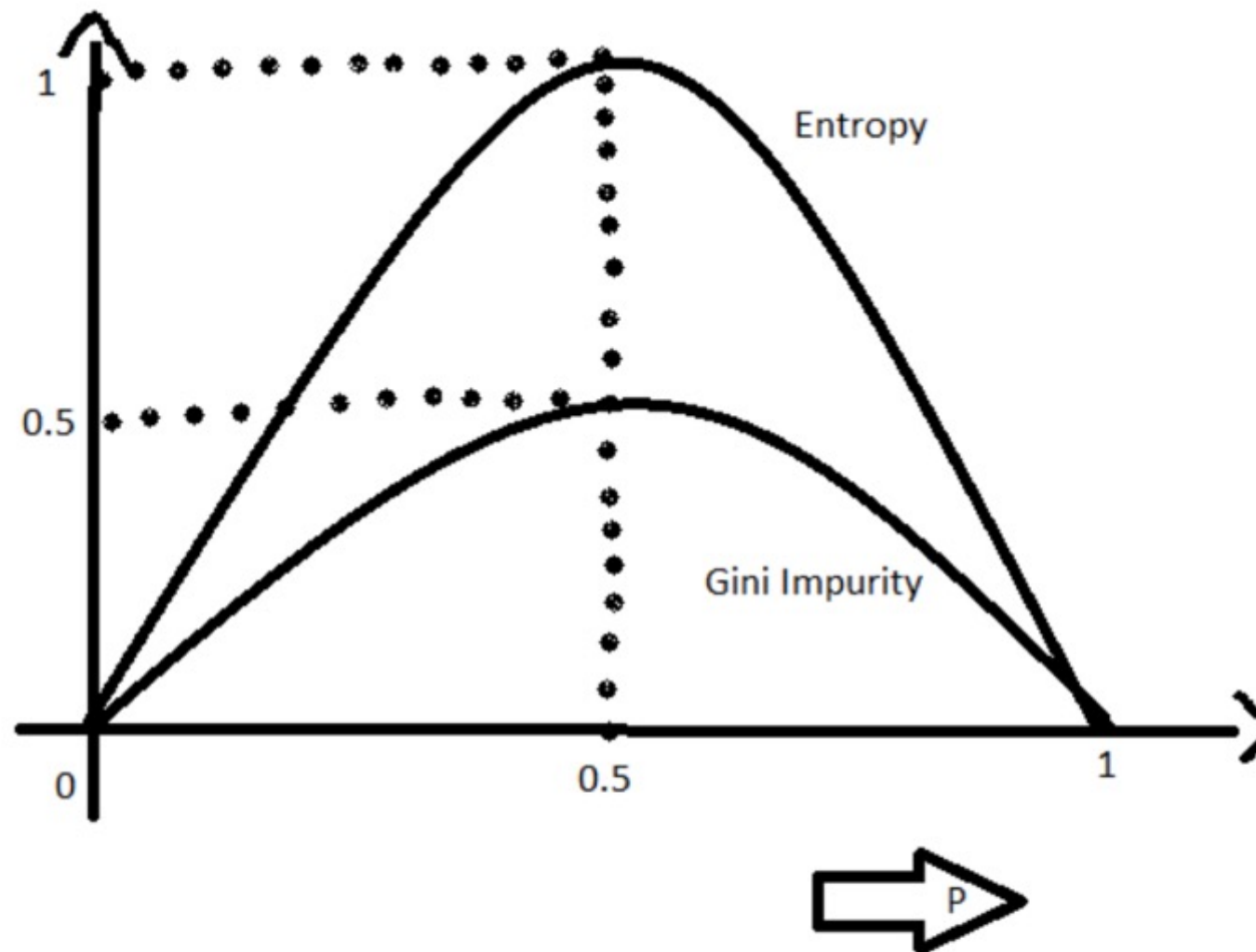
- ▶ 위에서 p_k 는 카테고리 k 에 대한 분포 확률을 말한다.

$$\text{지니}(7:3) = 1 - \left[\left(\frac{7}{10} \right)^2 + \left(\frac{3}{10} \right)^2 \right] = 1 - (0.49 + 0.09) = 0.42$$

$$\text{지니}(5:5) = 1 - \left[\left(\frac{5}{10} \right)^2 + \left(\frac{5}{10} \right)^2 \right] = 1 - (0.25 + 0.25) = 0.5$$

- ▶ 지니와 엔트로피 사용시 성능에는 큰 차이가 없으나 지니의 계산 속도가 빠르다

엔트로피와 지니계수



$$E(S) = \sum_{i=1}^c -p_i \log_2 p_i$$

$$Gini(E) = 1 - \sum_{j=1}^c p_j^2$$

특성 중요도

- ▶ `feature_importances_`
 - ▶ 결정 트리 모델을 만든 후에, 어떤 특성이 결정 트리를 생성할 때 중요한 역할을 했는지 상대적인 비중을 파악할 수 있다
 - ▶ 내부 변수인 `feature_importances_`에서 확인할 수 있다

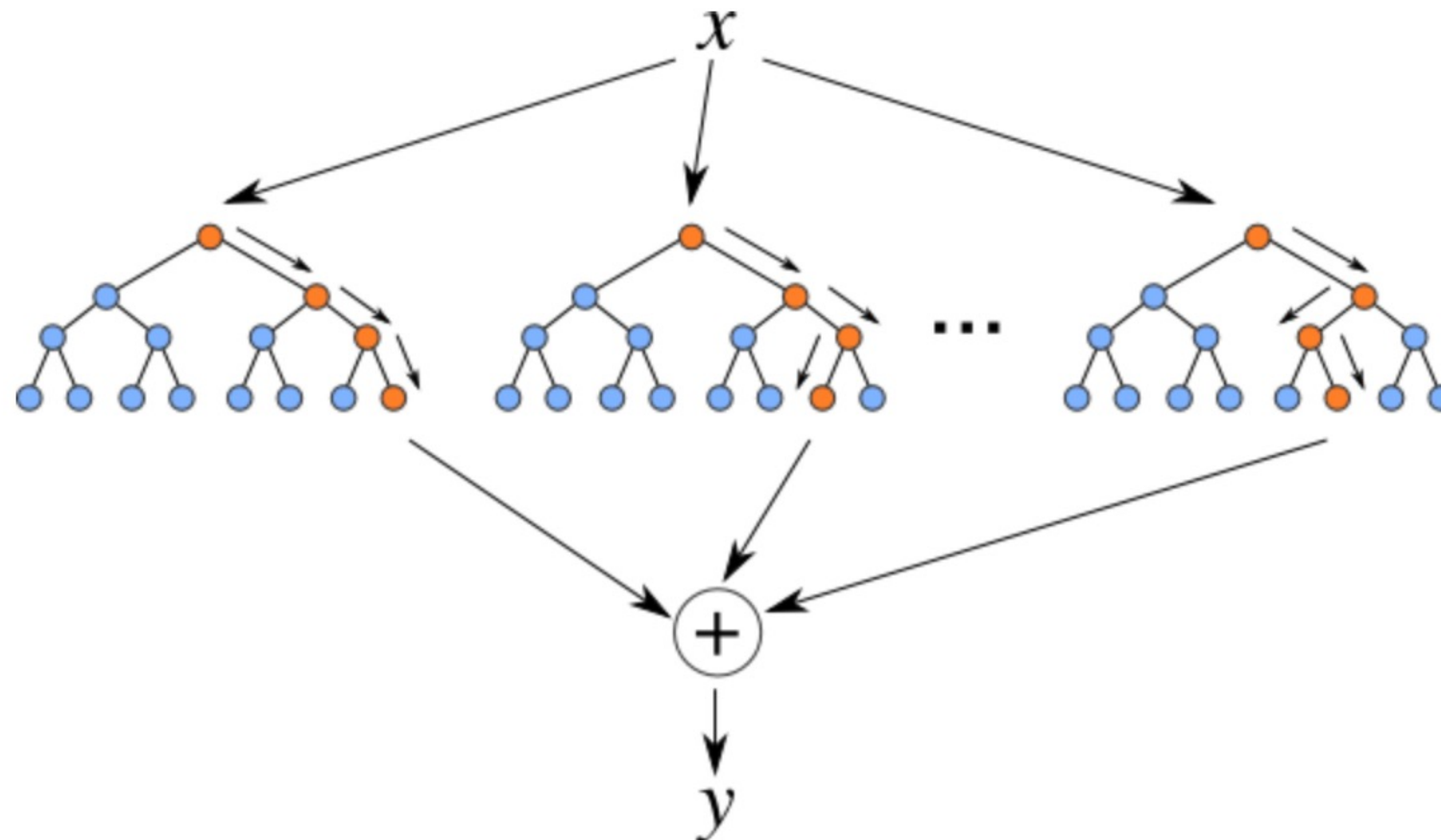
분류 확률

- ▶ 클래스 확률이라고도 한다
 - ▶ 분류 모델을 사용하여 예측을 수행하려면 `predict()`를 사용한다
 - ▶ 주어진 샘플이 어느 클래스에 속하는지 “확률”을 구할 수 있는데, 트리계열 및 부스팅, 신경망에서는 `predict_proba()` 함수를 사용한다
- ▶ 선형계열 모델
 - ▶ 선형계열 모델에서는 `predict_proba()` 함수를 제공하지 않는다
 - ▶ 즉 분류 확률을 제공하지 않는다
 - ▶ 선형계열 모델에서는 `decision_function()` 함수를 사용하여 각 클래스에 해당하는 “점수”를 얻는다
 - ▶ (참고) SVM에서는 모델 생성시에 인자 `probability=True`를 사용하면 클래스 확률을 얻을 수 있다
 - ▶ 즉, `Predictive_proba()` 함수 사용이 가능하다

랜덤 포레스트

랜덤 포레스트

- ▶ Random Forest, 결정 트리의 성능을 개선한 방법
- ▶ 간단한 구조의 결정 트리(weak learner)를 수백개 만들고 각 결정 트리의 동작 결과의 평균치를 사용하는 방법
 - ▶ 랜덤 포레스트를 구성하는 약한 결정 트리를 만들 때 훈련 데이터의 일부만 사용하거나 특성의 일부만 무작위로 선택한다
- ▶ 이를 앙상블(ensemble) 방법이라고 하며 결정 트리 모델보다 좋은 성능을 얻는다



간접투표(soft voting)

	P일 확률	Q일 확률	판정결과 (직접투표)
세부 모델 A	0.9	0.1	P
세부 모델 B	0.4	0.6	Q
세부 모델 C	0.3	0.7	Q
확률의 평균 (간접 투표)	$(1.6)/3 = \mathbf{0.533}$	$(1.4)/3 = 0.456$	P or Q

앙상블 기법

- ▶ 투표 방식
 - ▶ 일반적으로 하위 모델로 서로 다른 모델을 사용
 - ▶ 하드 보팅 또는 소프트 보팅 선택 가능
- ▶ 배깅 방식
 - ▶ 샘플링을 다르게 선택하는 방식 (같은 세부 모델을 사용)
 - ▶ 랜덤 포레스트: 전체 데이터 수와 같게 샘플링하되 중복을 허용한 경우
- ▶ 부스팅
 - ▶ 약한 모델 예측 결과를 순차적으로 적용하면서 성능을 개선하는 방식
 - ▶ (예) Light GBM
- ▶ 스택킹
 - ▶ 모델을 여러 층을 쌓아서, 각 모델의 결과를 다시 학습데이터로 사용
- ▶ 앙상블에서는 기본적으로 트리 모델을 주로 사용한다
 - ▶ 약한 학습기를 만들기가 편리하다

배깅

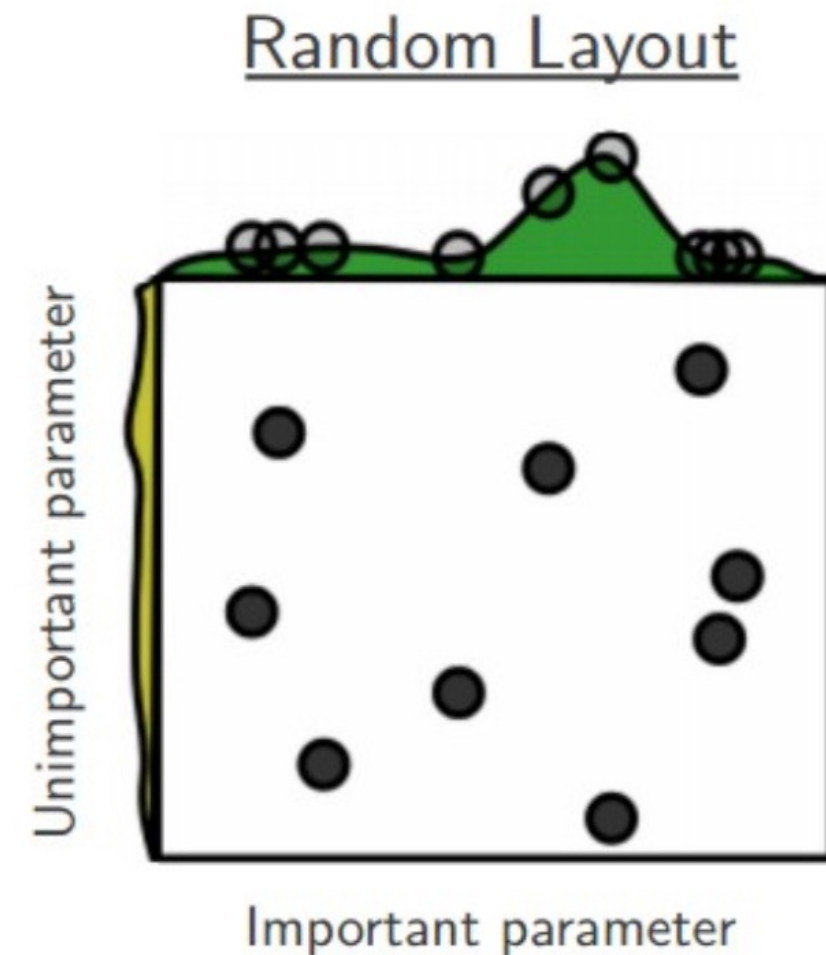
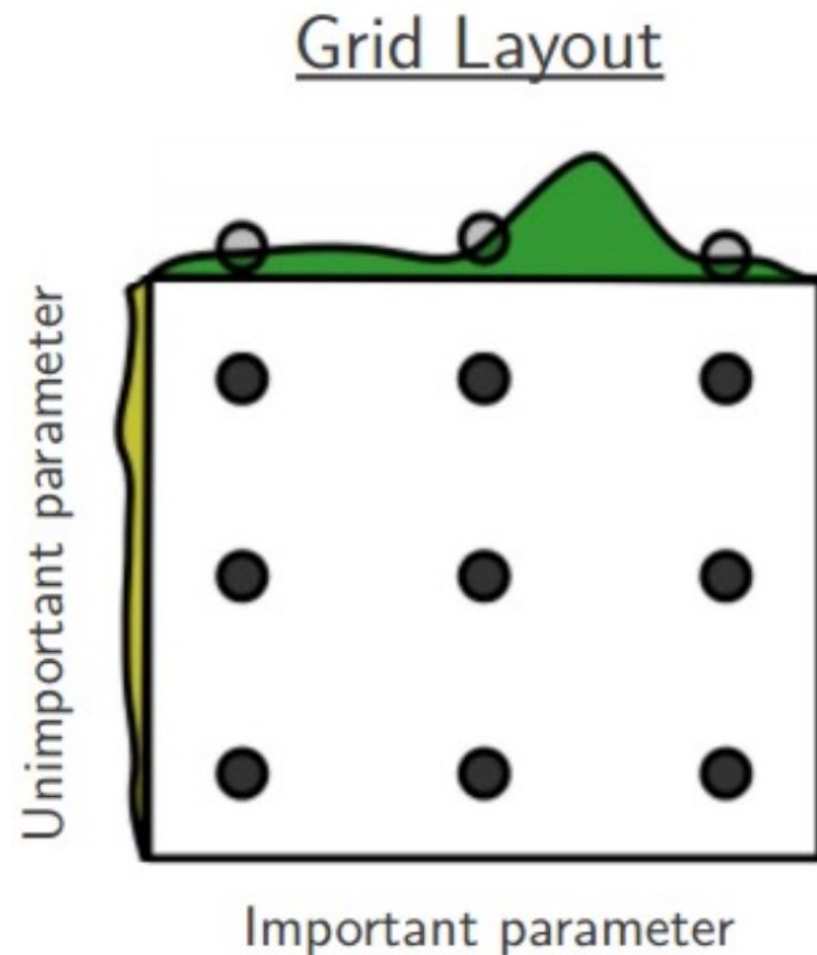
- ▶ bootstrap aggregation의 줄임말이며 전체 훈련 데이터에서 “중복을 허용”하여 데이터를 샘플링을 하는 방법
 - ▶ 배깅으로 샘플을 취하면 전체 데이터 중에서 평균 63%의 데이터만 선택이 된다.
- ▶ 배깅을 수행하면 학습에 선택되지 않는 샘플은 평균 37%가 되는데 이 샘플을 oob(out of bag) 샘플이라고 한다.
 - ▶ 이 oob 데이터를 검증 데이터로 사용하면 별도로 검증 데이터를 나누지 않고도 평가를 할 수 있어 편리하다
- ▶ 결정 트리를 기반으로 배깅을 적용한 방식이 바로 랜덤 포레스트 모델이다

모델 최적화

모델 최적화

- ▶ 최적의 머신러닝 모델을 만드는 과정
 - ▶ 모델 구조 선택
 - ▶ 모델 학습 - 파라미터 학습
 - ▶ 모델 최적화 - 최적의 하이퍼 파라미터 선택
- ▶ 하이퍼파라미터의 최적화
 - ▶ 과대적합과 과소적합을 피하면서 성능평가를 최대로 하는 하이퍼 파라미터를 찾는 것

그리드 탐색과 랜덤 탐색

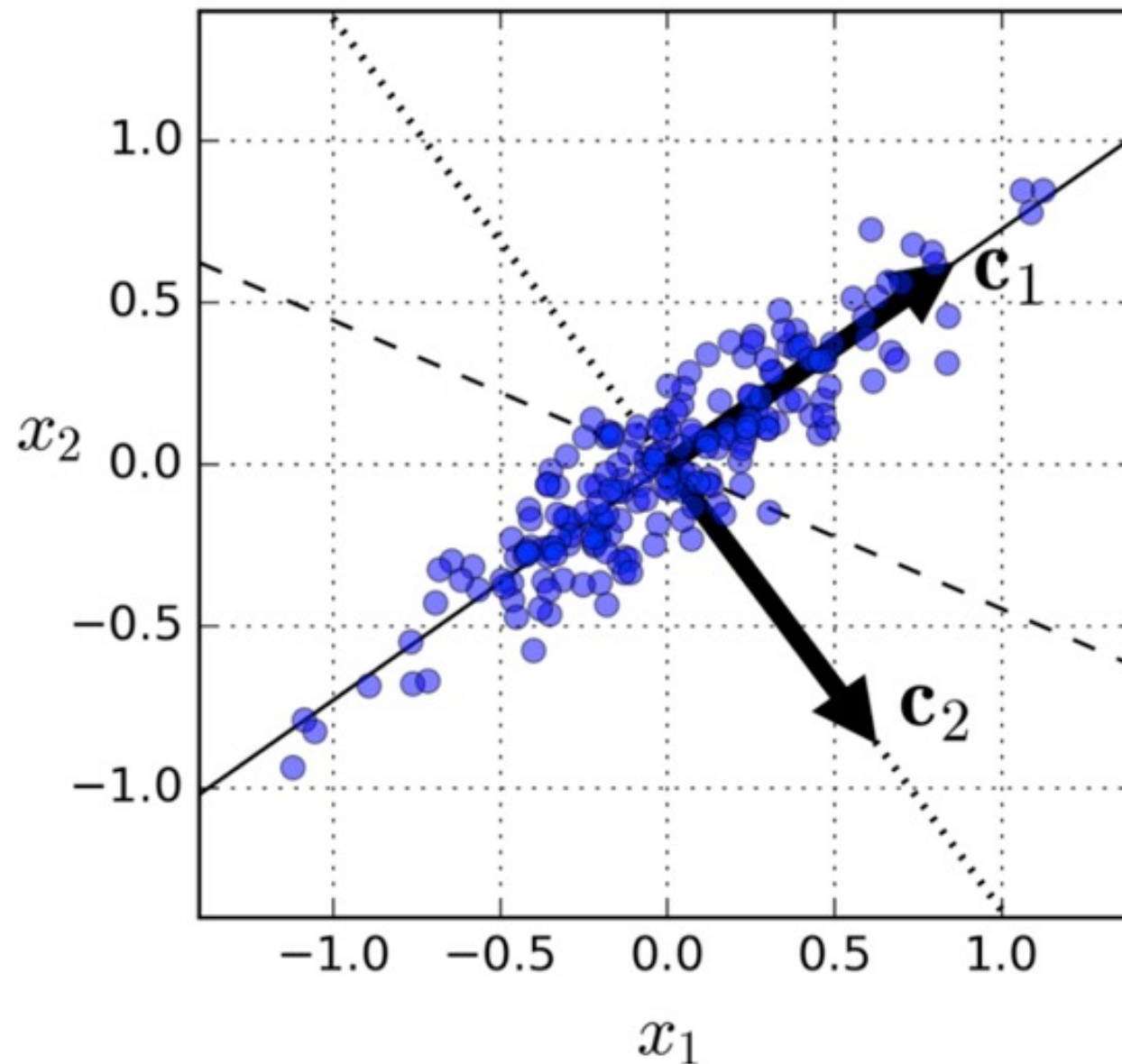


특성 공학

- ▶ Feature Engineering
- ▶ 분석에 적절하게 특성을 변형하거나, 차원을 축소하는 것
 - ▶ (예) BMI(비만도) = 몸무게 / 키*키
- ▶ 차원 축소
 - ▶ 머신러닝의 성능을 떨어뜨리지 않으면서 특성의 수를 줄이는 것
 - ▶ 같은 정보량을 가지면서 데이터의 크기를 줄여 동작 속도와 성능을 개선
 - ▶ 특성의 관계를 보기 좋게 시각화하기 위해서도 차원축소가 필요하다
- ▶ 차원 축소 방법
 - ▶ 사람에 의한 특성 선택
 - ▶ 목적 변수와의 상관계수 이용
 - ▶ 주성분 분석 (PCA)

PCA

- ▶ PCA(Principal components analysis)
 - ▶ 기존의 특성들을 차원수가 적은 다른 특성들로 표현



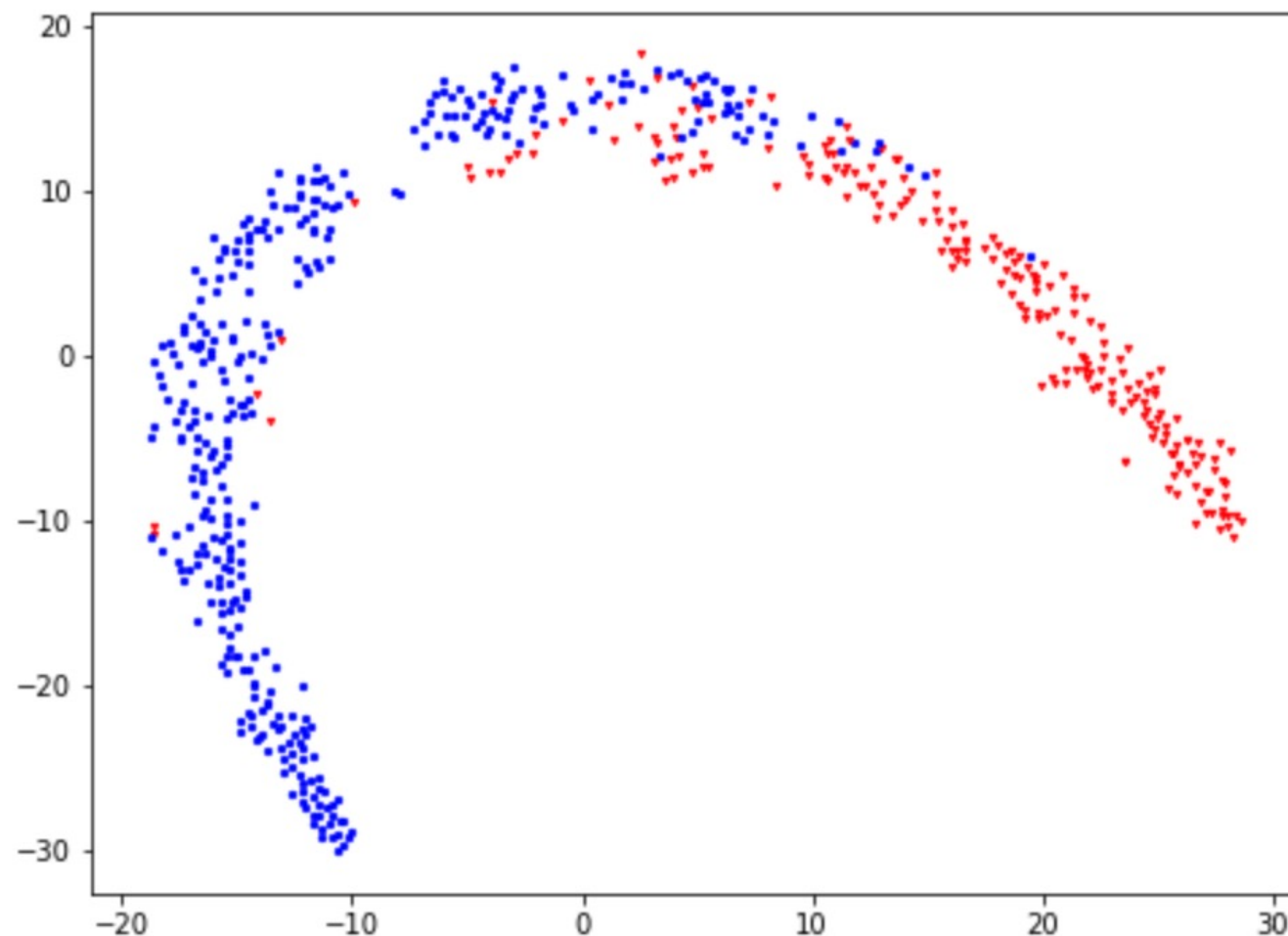
PCA

▶ 주성분 분석

- ▶ 여러 특성들을 조합하여 이들을 대표할 수 있는 적은 수의 특성(이를 주성분이라고 한다)를 찾아내는 작업을 말한다
- ▶ 주성분은 기존의 속성들의 선형 조합으로 새로운 변수를 정의한다
- ▶ 주성분 분석을 하려면 최종적으로 필요한 주성분의 개수를 알려주어야 한다
- ▶ 주성분을 만들기 위해서 기존의 특성에 각각 어떤 가중치를 곱하였는지를 알려면 내부변수 `pca.components_`를 보면 된다.

t-SNE

- ▶ 시각화를 위해서 고차원의 특성을 가진 데이터를 저차원으로 축소하는 기술로 t-SNE가 널리 사용된다
- ▶ t-distributed stochastic neighbor embedding



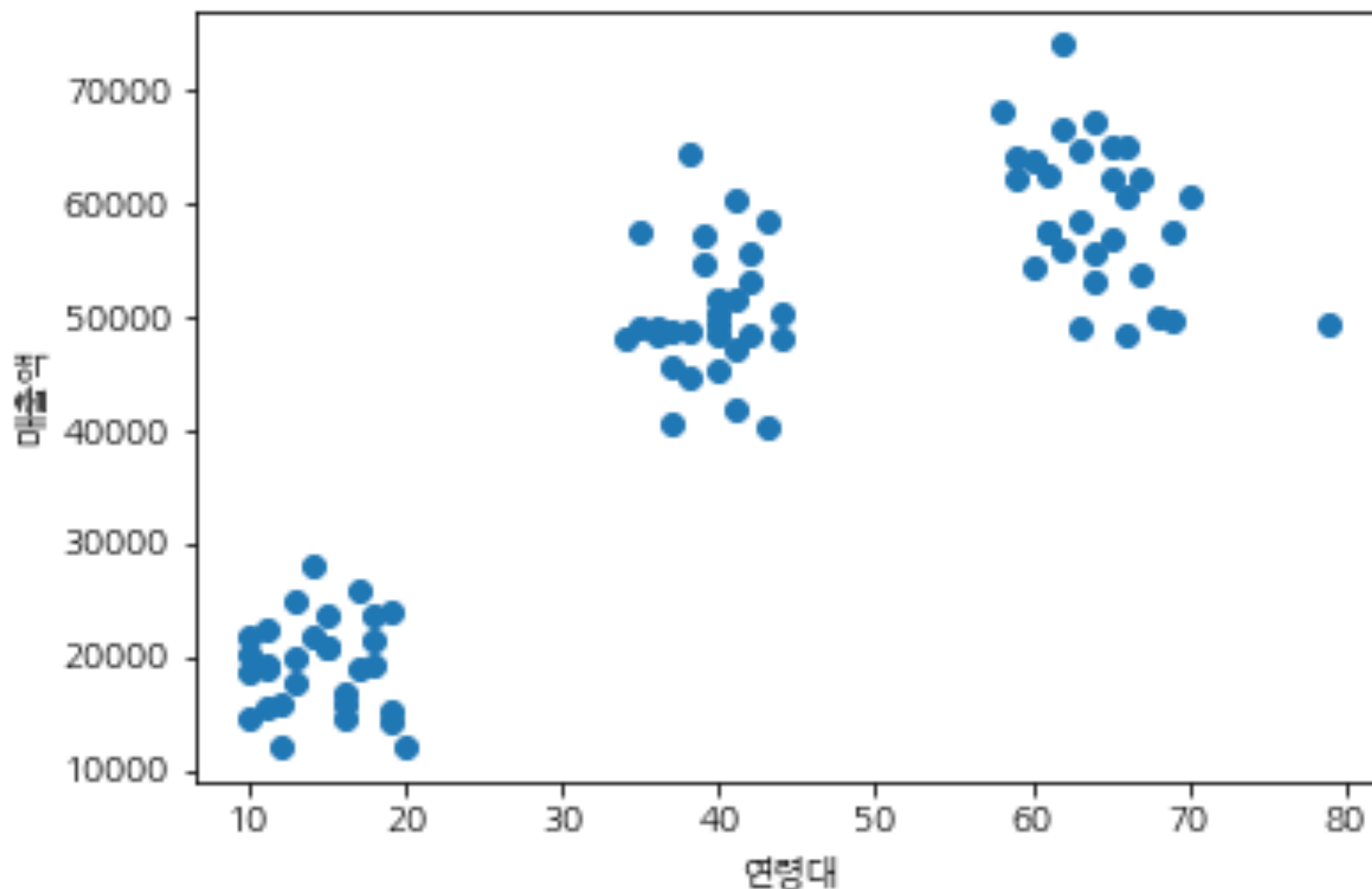
최적 제어 선택

- ▶ 시뮬레이션 방법
 - ▶ 우수한 예측 모델을 이용하여 제어 변수의 변화에 따른 출력을 시뮬레이션
 - ▶ 빠른 추론 inference 기능이 필요
- ▶ 제어 변수를 목적변수화
 - ▶ 목적변수와 제어 변수를 바꾸어 새로운 머신러닝 모델을 학습
 - ▶ 머신러닝 모델은 원인-결과(causality) 관계만 찾는 것은 아니다
- ▶ 전문가 mimic
 - ▶ 도메인 전문가의 의사결정 기록을 레이블로 하여 학습
 - ▶ 다양한 경우의 학습 데이터 필요

클러스터링

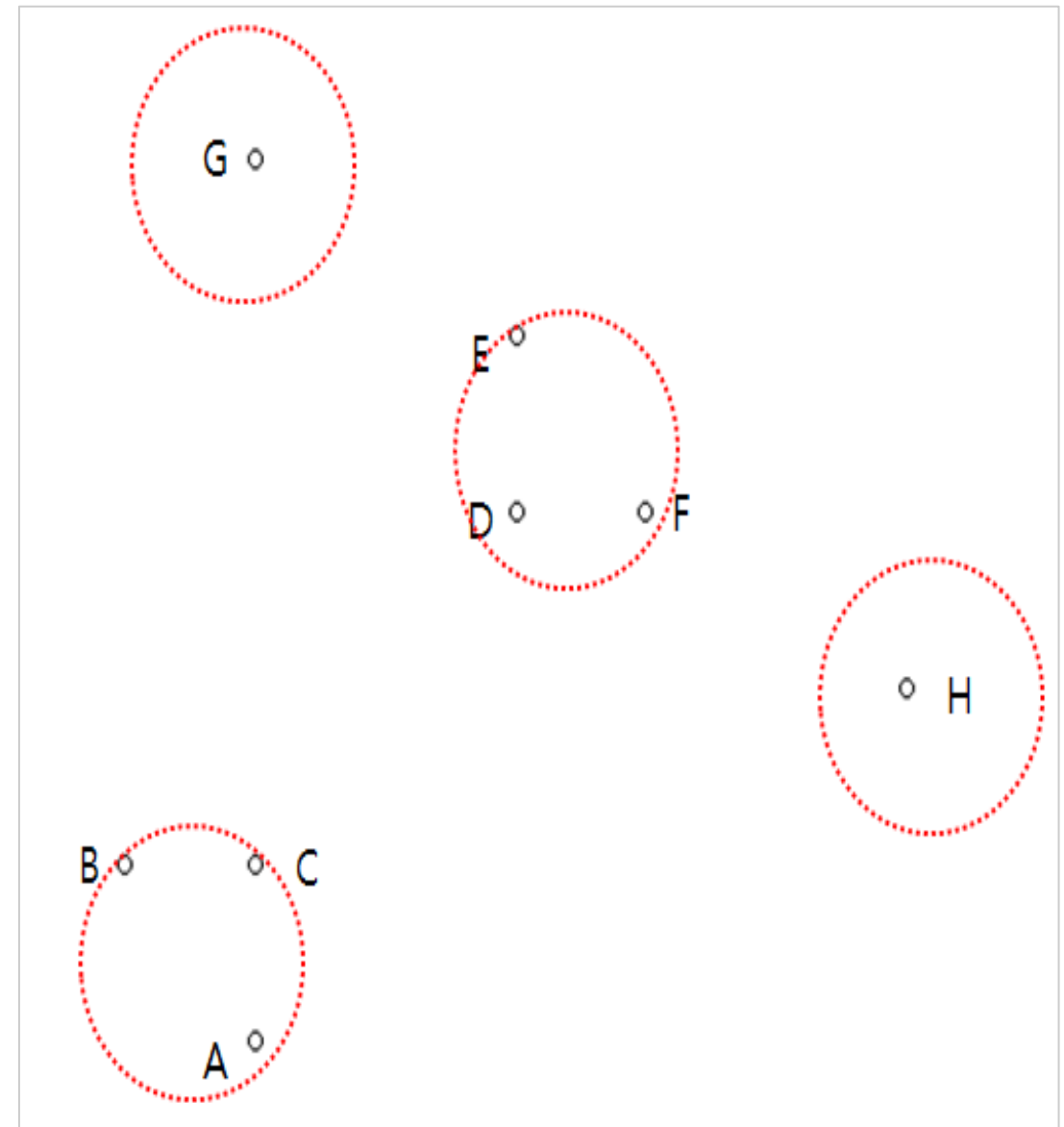
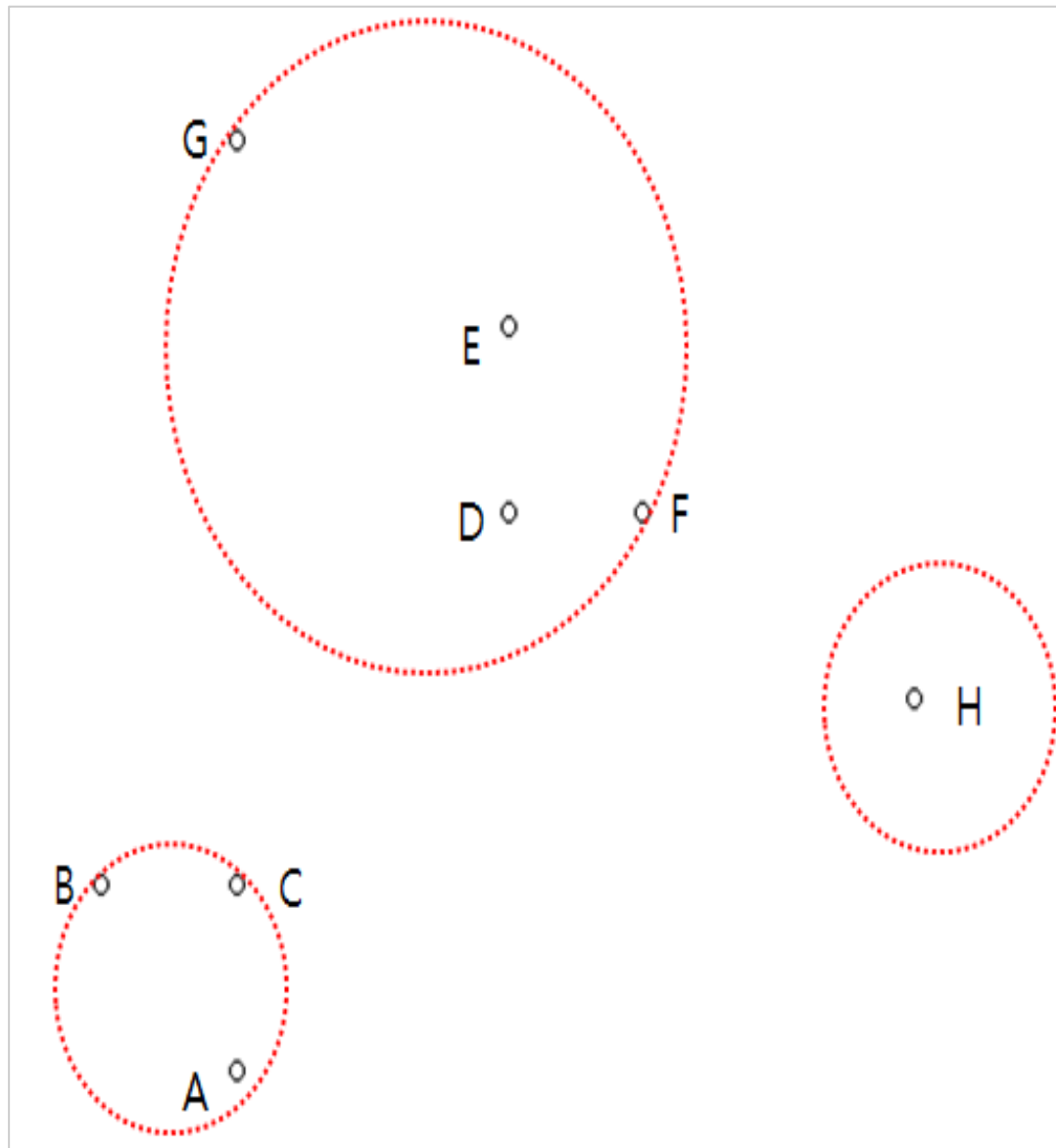
정의

- ▶ 클러스터링(군집화: clustering)란 성격이 비슷한 항목들을 그룹으로 묶는 작업을 말한다
- ▶ 같은 클러스터내 샘플간의 거리는 작을수록, 다른 클러스터에 속한 샘플과는 거리가 클수록 잘 나누어진 것



클러스터 수, k

- ▶ 적절한 군집의 수(k)를 먼저 찾아야 한다

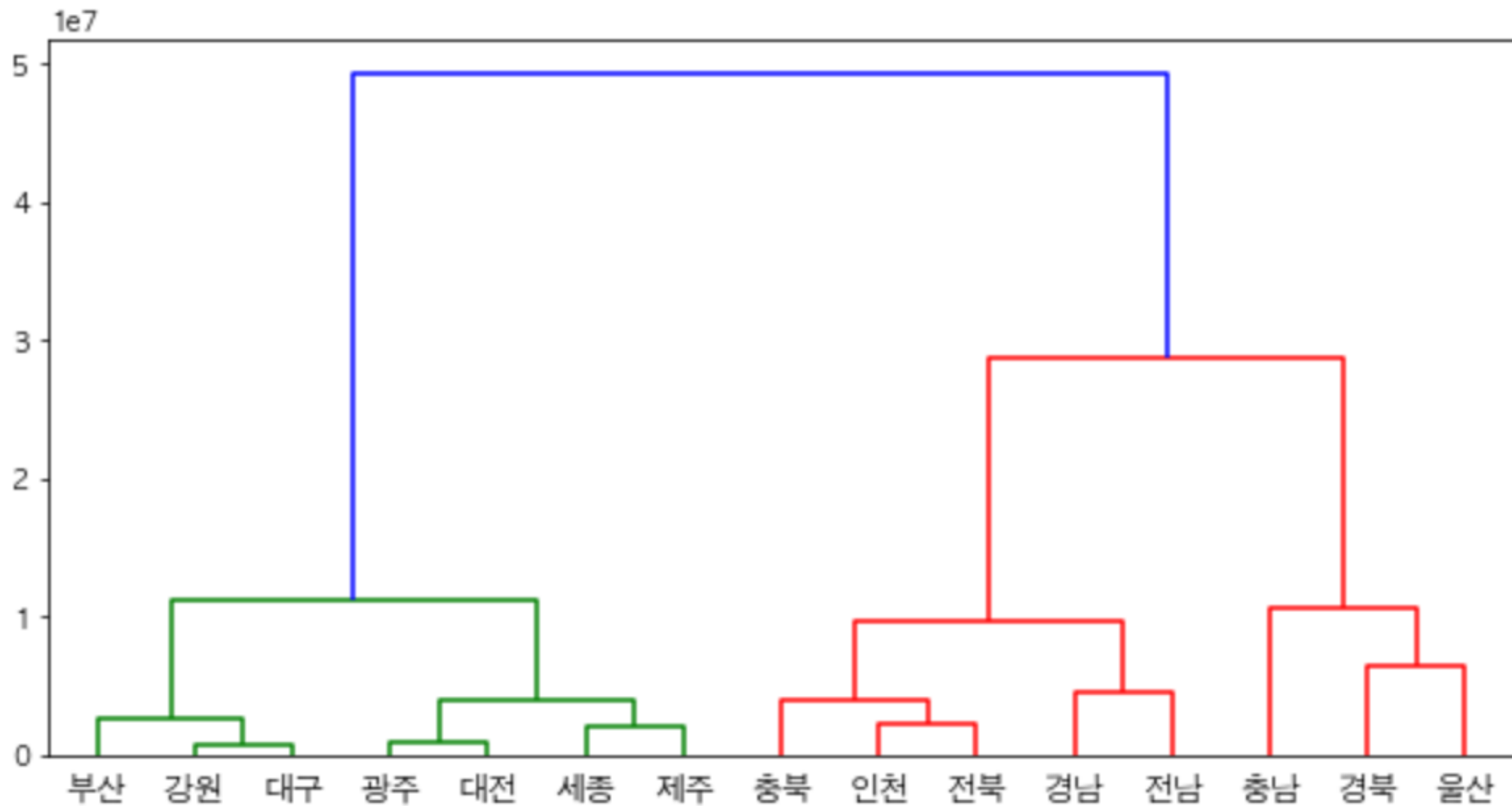


응용

- ▶ 고객 세분화
- ▶ 차원 축소
- ▶ 이상치 탐지
- ▶ Label propagation
 - ▶ 같은 클러스터에 속한 샘플은 레이블이 같다고 가정하여 레이블을 빠르게 생성한다

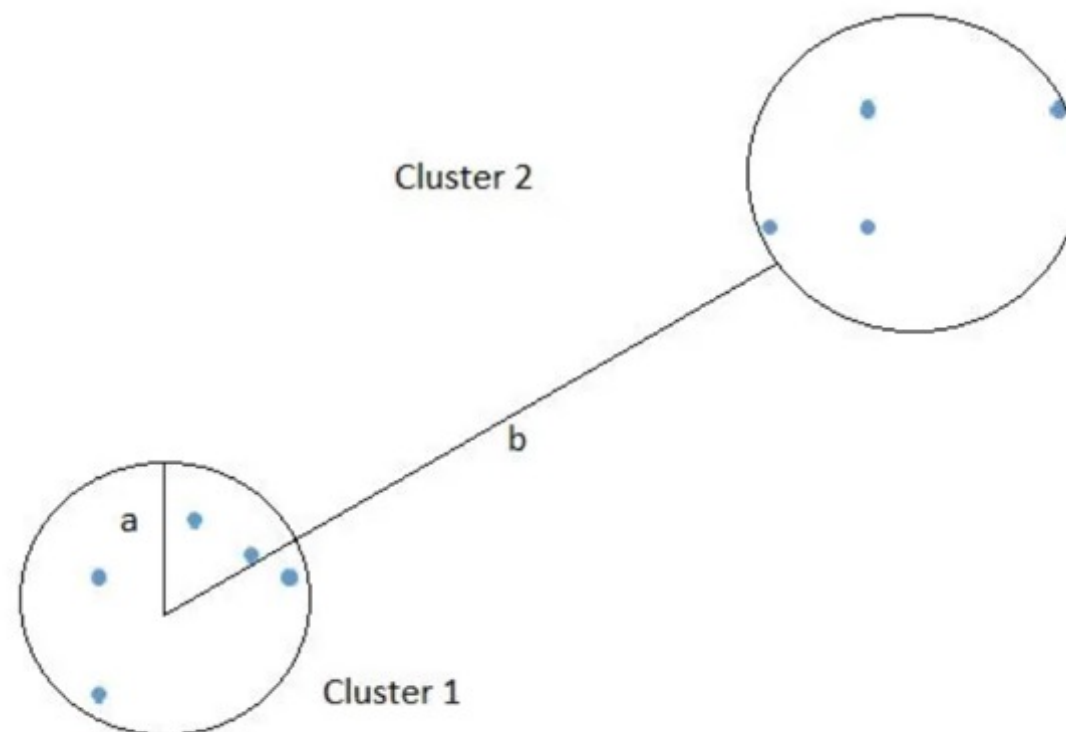
덴드로그램

- ▶ Y 축은 클러스터를 구성하는 반경(거리)을 나타낸다



최적의 클러스터수 찾기

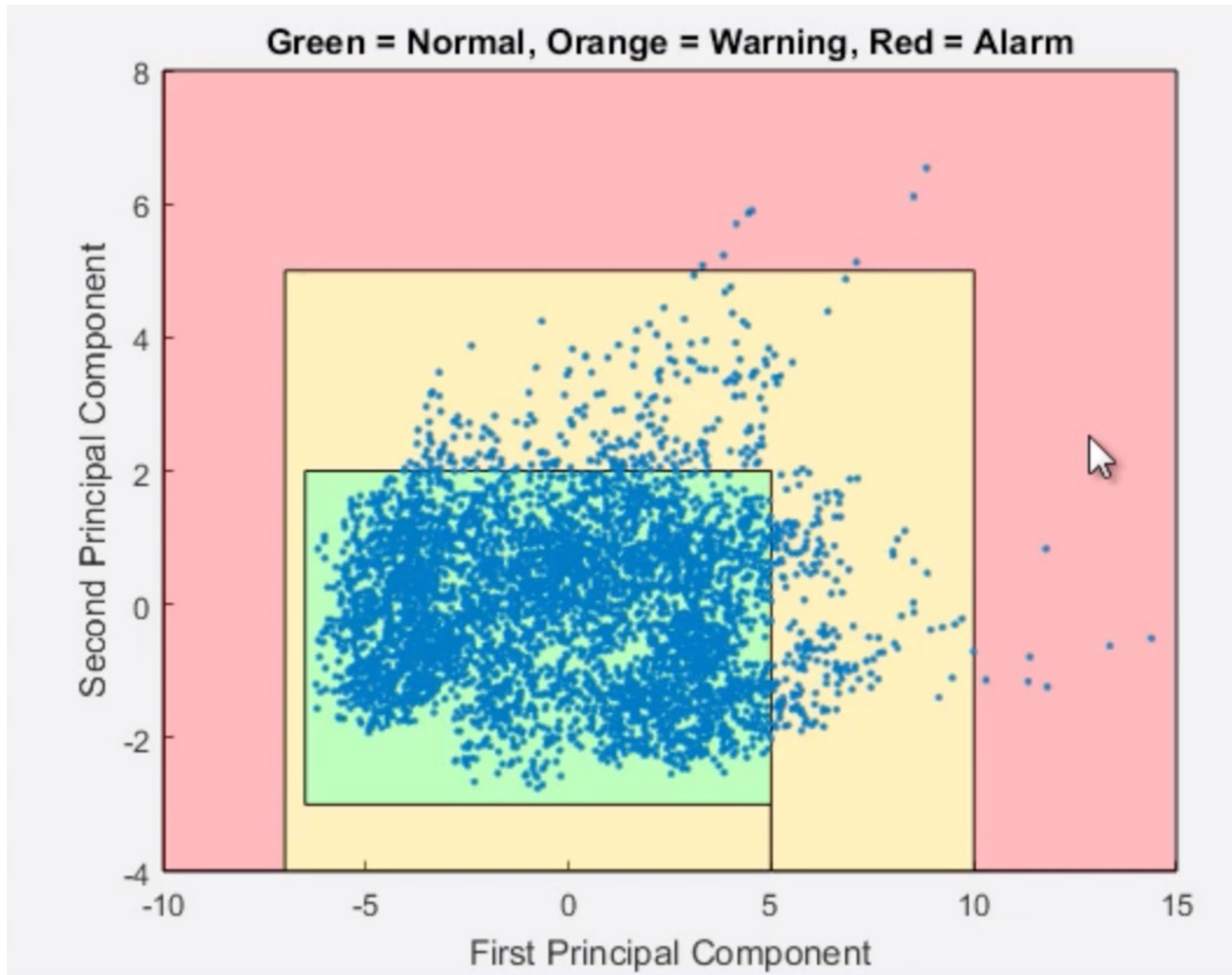
- ▶ 덴드로그램 보기
- ▶ Inertia 변화를 보고 변곡점 찾기
- ▶ 실루엣 지수가 큰 곳 찾기
 - ▶ 각 샘플별로 실루엣 지수를 구하고 이의 평균치를 본다
 - ▶ 샘플별 실루엣 지수: $(b-a) / \max(a, b)$
 - ▶ a: 동일 클러스터 내 다른 샘플들과의 평균 거리
 - ▶ b: 가장 가까운 다른 클러스터까지의 평균 거리



이상치 검출

- ▶ Outlier Detection
- ▶ 이상치 검출 방법
 - ▶ 단변수이면, 예를 들어, 3시그마 이상 떨어진 샘플 찾기
 - ▶ PCA 공간에서 멀리 떨어진 샘플 찾기
 - ▶ 클러스터링과 시각화로 이상치 그룹 찾기
 - ▶ 이상치를 지도학습으로 훈련하여 분류하는 방법

PCA를 이용하는 방법



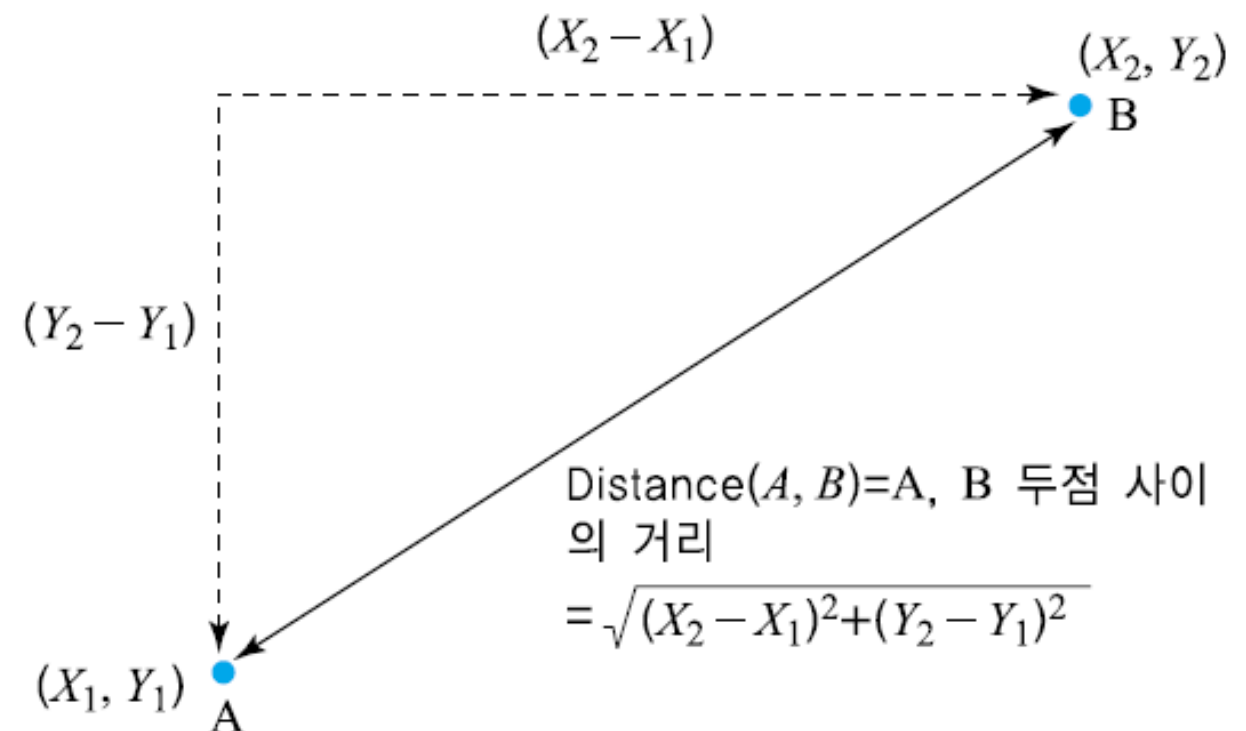
유사도와 거리

유사도와 거리

- ▶ 머신러닝은 유사도와 거리를 정의하는 것에서 출발한다
- ▶ 거리가 가까운 샘플이 비슷한 성격을 갖는다는 가정을 사용
- ▶ 유사도 $s(\text{similarity})$ 는 $0 \leq s \leq 1$ (1에 가까울수록 유사도 높음)
- ▶ 유사도의 상대 개념으로 거리(distance) 사용
 - ▶ $d = 1 - s$

유클리디언 거리

- ▶ 기하학의 공간(space) 상의 직선 거리
 - ▶ 유클리디언 (Euclidian) 거리

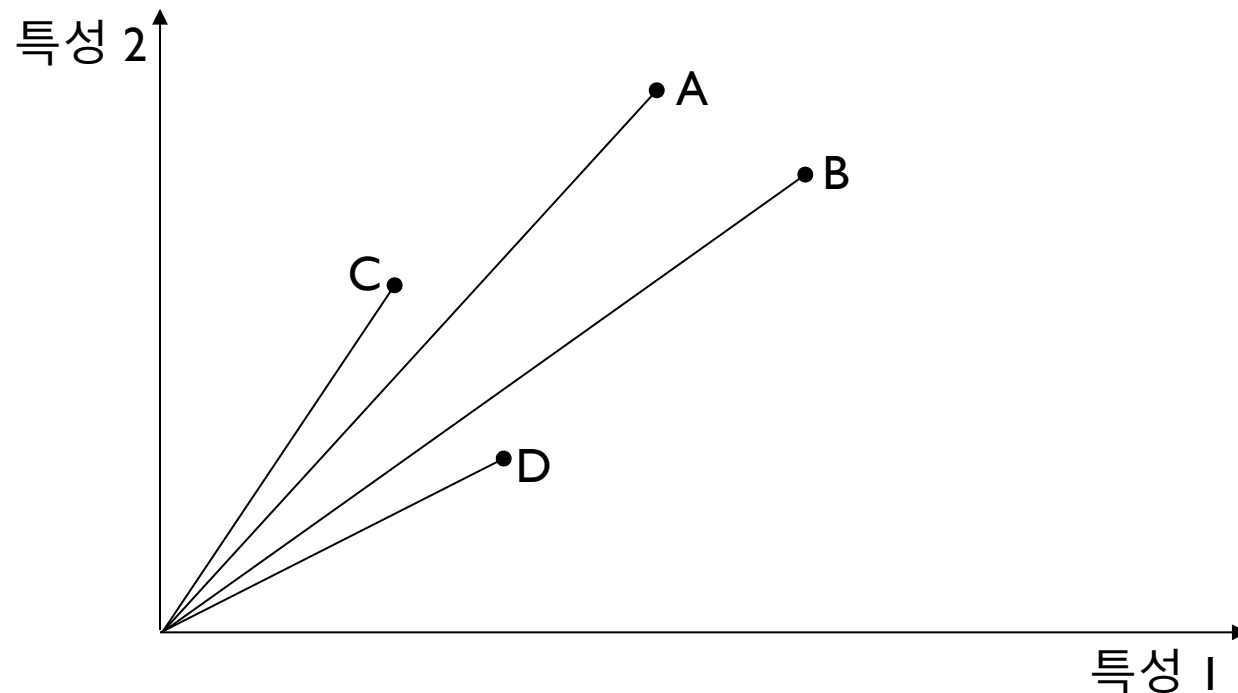


- ▶ n 차원 공간상의 두 점의 거리

$$\sqrt{(p_1 - q_1)^2 + (p_2 - q_2)^2 + \dots + (p_n - q_n)^2} = \sqrt{\sum_{i=1}^n (p_i - q_i)^2}$$

코사인(cosine) 유사도

- ▶ 공간상의 두 점이 만드는 각도를 기준으로 유사도를 측정하는 방법 (-1 ~ +1 사이의 값을 갖는다)
- ▶ 공간상 거리가 멀어도 두 점이 가리키는 방향이 같으면 서로 비슷하다고 보는 것



$$s_{\cos}(x, y) = \frac{X \cdot Y}{|X||Y|}$$

- ▶ A와 C가 가깝고 B와 D가 가깝다고 정의

자카드 유사도

- ▶ 어떤 두 항목이 겹치는 부분의 절대량만을 보지 않고, 두 항목의 공통 부분이 얼마나 많은지를 고려하여 이에 대한 상대적인 값을 유사도로 사용하는 것

$$S_{Jaccard}(X, Y) = \frac{|X \cap Y|}{|X \cup Y|}$$

- ▶ A, B, C가 각각 지난해 본 영화의 총 개수가 20편, 50편, 200편
- ▶ $J(A, B) = 5 / (20 + 50 - 5) = 0.076$
- ▶ $J(A, C) = 10 / (20 + 200 - 10) = 0.047$
- ▶ 즉, $0.076 > 0.047$ 이므로 A와 B가 더 가깝다고 할 수 있음

딥러닝

딥러닝 모델

- ▶ **인공 신경망(Artificial Neural Network:ANN)**
 - ▶ 여러 계층(layer)의 매트릭스 연산을 사용하며 활성화 함수를 도입
- ▶ **MLP, CNN, RNN이 기본**
 - ▶ Multi-Layer Perceptron
 - ▶ Convolution Neural Network
 - ▶ Recurrent Neural Network –LSTM, GRU 등의 변형
- ▶ **Transformer, Graph Network 모델**
 - ▶ 복잡한 현상을 모델링할 수 있다

딥러닝 특징

- ▶ 기존 머신러닝 모델은 전처리가 필수이다
- ▶ 딥러닝은 전처리의 비중을 줄이고 (특성 공학을 줄이고) 모델이 특성을 스스로 추출하도록 한다
- ▶ 딥러닝은 **표현 학습**을 수행한다고도 한다
 - ▶ Representation learning
 - ▶ 예측에 필요한 특성(feature)을 데이터를 보고 스스로 찾아내는 것
- ▶ 딥러닝은 비정형 데이터 분석에 유용하다
 - ▶ 이미지, 텍스트, 사운드 등 동일한 성격의 다량의 데이터

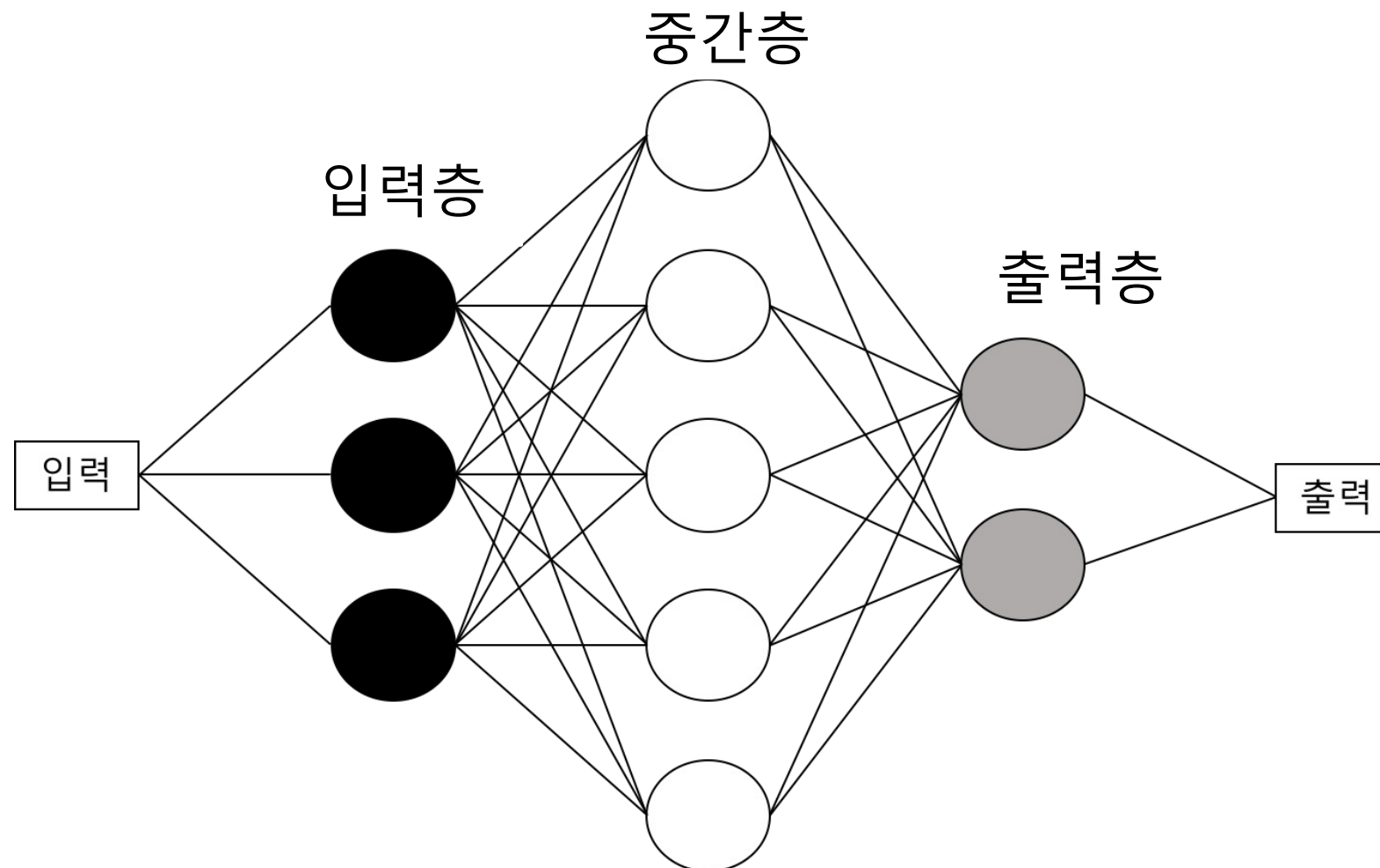
딥러닝 특징

- ▶ 전이학습 (transfer learning)
 - ▶ 신경망의 중요한 장점은 다른 데이터로 학습한 모델을 재사용하기가 쉽다
 - ▶ 전이학습을 이용하면 학습시간도 줄일 수 있고 적은 데이터로도 성능이 좋은 모델을 만들 수가 있다
- ▶ 딥러닝의 단점
 - ▶ 딥러닝은 블랙박스 형태로 동작한다. 성능은 우수하나 내부 동작 이유를 알 수는 없다

다층 퍼셉트론(MLP)

MLP 구조

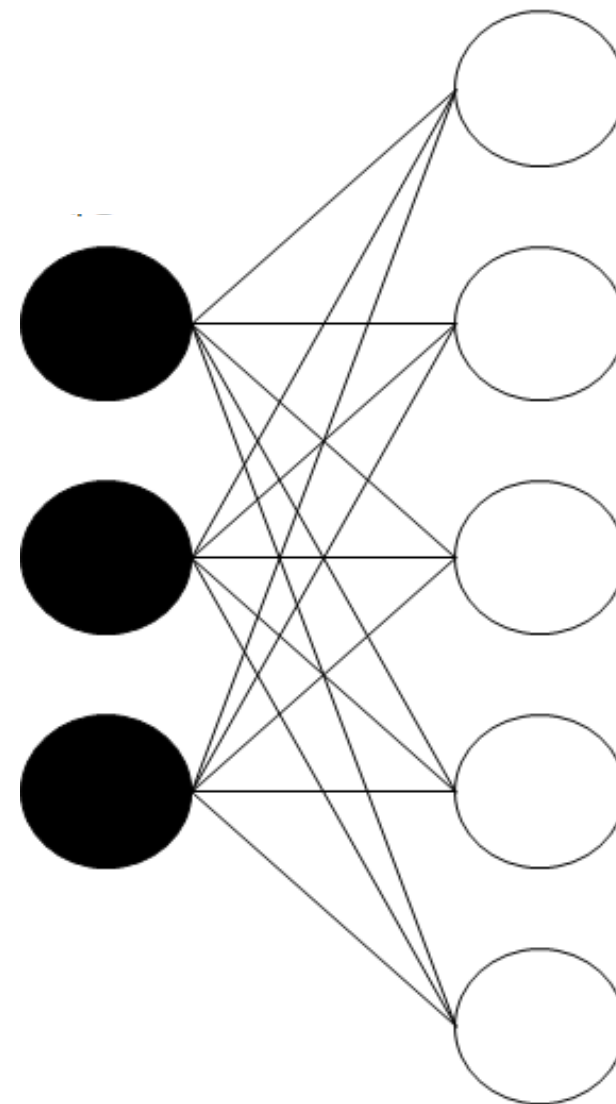
- ▶ 입력 데이터에 가중치를 곱하여 중간 출력을 만들고 이 중간 출력에 다시 가중치를 곱하여 다음 계층의 입력으로 사용하는 구조



전결합망

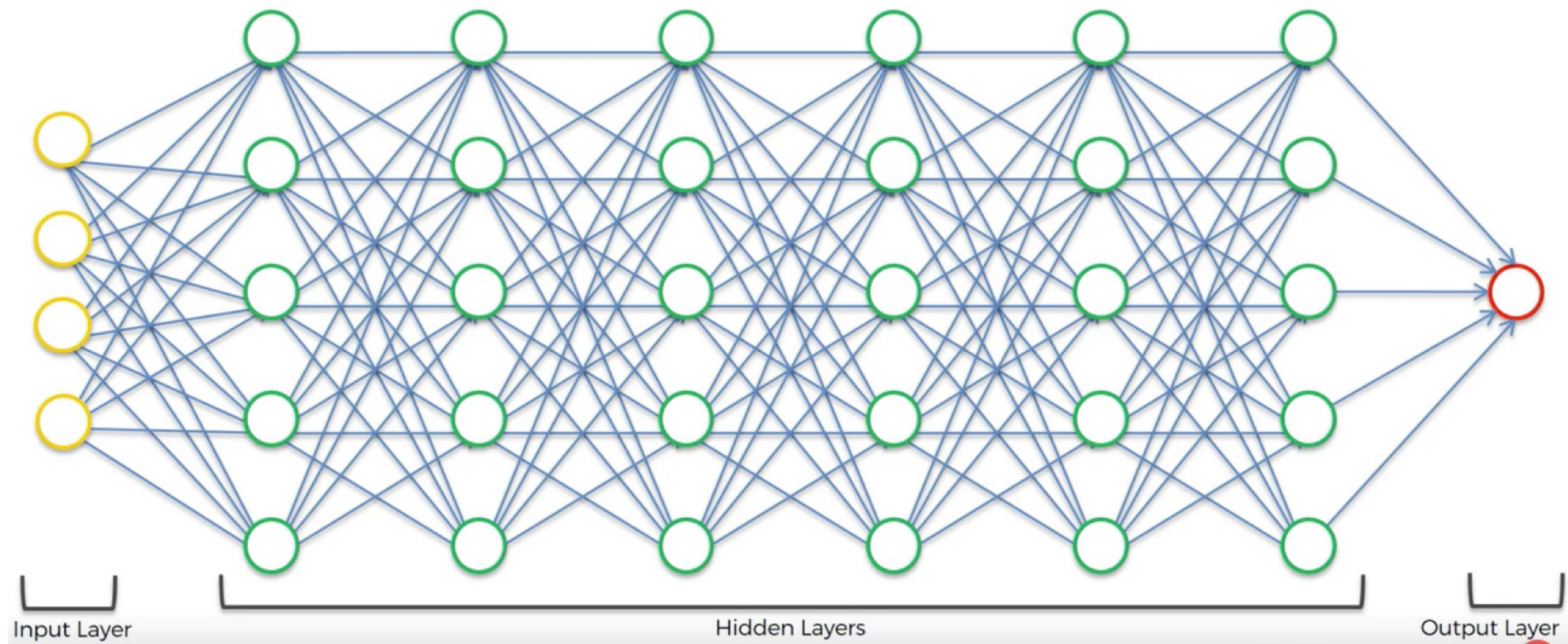
- ▶ MLP는 전결합망 (FC: fully connected) 네트워크를 사용한다
- ▶ 전결합망
 - ▶ 각 계층의 모든 출력이 다음 단계의 모든 입력으로 연결되는 가중치를 갖는 구조
 - ▶ 스칼라 값인 편이(bias)가 더해진다.

$$Y = \Sigma(W_{ij} \times X_{ij} + b_i)$$



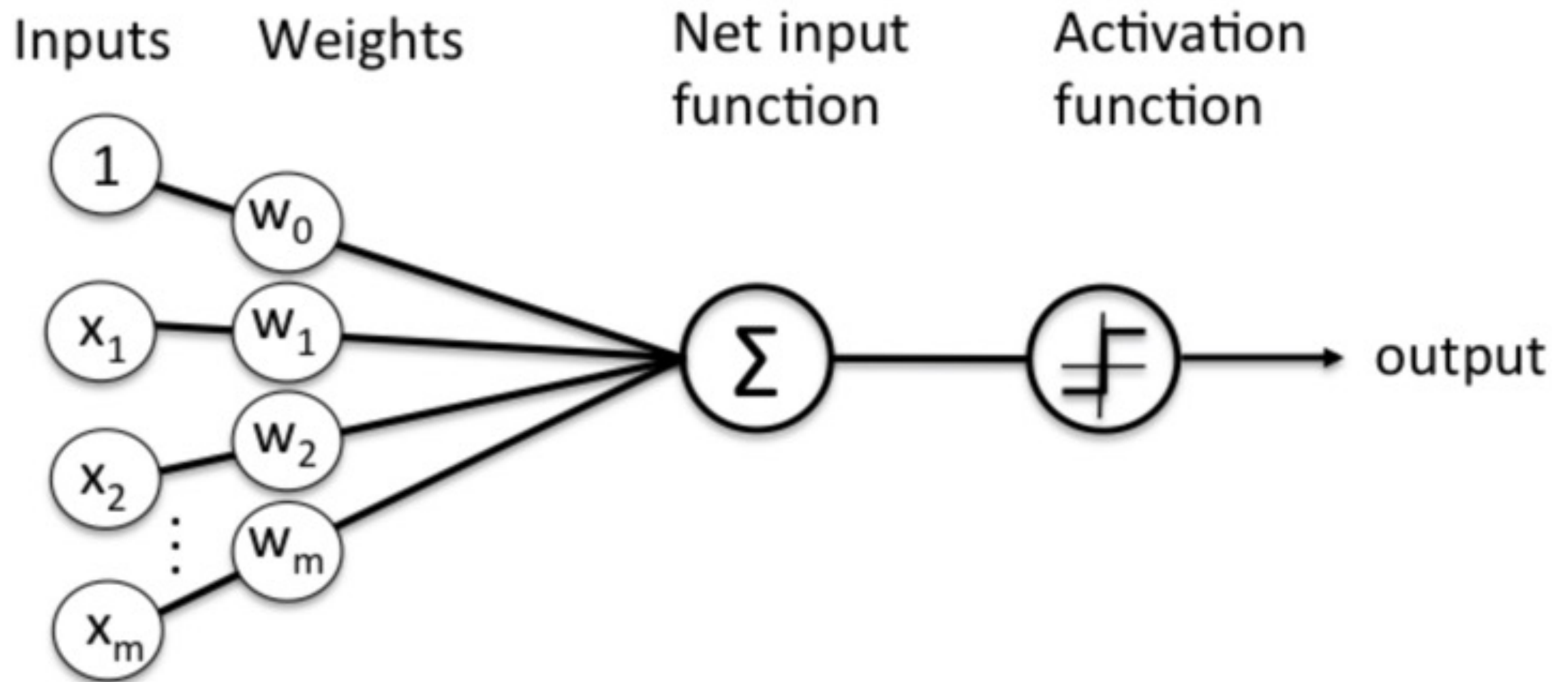
MLP 예

- ▶ 히든 계층이 6개인 예 (MLP는 2개 층만 사용해도 동작한다)



활성화 함수

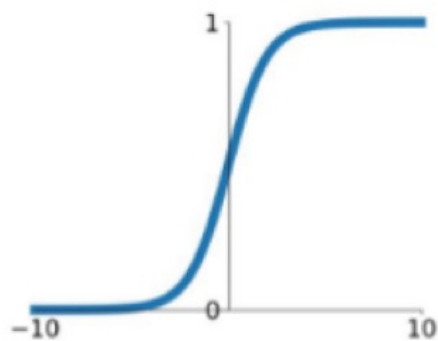
- ▶ 모든 가중합은 다음 단계로 넘어가기 전에 활성화 함수를 통과한다



활성화 함수 종류

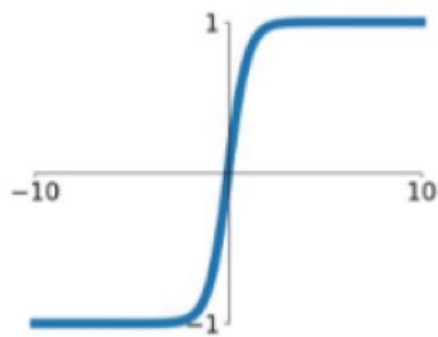
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



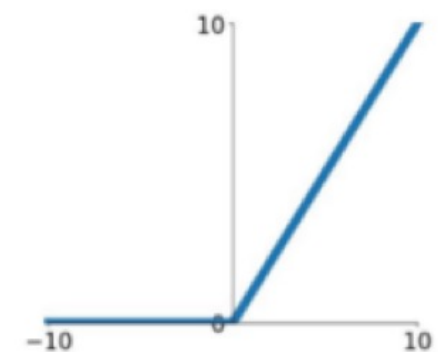
tanh

$$\tanh(x)$$



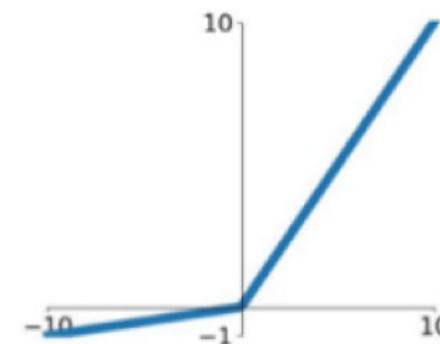
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$

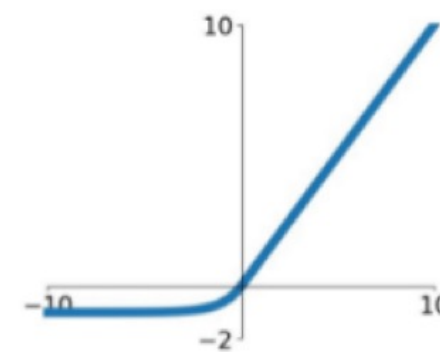


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

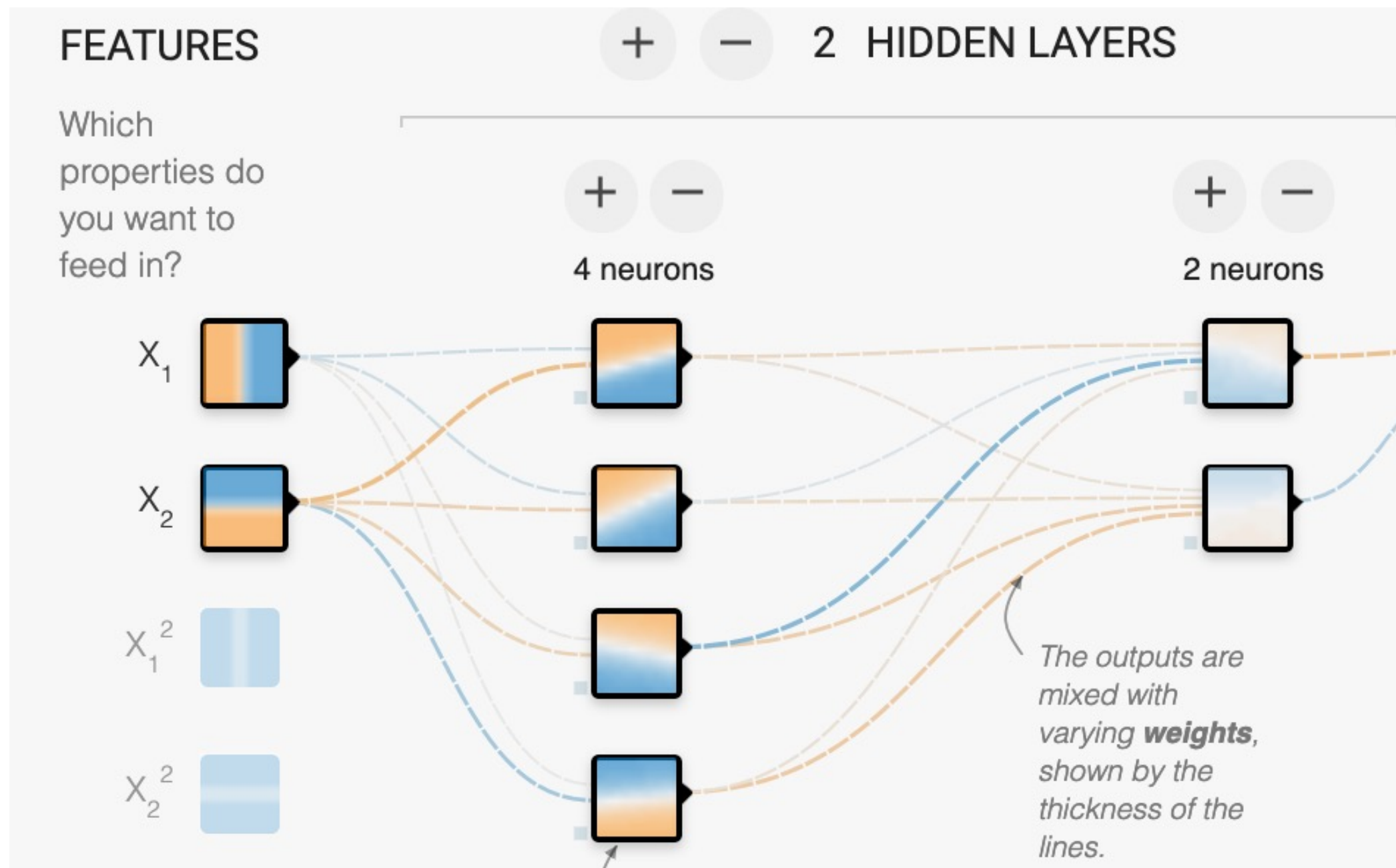
ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



플레이그라운드

- ▶ MLP의 동작 시뮬레이션 도구
 - ▶ playground.tensorflow.org



CNN

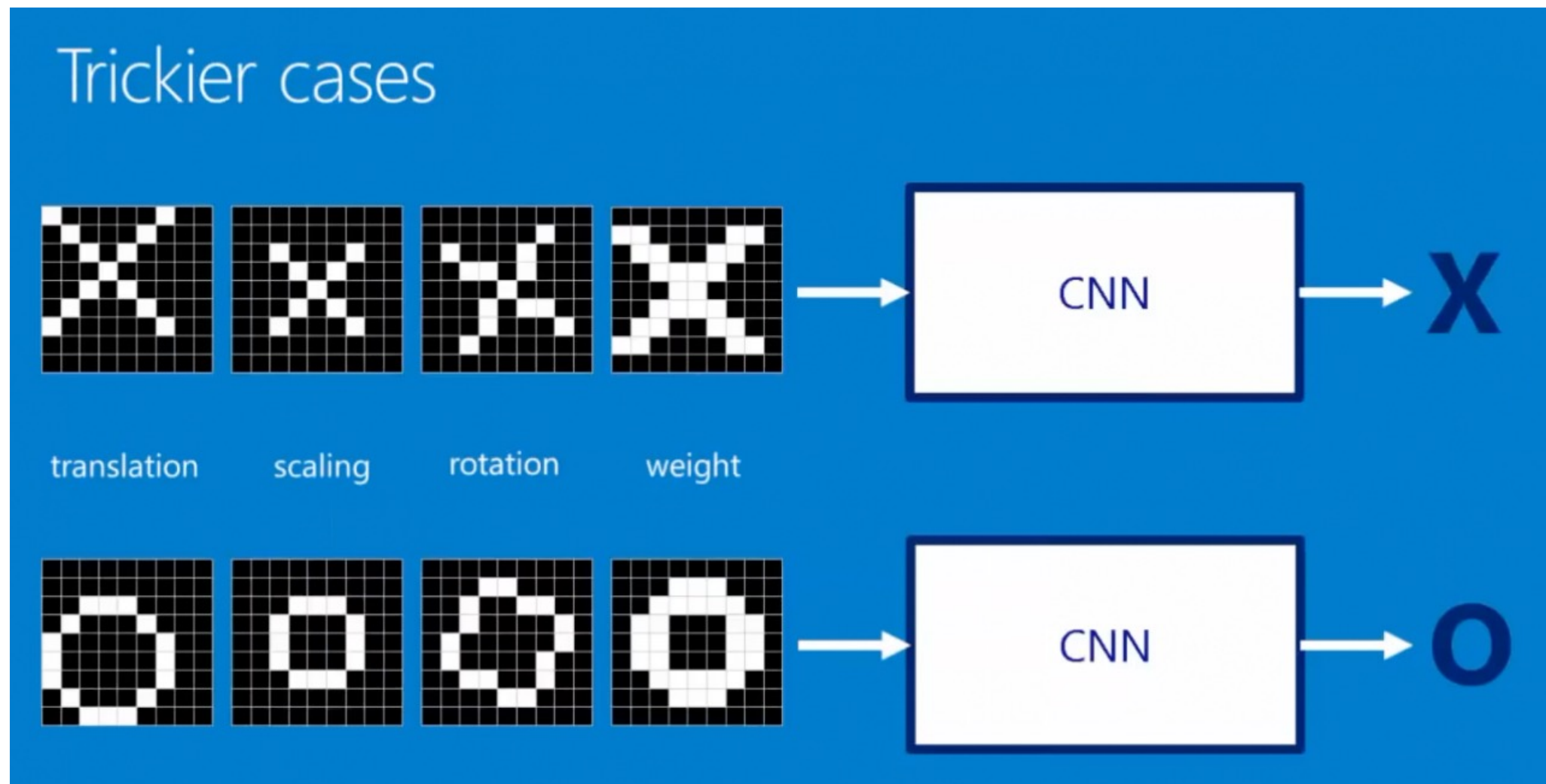
합성곱신경망 (CNN)

- ▶ 아래와 같이 임의의 크기와 모양을 가진 이미지 판독에서는 MLP가 동작하지 않는다
 - ▶ MLP에서는 입력신호 전체를 한번에 다룬다
- ▶ 합성곱 신경망 (Convolution Neural Network, CNN)은 이를 개선하였다



CNN 개념

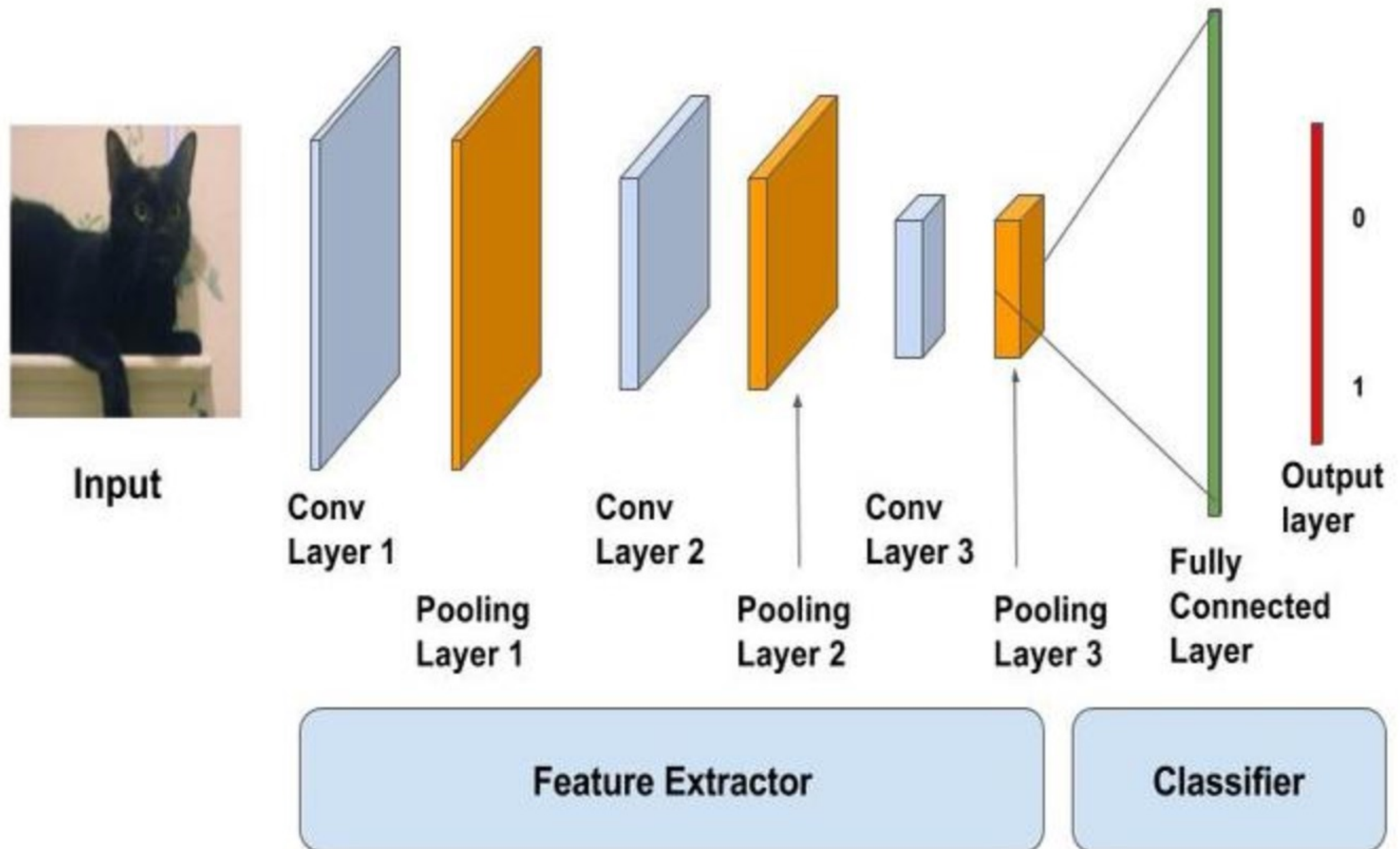
- ▶ 특성의 위치, 굵기, 회전각이 달라도 이미지를 이해하는 원리는 기본 특성들의 조합 정보를 이용하는 것



CNN

- ▶ LeCun 교수가 1998년 제안
- ▶ 이미지 정보의 공간적인 특징을 추출하는데 효과적
- ▶ 구성:
 - ▶ 컨볼루션 계층 – 특성을 추출하는 기능
 - ▶ 풀링 계층 – 특성을 조합하는 기능
 - ▶ 전결합망 – 최종 분류 기능 (MLP 구조를 갖는다)

CNN

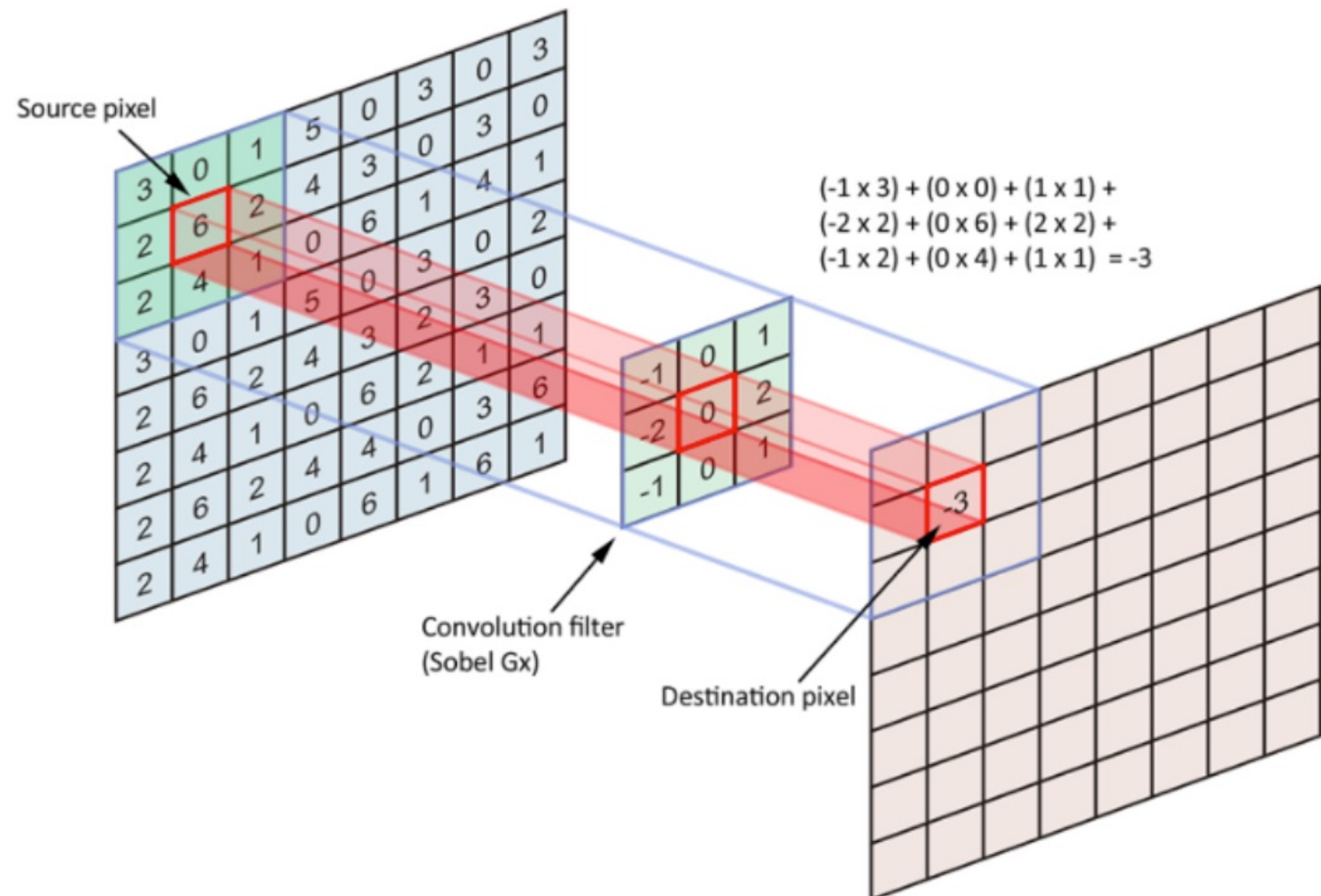


CNN

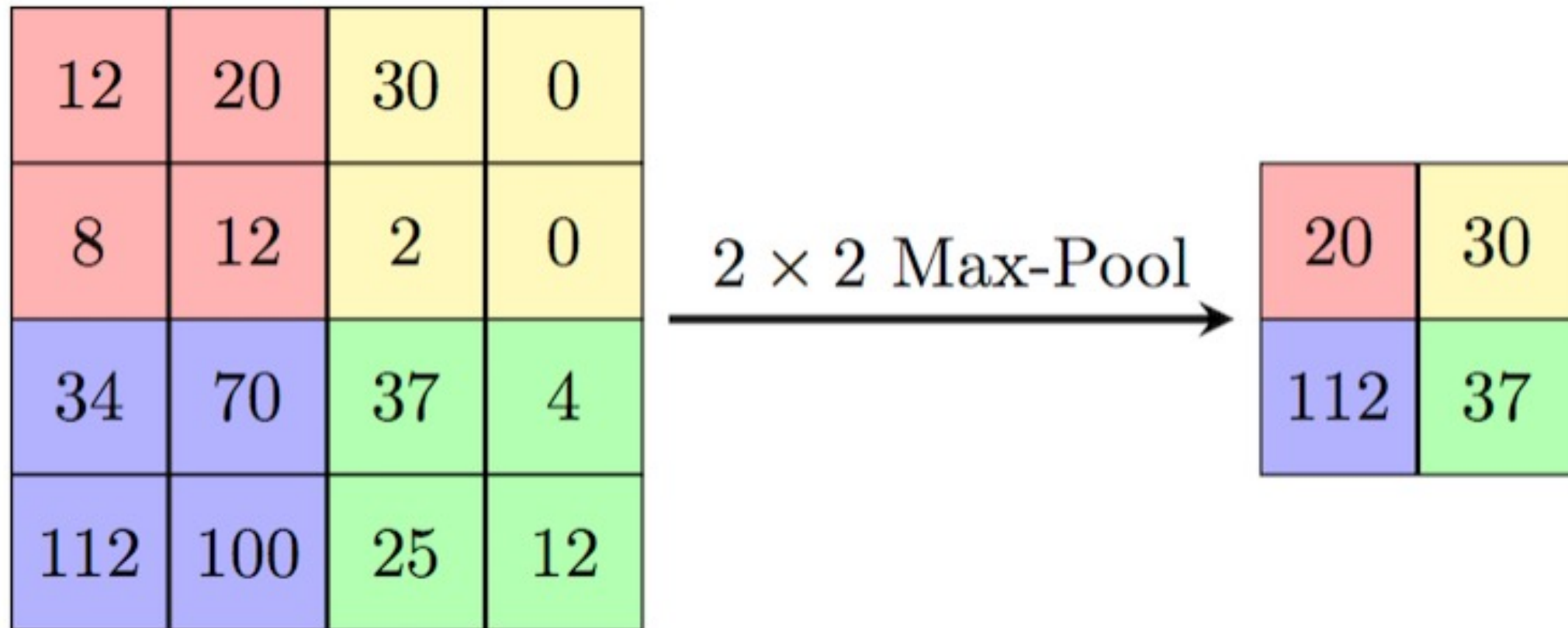
- ▶ 합성곱 필터의 출력을 활성화값(activation)이라고 부른다
- ▶ 활성화값의 집합, 즉 다음 계층의 입력으로 사용되는 배열을 특성맵(feature map)이라고 한다
 - ▶ 합성곱 필터를 커널(kernel)이라고도 하며 커널수가 많을수록 다양한 패턴을 찾아낼 수 있다
 - ▶ 특성맵을 수십~수백 가지를 사용한다 (필터 수로 조정)
- ▶ 풀링 (max pooling)
 - ▶ 합성곱을 수행 결과를 다음 계층으로 전달할 때, 모든 정보를 전달하지 않고 풀링을 통해 일부만 샘플링하여 넘겨준다
 - ▶ 풀링을 하여 좀 더 가치 있는 정보를 축소하여 다음 단계로 넘겨준다
- ▶ CNN은 합성곱 필터와 풀링을 여러 계층 반복 수행한다
 - ▶ 마지막 단계에서는 전결합망을 사용하여 회귀 또는 분류를 수행한다

CNN 동작

- ▶ 합성곱동작
- ▶ 다층 필터링

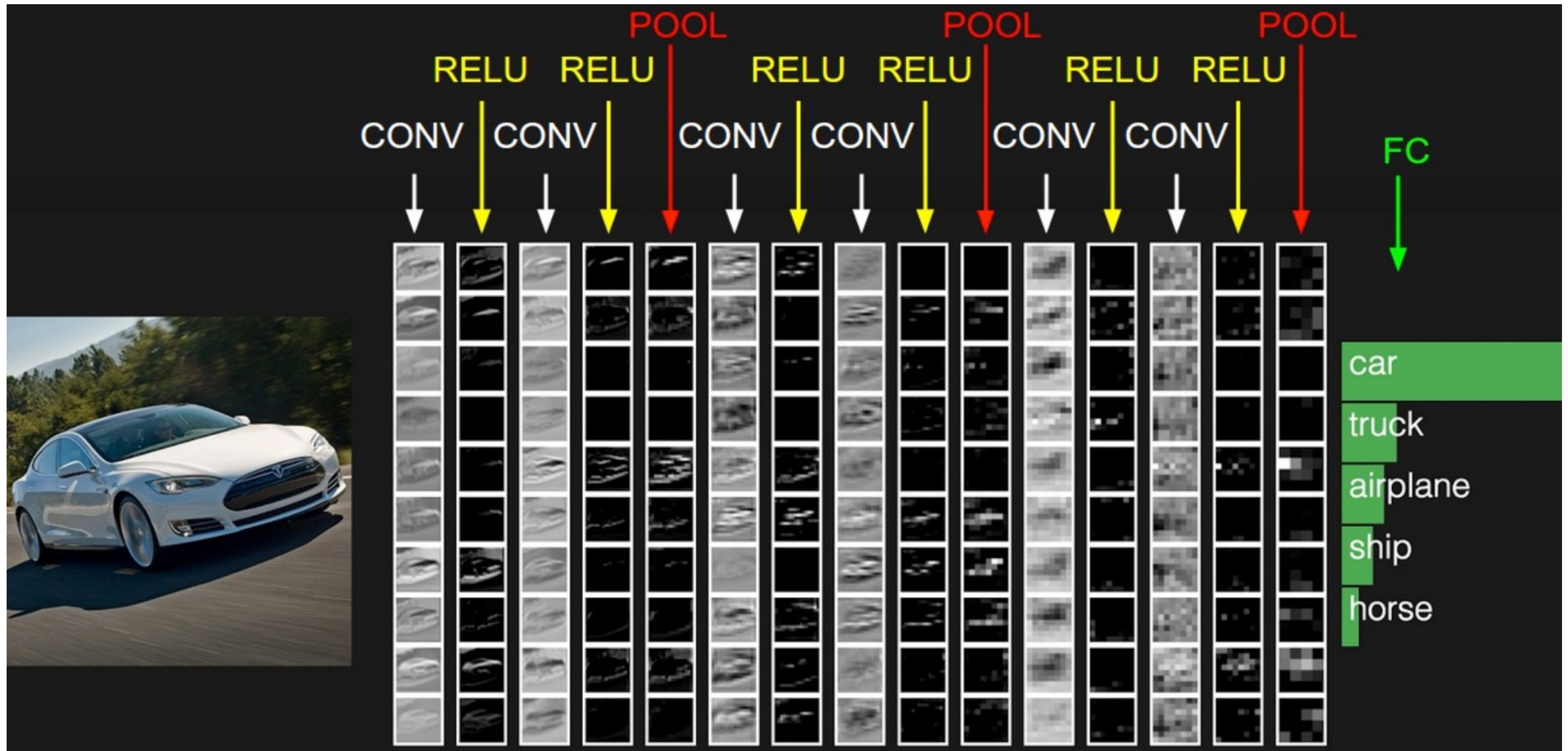


최대풀링(max pooling)



- ▶ 특정한 패턴이 공간상의 어느 위치에 있든 이 활성값이 다음 단계로 넘어가면서 좌우로 조금씩 움직일 수 있다
- ▶ 풀링은 과대적합을 해소하는 데에도 기여한다

이미지 인식

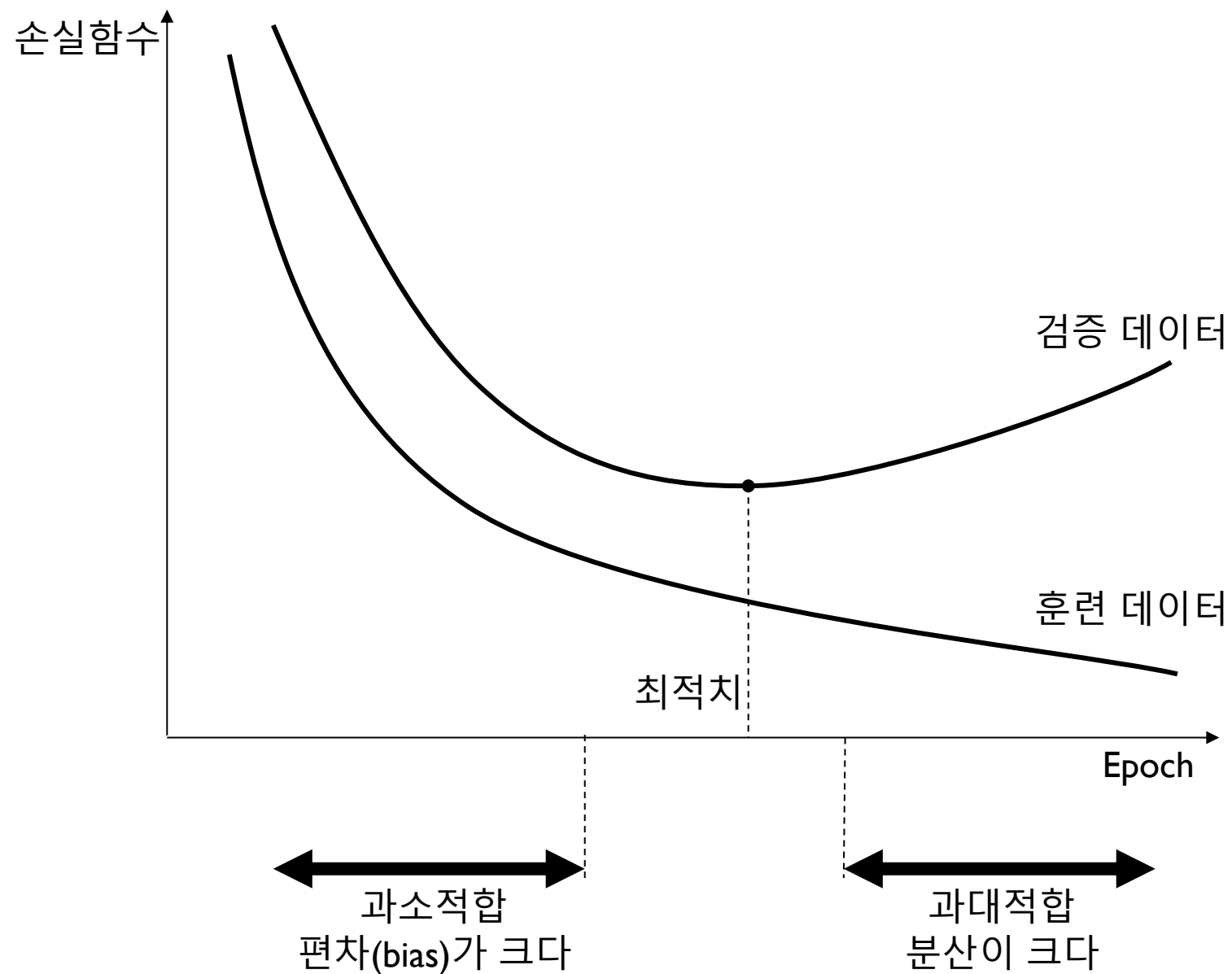


이미지 분석

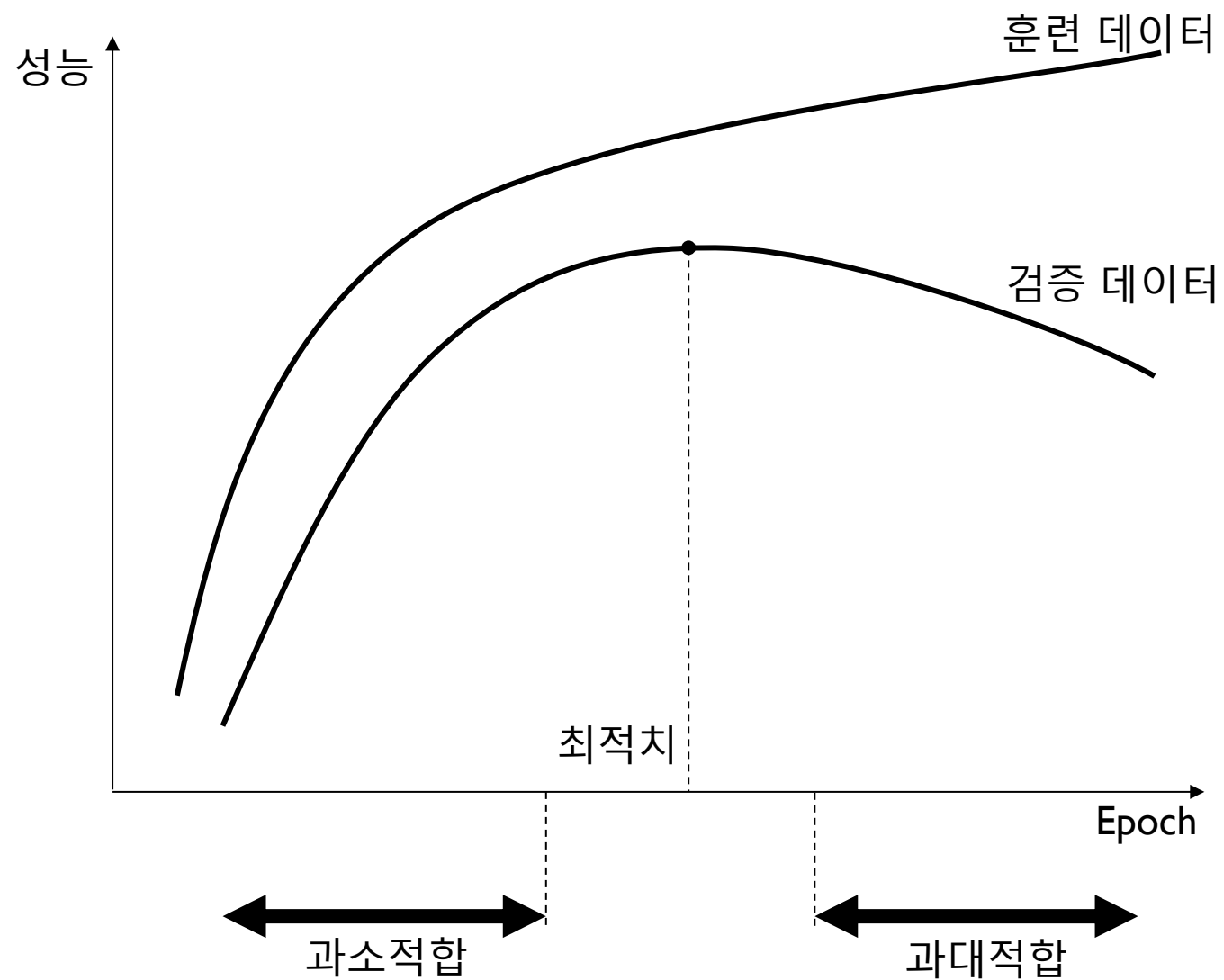
이미지 분류

- ▶ 고양이 강아지 구분 예
 - ▶ 캐글에서 제공하는 데이터는 25,000 장이나 여기서는 3천 장의 사진만 사용한다
 - ▶ 데이터 확장(augmentation)과 전이학습 적용
- ▶ 과대적합
 - ▶ 신경망은 파라미터 수가 많아서 과대적합될 가능성이 항상 존재한다
 - ▶ 과대적합이 발생하면 모델을 단순하게 제한하거나 학습 횟수(epoch)를 줄여야 한다

이포크에 따른 손실함수의 변화

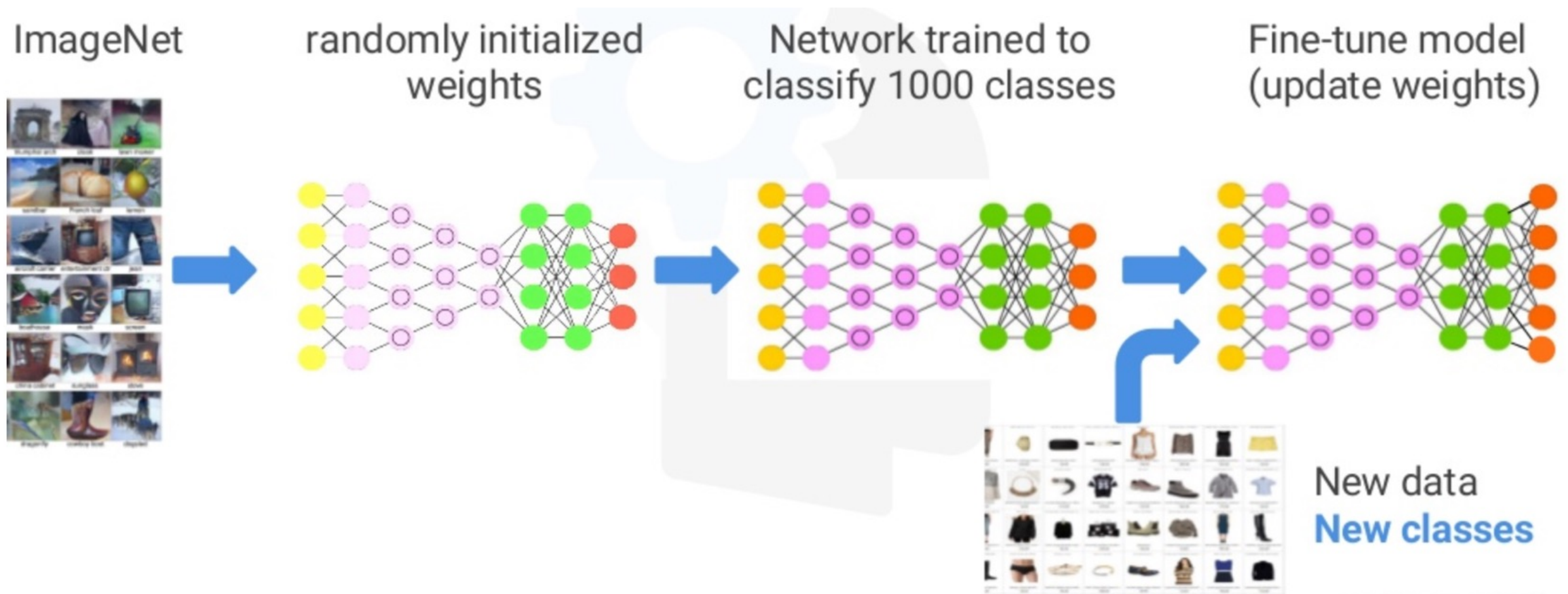


이포크에 따른 성능의 변화

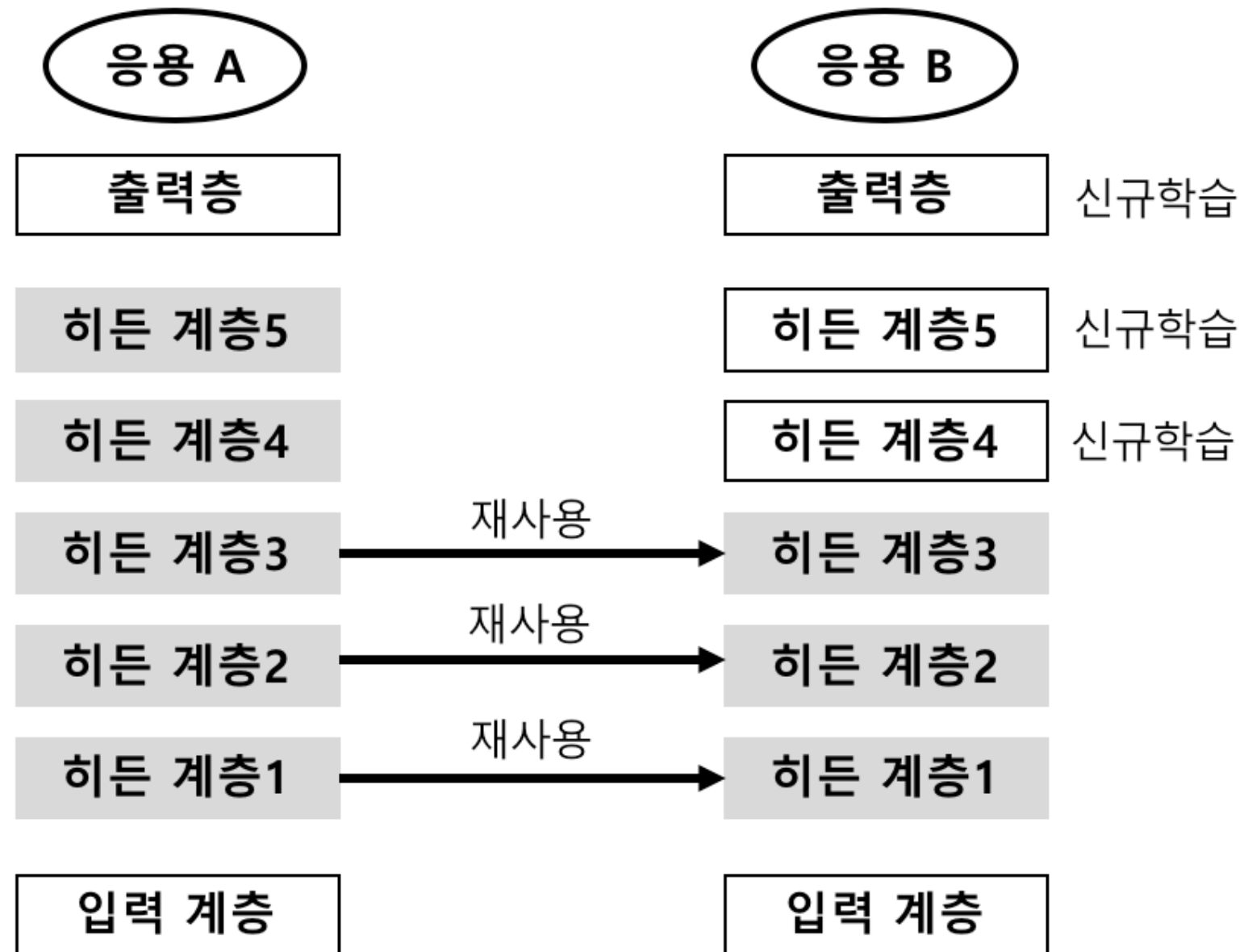


전이학습 (transfer learning)

- ▶ 다른 곳에서 만든 신경망 모델을 재사용하는 것
- ▶ 학습시간도 줄일 수 있고 적은 데이터로도 성능이 좋은 모델을 만들 수 있다



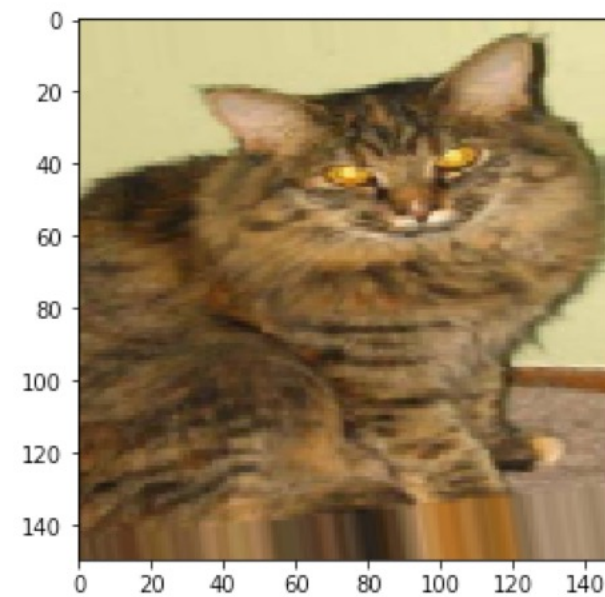
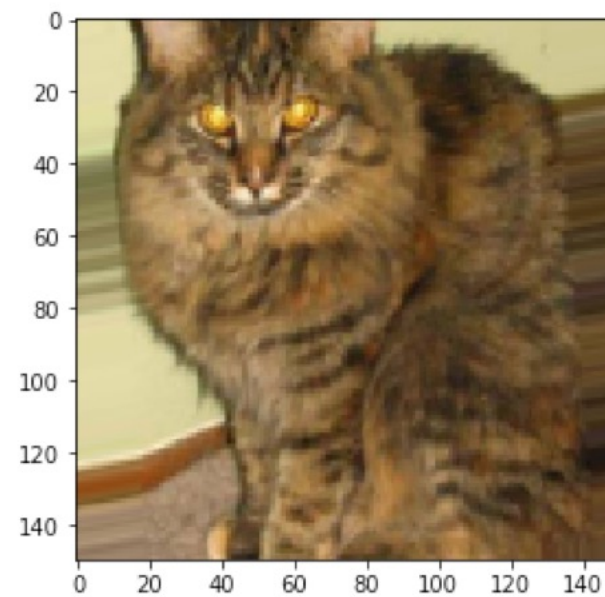
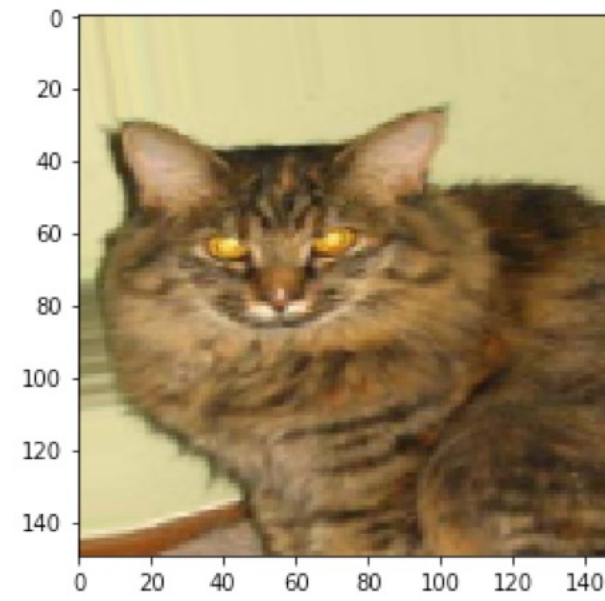
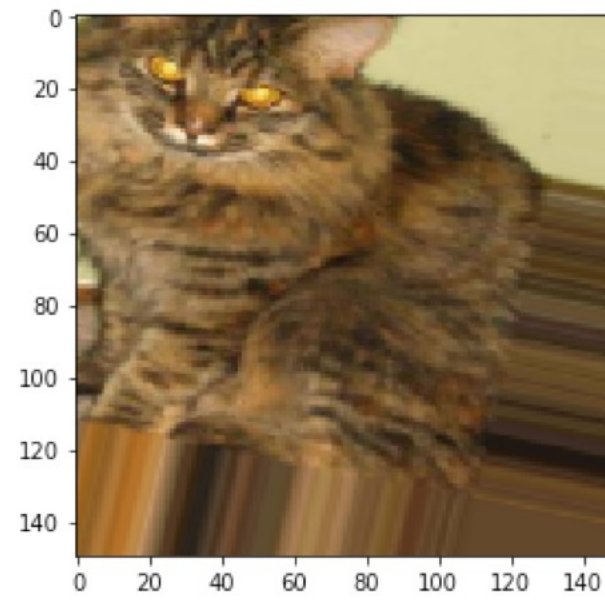
전이 학습



전이 학습

데이터 확장

- ▶ Data Augmentation: 한 장의 이미지를 서로 다른 이미지인 것처럼 활용하는 것



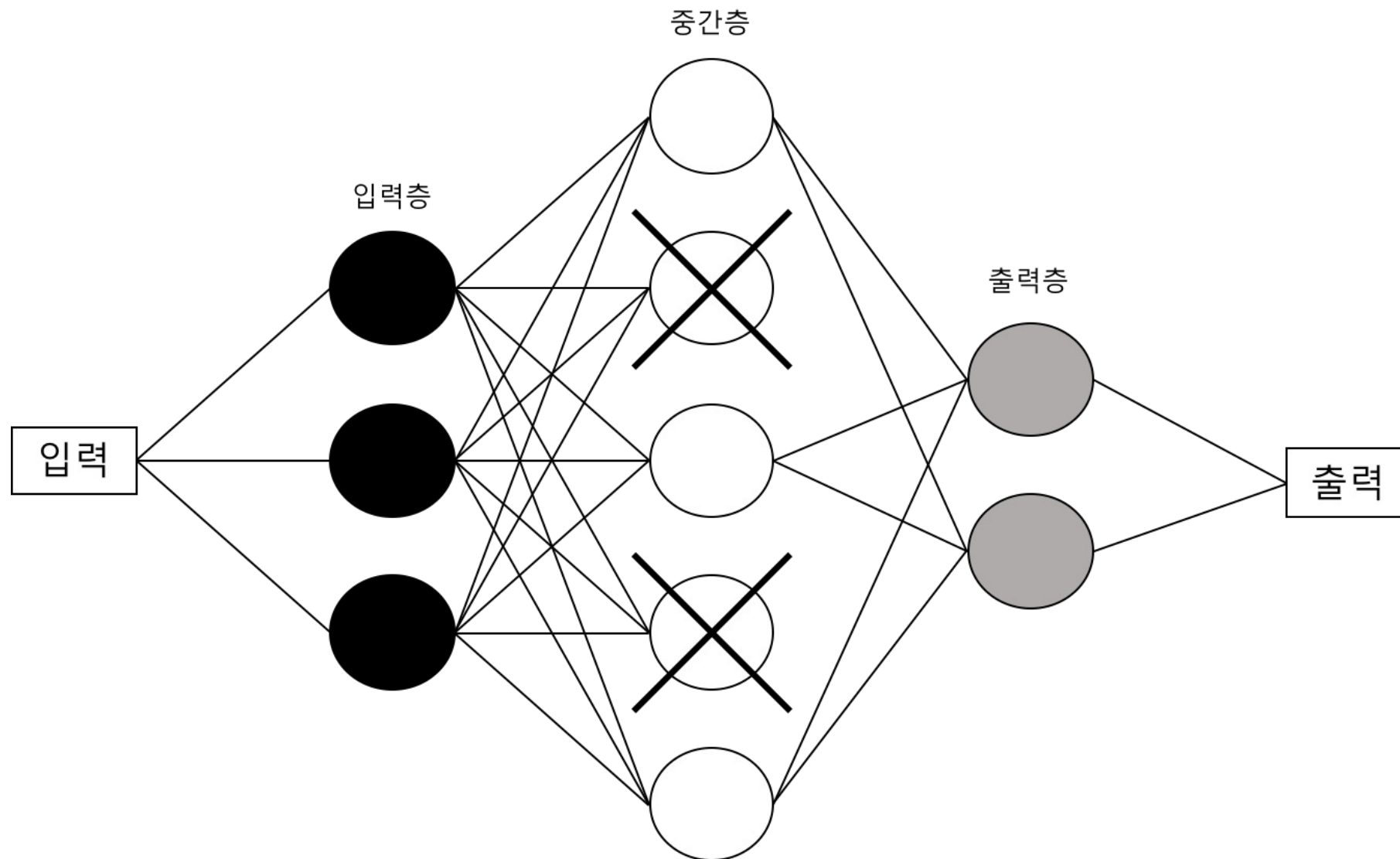
Data Augmentation

▶ 옵션

- ▶ flipping: 이미지를 반전한다
- ▶ random_cropping: 이미지의 일부를 제거한다
- ▶ 확대 및 축소: 이미지의 크기 변화
- ▶ 화이트닝: 주성분 분석으로 주요 부분만 남긴다
- ▶ 정규화: 표준정규화 수행
- ▶ rotation_range: 이미지를 랜덤하게 회전하는 범위를 지정한다 (0-180)
- ▶ width_shift 와 height_shift: 폭과 높이를 랜덤하게 이동
- ▶ shear_range: 깎는 변형 (shearing)의 범위를 정한다.
- ▶ zoom_range: 이미지 내에서 랜덤하게 주밍하는 범위를 정한다.
- ▶ horizontal_flip: 이미지의 반을 수평 방향으로 랜덤하게 뒤집는다
- ▶ fill_mode: 회전이나 상하좌우 이동 후에 생기는 빈 영역을 채우는 옵션

드롭아웃

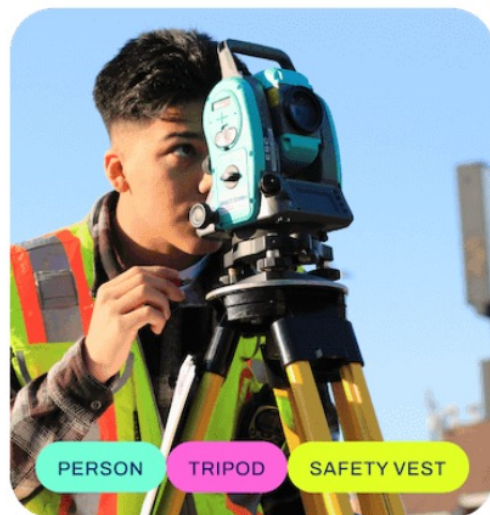
- ▶ 랜덤하게 신경망 유닛을 선택하여 신호를 전달하지 않는 방법
- ▶ 입력과 출력간의 특정 관계를 기억하지 못하게 한다
- ▶ 드롭아웃은 학습을 하는 동안에만 적용되다



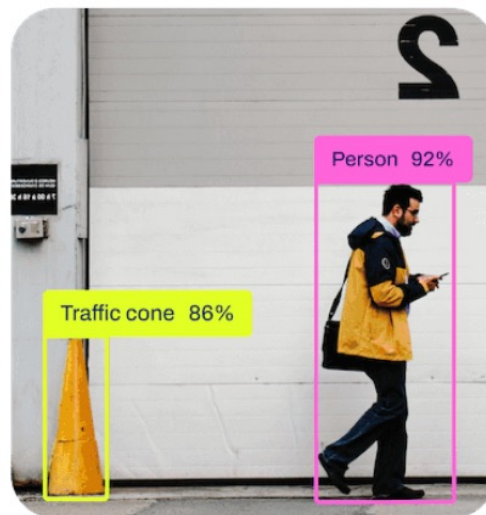
객체 검출

Detect, Segment, Pose

Classify



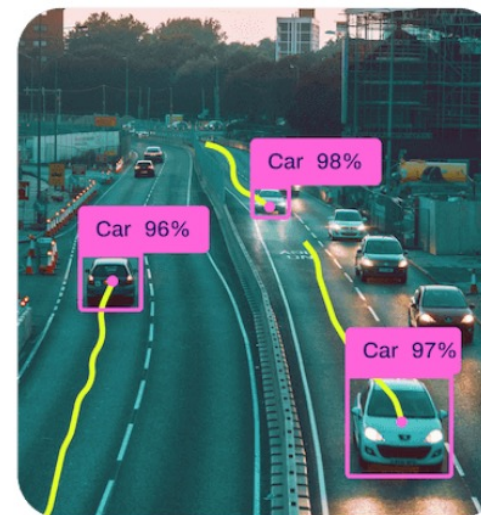
Detect



Segment



Track

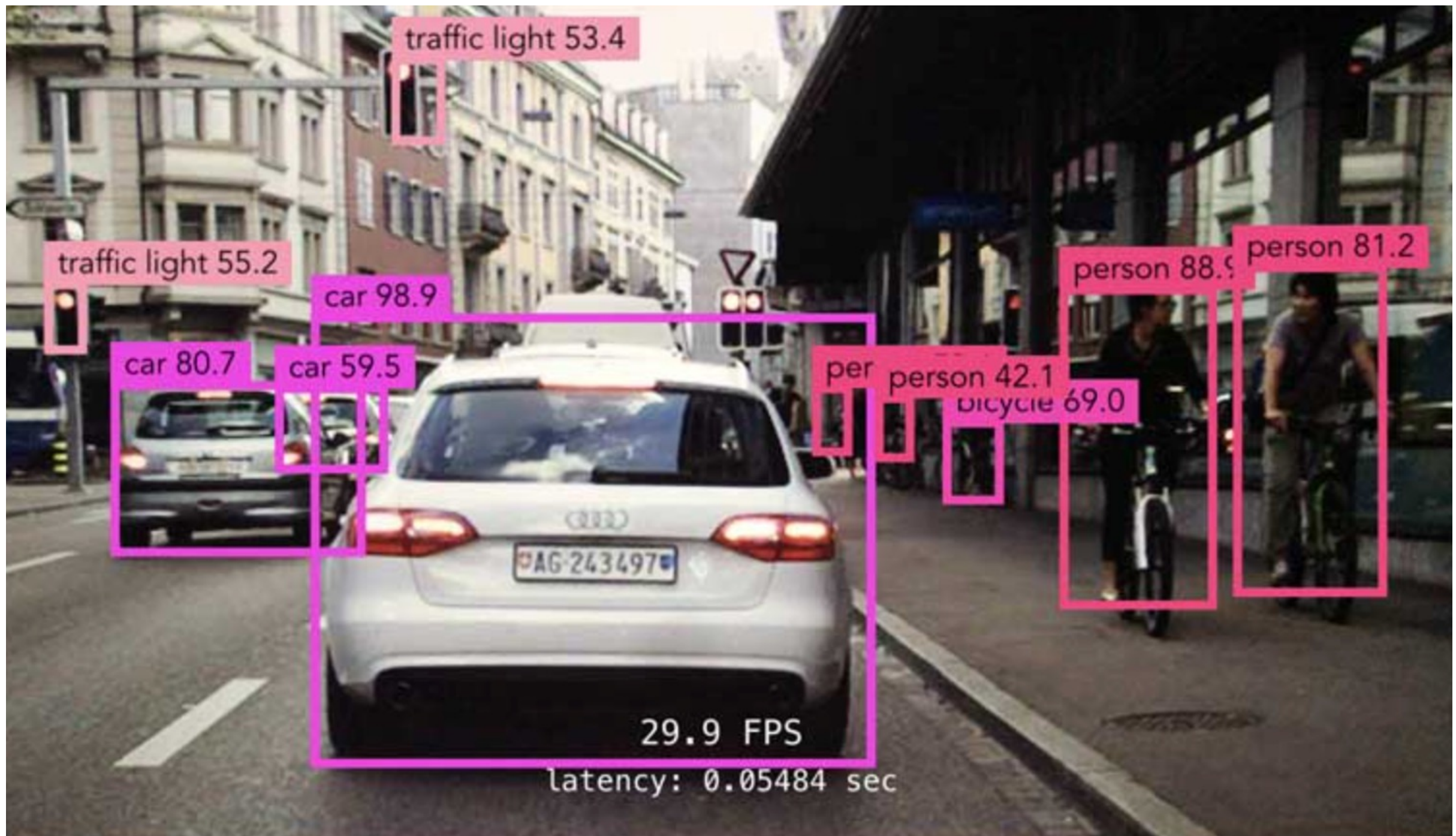


Pose



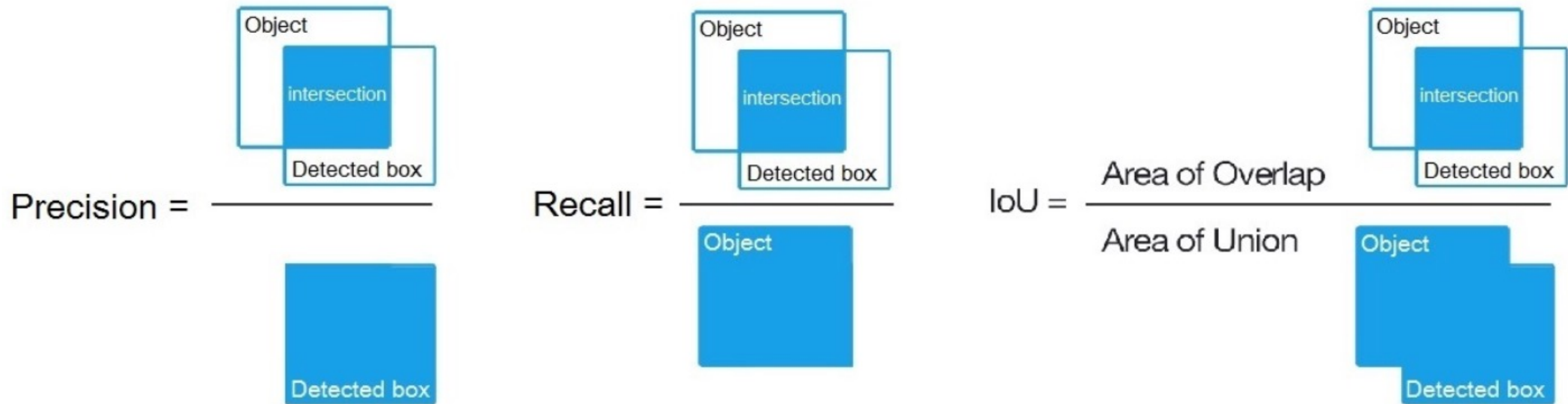
Object Detection

- Confidence score



성능평가 - IOU

- ▶ Intersection over Union
- ▶ 참고: 정밀도(precision), 리콜(recall)



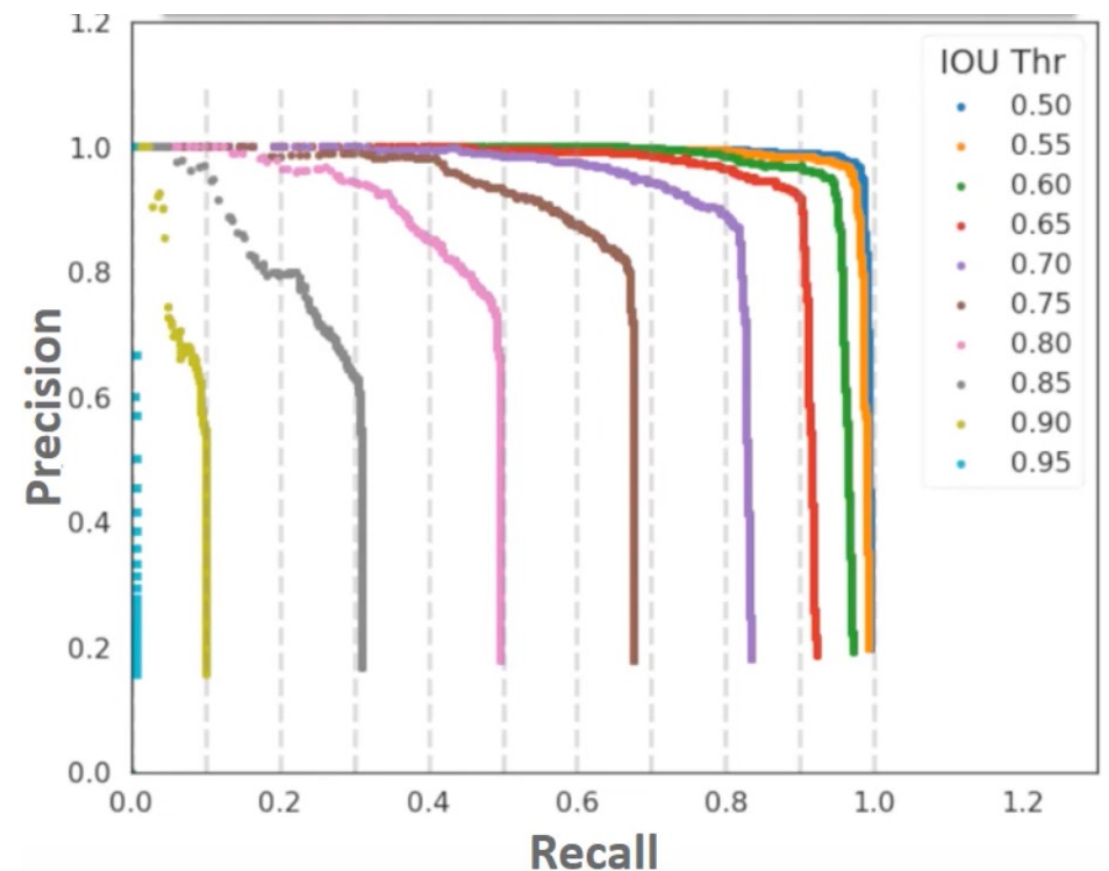
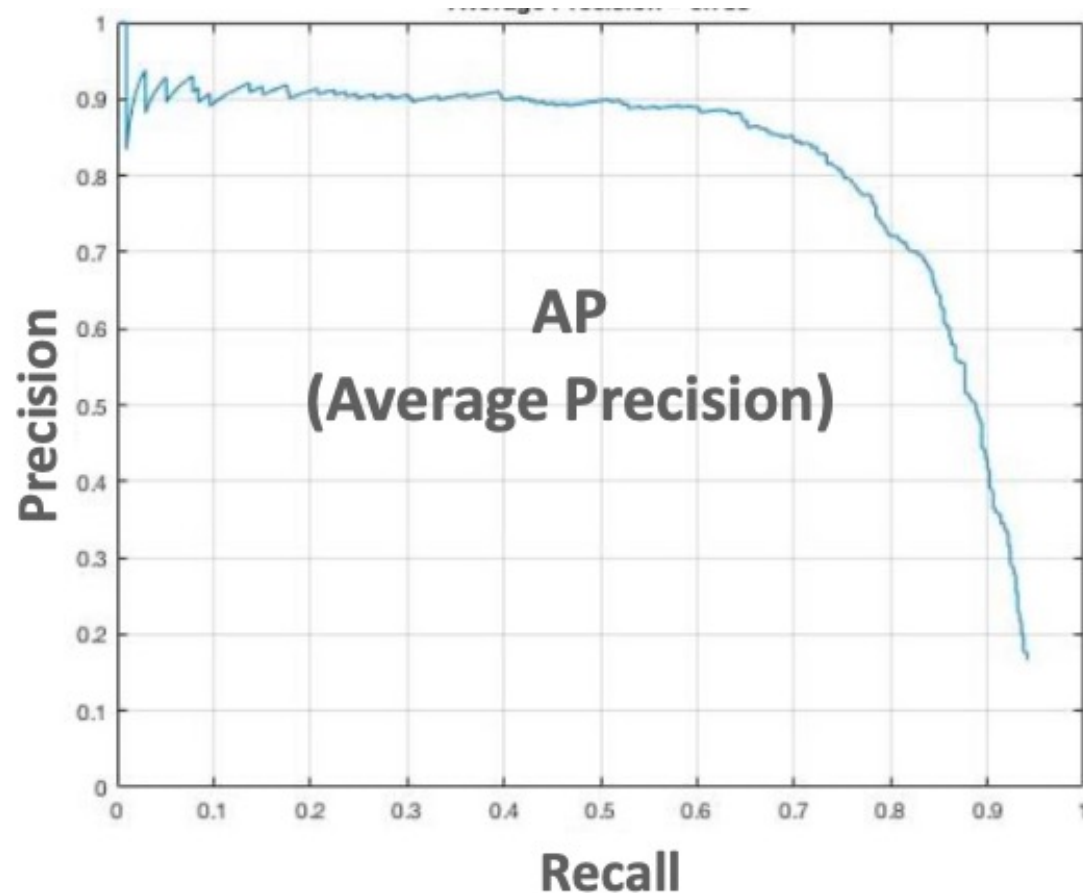
중복 객체 제거

- ▶ NMS(Non Max Suppression)
 - ▶ 특정 Confidence threshold 이하 bounding box를 먼저 제거
 - ▶ 높은 confidence score를 가진 box와 겹치는 다른 box를 모두 조사하여 IOU가 특정 threshold 이상인 box를 모두 제거(1등만 남김)



성능평가 - mAP

- ▶ mAp(mean Average Precision)
 - ▶ 여러 클래스들의 AP를 평균한 값 [블로그](#)
- ▶ PASCAL VOC (Visual Object Classes Challenges)
 - ▶ 리콜값이 0, 0.1, 0.2, ... 1.0까지 11개의 mAP를 구함
- ▶ Coco
 - ▶ IOU 값이 0.5, 0.55, 0.6, ..., 0.95 경우의 mAP를 구함



YOLO v8

- ▶ 객체 검출과 객체 세분화를 쉽게 구현
- ▶ 사용 절차
 - ▶ 데이터 준비: 이미지와 레이블링(annotation)
 - ▶ 데이터셋 다운로드: Roboflow 등 사용
 - ▶ yaml 파일 생성 (데이터 저장 폴더 정보 등을 설정하는 파일)
 - ▶ 모델 설치: ultralytics에서 YOLO 설치
 - ▶ `model = YOLO("....pt")` # OD 또는 seg 선택
 - ▶ `model.train(data = "...yaml")`
 - ▶ `model.predict(source='...jpg')`
 - ▶ `runs/detect/predict` 폴더에 생성

YOLOv8 detect 성능

Model	size (pixels)	mAP ^{val} 50-95	Speed CPU ONNX (ms)	Speed A100 TensorRT (ms)	params (M)	FLOPs (B)
YOLOv8n	640	37.3	80.4	0.99	3.2	8.7
YOLOv8s	640	44.9	128.4	1.20	11.2	28.6
YOLOv8m	640	50.2	234.7	1.83	25.9	78.9
YOLOv8l	640	52.9	375.2	2.39	43.7	165.2
YOLOv8x	640	53.9	479.1	3.53	68.2	257.8

- **mAP^{val}** values are for single-model single-scale on [COCO val2017](#) dataset.
Reproduce by `yolo val detect data=coco.yaml device=0`
- **Speed** averaged over COCO val images using an [Amazon EC2 P4d](#) instance.
Reproduce by `yolo val detect data=coco128.yaml batch=1 device=0|cpu`

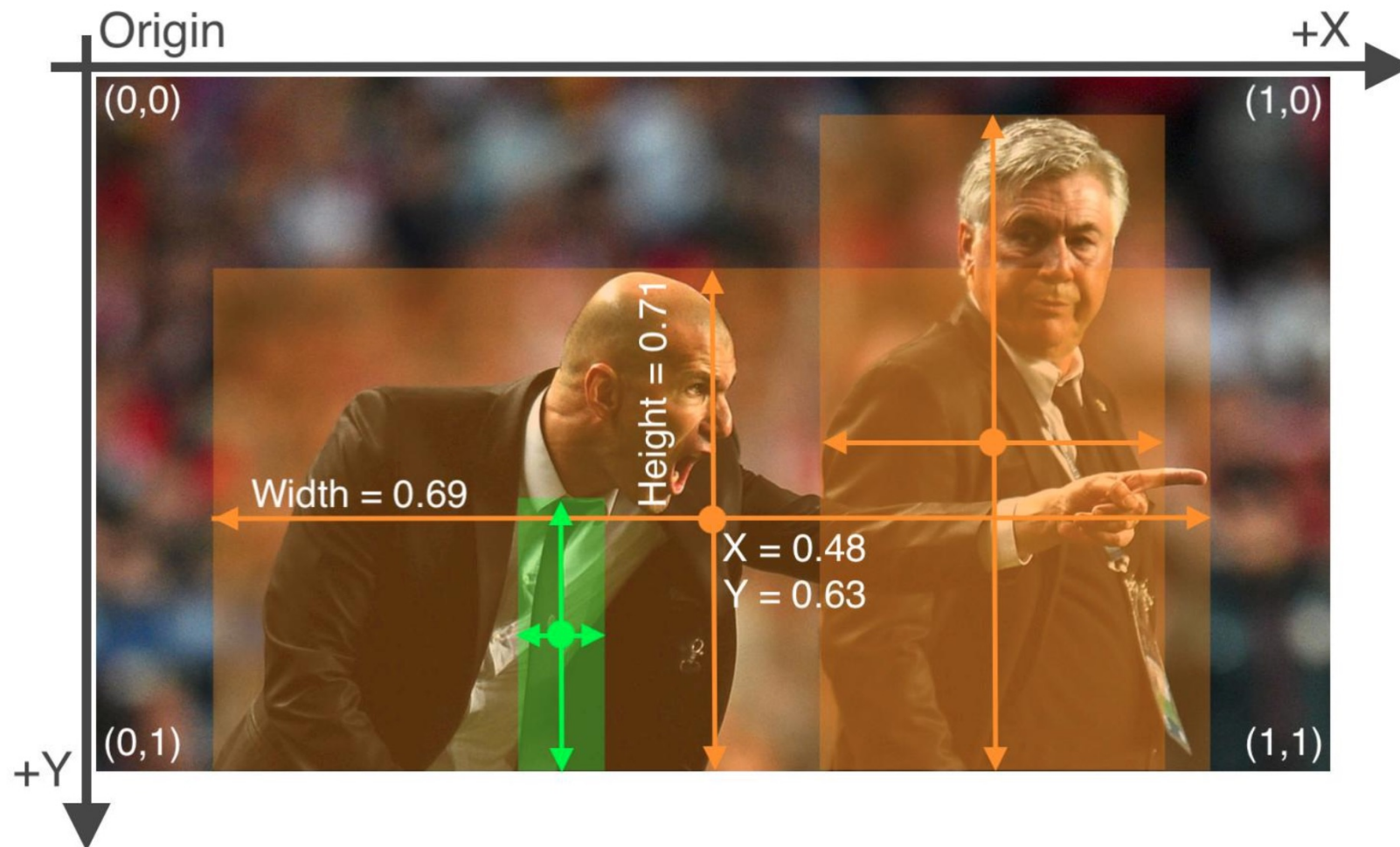
Segment

Model	size (pixels)	mAP ^{box} 50-95	mAP ^{mask} 50-95	Speed CPU ONNX (ms)	Speed A100 TensorRT (ms)	params (M)	FLOPs (B)
YOLOv8n-seg	640	36.7	30.5	96.1	1.21	3.4	12.6
YOLOv8s-seg	640	44.6	36.8	155.7	1.47	11.8	42.6
YOLOv8m-seg	640	49.9	40.8	317.0	2.18	27.3	110.2
YOLOv8l-seg	640	52.3	42.6	572.4	2.79	46.0	220.5
YOLOv8x-seg	640	53.4	43.4	712.1	4.02	71.8	344.1

- **mAP^{val}** values are for single-model single-scale on [COCO val2017](#) dataset.
Reproduce by `yolo val segment data=coco.yaml device=0`
- **Speed** averaged over COCO val images using an [Amazon EC2 P4d](#) instance.
Reproduce by `yolo val segment data=coco128-seg.yaml batch=1 device=0|cpu`

이미지와 레이블 데이터셋

- ▶ 이미지와 annotation 파일명(txt)이 이미지 별로 동일해야 한다
 - ▶ image7.jpg → image7.txt (아래는 3개의 객체가 들어 있다)
 - ▶ image7.txt 의 첫번째 행 bbox(bounding box) : (0.48, 0.63, 0.69, 0.71)



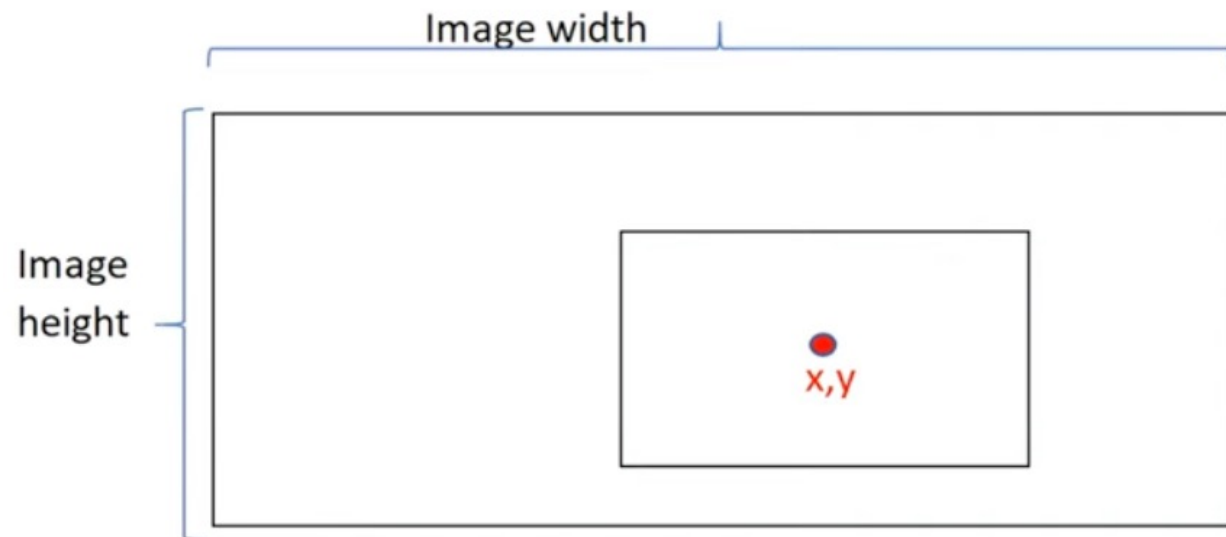
Dataset

- ▶ **COCO(Common Objects in Context):**
 - ▶ 가장 널리 사용되는 데이터 세트
 - ▶ 200,000개 이상의 이미지가 포함
 - ▶ 사람, 자동차, 개 등 80개의 클래스를 포함
 - ▶ Bounding Box, 인스턴스 분할 마스크, keypoints 등 제공

- ✓ Object segmentation
- ✓ Recognition in context
- ✓ Superpixel stuff segmentation
- ✓ 330K images (>200K labeled)
- ✓ 1.5 million object instances
- ✓ 80 object categories
- ✓ 91 stuff categories
- ✓ 5 captions per image
- ✓ 250,000 people with keypoints



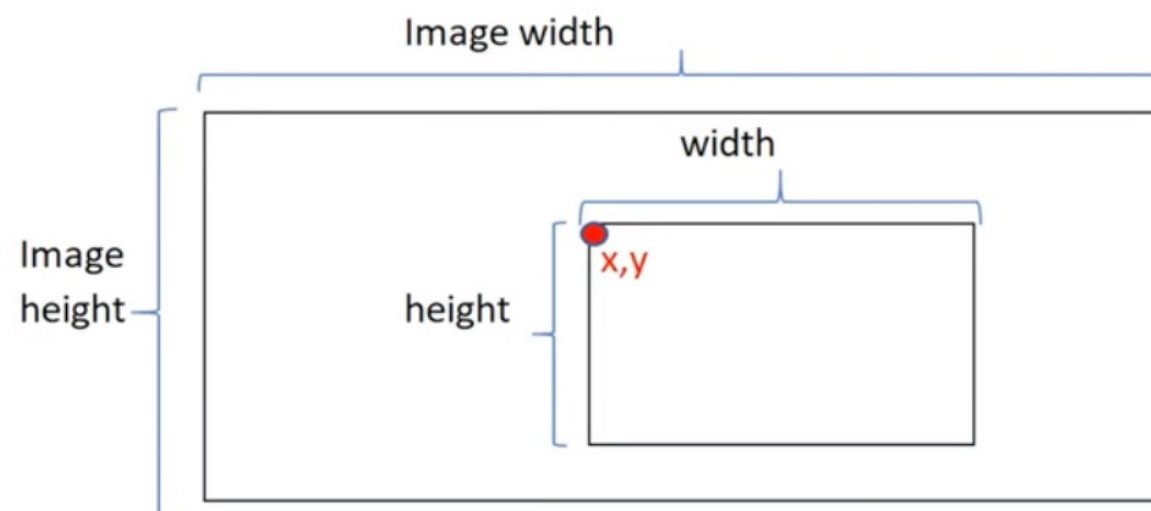
YOLO와 coco format



$$x_{yolo} = x / \text{Image width} \quad y_{yolo} = y / \text{Image height}$$

$$width_{yolo} = \text{width} / \text{Image width} \quad height_{yolo} = \text{height} / \text{Image height}$$

YOLO format bounding box = $\langle x_{yolo}, y_{yolo}, width_{yolo}, height_{yolo} \rangle$



$$x_{coco} = x \quad y_{coco} = y$$

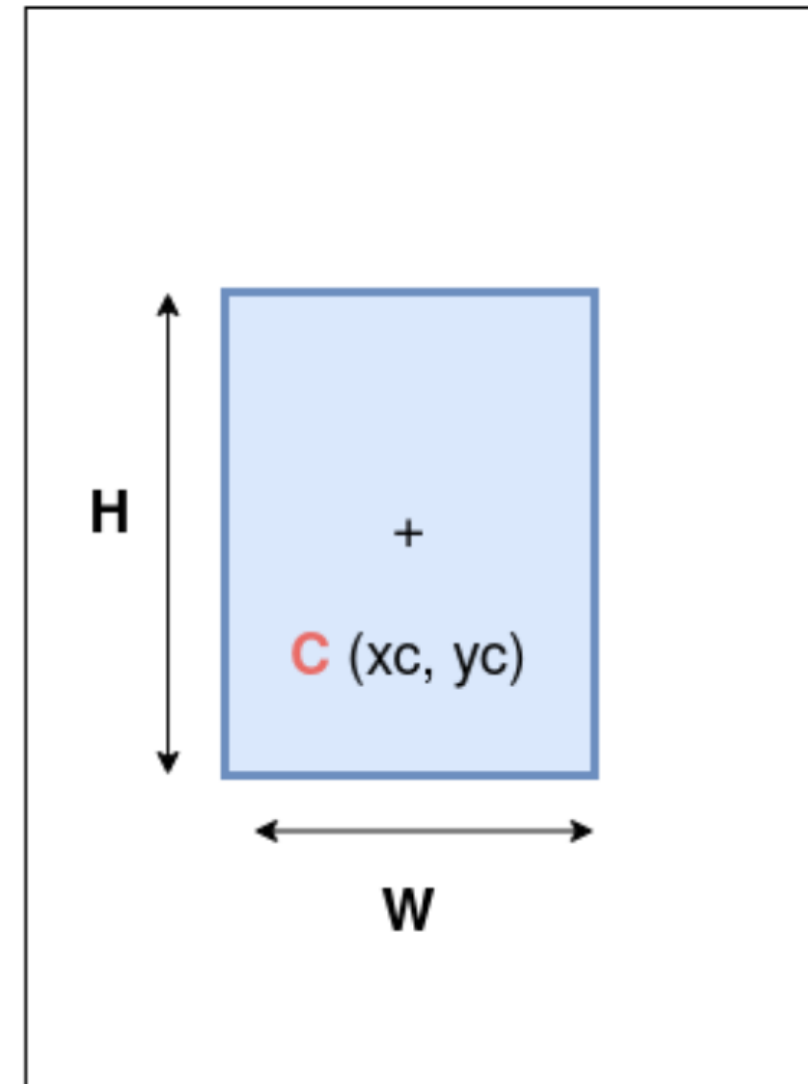
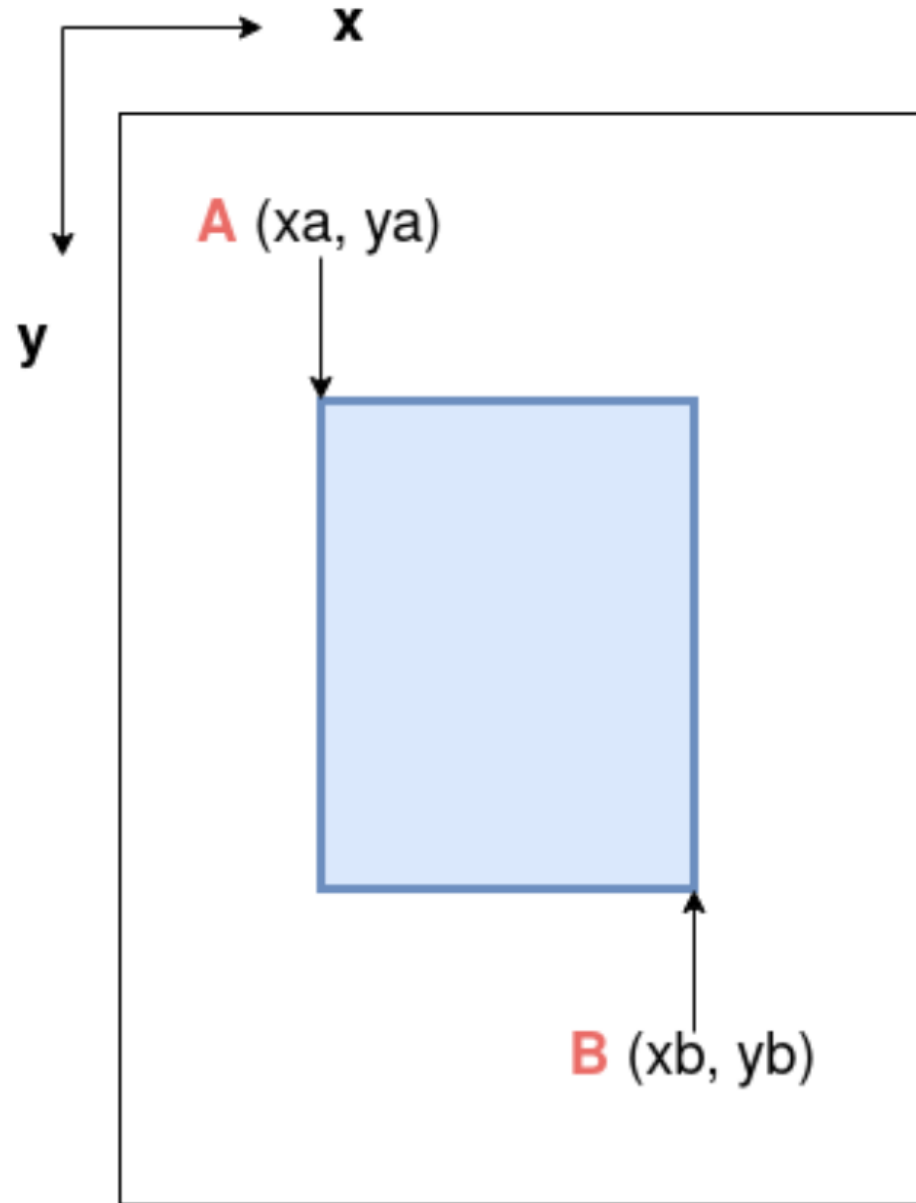
$$width_{coco} = \text{width} \quad height_{coco} = \text{height}$$

COCO format bounding box = $\langle x_{coco}, y_{coco}, width_{coco}, height_{coco} \rangle$

```
1 0.711719 0.302083 0.418750 0.334722
1 0.730469 0.807639 0.431250 0.315278
1 0.262109 0.715278 0.477344 0.238889
1 0.237500 0.254861 0.332812 0.262500
```

```
"annotations": [
  {
    "id": 1,
    "image_id": 1,
    "category_id": 2,
    "iscrowd": 0,
    "area": 35041.209375000006,
    "bbox": [
      251,
      67,
      209.375,
      167.36100000000002
    ],
    "segmentation": [
      [
        251,
        67,
        460.375,
        67,
        460.375,
        234.36100000000002,
        251,
        234.36100000000002
      ]
    ]
  }
]
```

xyxy, xywh 포맷



데이터셋 준비

- ▶ Roboflow 이용
- ▶ 개인 데이터셋 업로드 후 레이블링
- ▶ 제공되는 공개 데이터셋 사용

Labelimg

- ▶ 개인 PC에서 레이블링 작업을 하는 도구
- ▶ 블로그
 - ▶ <https://yeko90.tistory.com/entry/free-image-label-tool-labelimg>
- ▶ `git clone https://github.com/HumanSignal/labelimg.git`
 - ▶ (맥에서 설치 필요) `xcode-select --install`
- ▶ 실행
 - ▶ `python labelimg.py`
 - ▶ 저장 포맷을 yolo로 설정해야 한다 (txt로 저장된다)
 - ▶ 레이블링 표현에는 xml, json, txt 등의 포맷이 있다

yaml 파일

- ▶ 객체 이름, 클래스수 (nc), 데이터가 있는 폴더명을 지정
- ▶ PyYAML을 사용하여 파이썬으로 설정
 - ▶ 딕셔너리로 설정할 내용을 지정

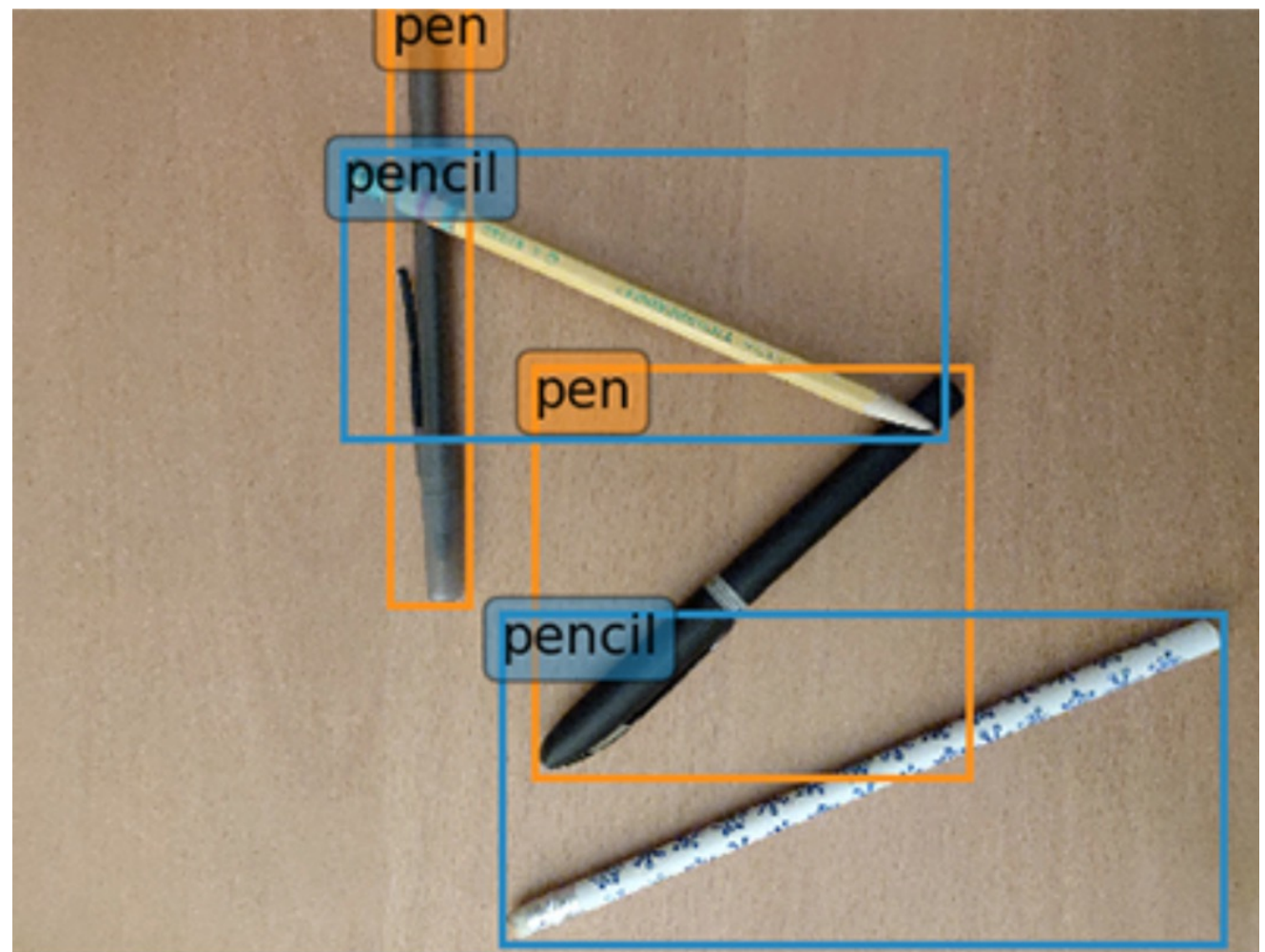
```
{ 'names': ['fish',  
            'jellyfish',  
            'penguin',  
            'puffin',  
            'shark',  
            'starfish',  
            'stingray'],  
  'nc': 7,  
  'test': '/content/Aquarium_Data/test/images',  
  'train': '/content/Aquarium_Data/train/images/',  
  'val': '/content/Aquarium_Data/valid/images/' }
```

predict 옵션

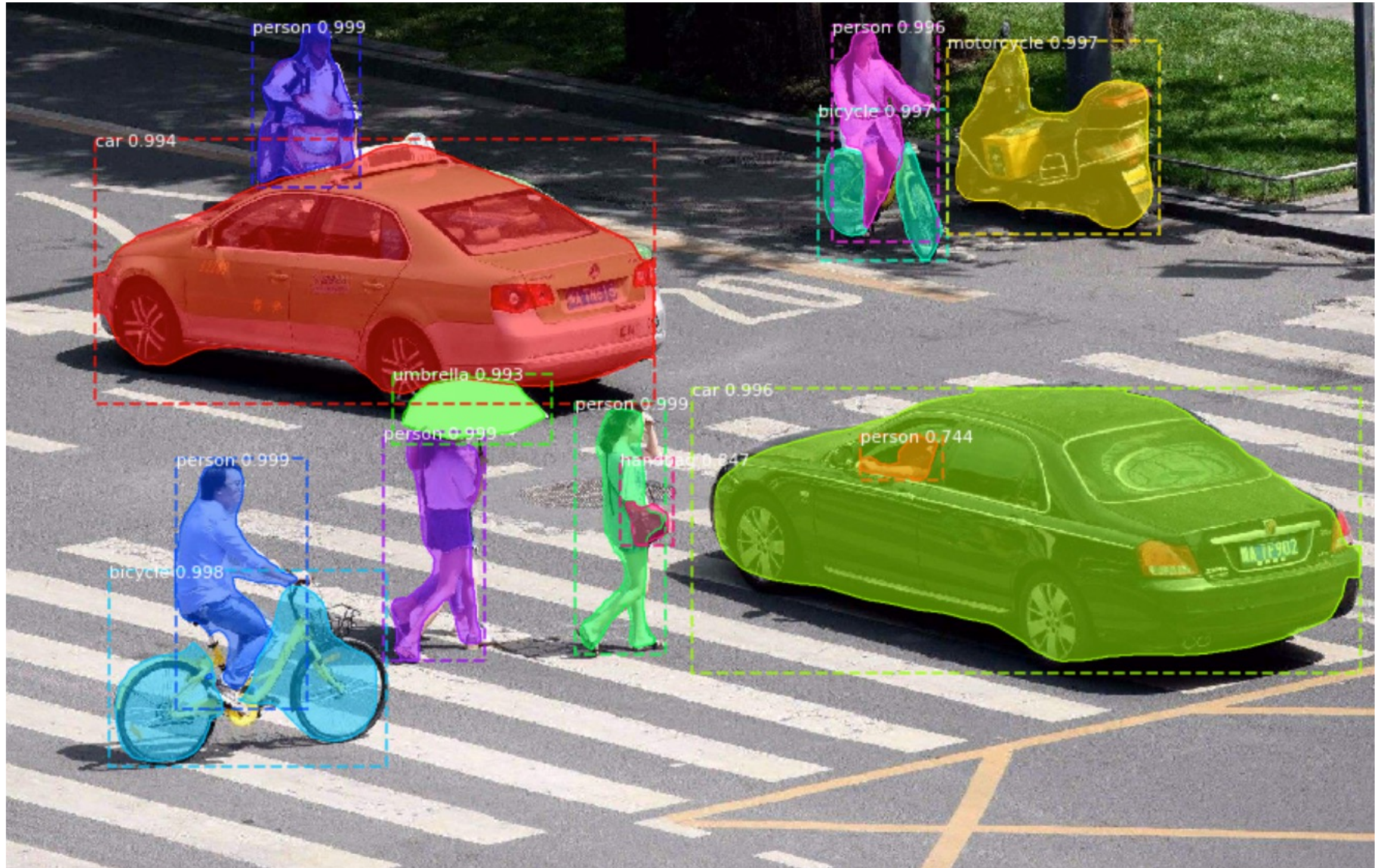
Name	Type	Default	Description
source	str	'ultralytics/assets'	source directory for images or videos
conf	float	0.25	object confidence threshold for detection
iou	float	0.7	intersection over union (IoU) threshold for NMS
imgsz	int or tuple	640	image size as scalar or (h, w) list, i.e. (640, 480)
half	bool	False	use half precision (FP16)
device	None or str	None	device to run on, i.e. cuda device=0/1/2/3 or device=cpu
show	bool	False	show results if possible
save	bool	False	save images with results
save_txt	bool	False	save results as .txt file
save_conf	bool	False	save results with confidence scores
save_crop	bool	False	save cropped images with results

Labeling

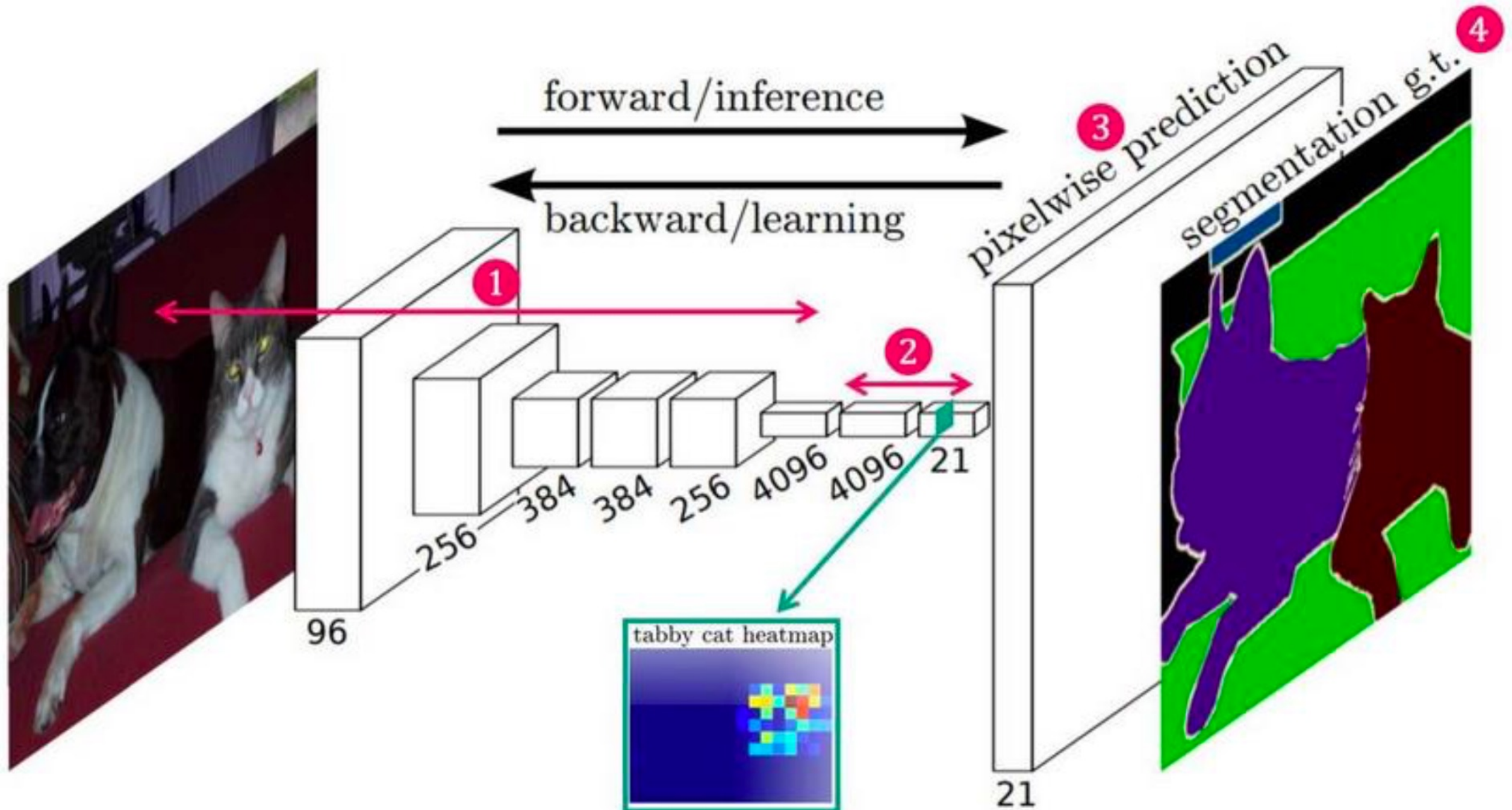
- ▶ 모든 객체를 레이블링
- ▶ 짝 차게
- ▶ 클래스는 가능한 세분한 후 나중에 합친다
- ▶ 이미지 크기와 픽셀 고려



Object Segmentation



Object Segmentation

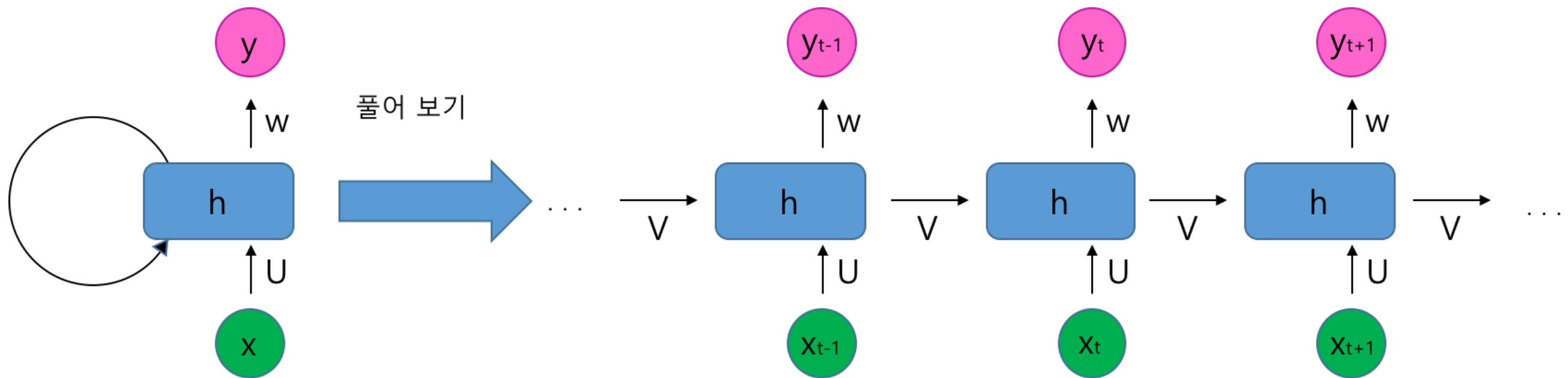


RNN

순환신경망 (RNN)

- ▶ Recurrent Neural Network
- ▶ 합성곱 신경망(CNN)은 2차원 (공간) 필터에서 좋은 성과를 얻는 것이 입증되어 이미지 학습 성능을 급격히 개선했다
- ▶ 그러나 시간적적으로 연속적으로 발생하는 데이터 예를 들어 자연어처리, 시계열 분석 등에는 순환신경망이 잘 동작한다
 - ▶ CNN에서는 과거의 기억(상태정보)을 사용하지 않지만 RNN은 과거의 기억을 사용하는 구조를 갖는다
 - ▶ 순환신경망은 음성인식, 텍스트 분석, 챗봇, 감성분석, 언어 모델링 (language modeling) 등에 도입되었다

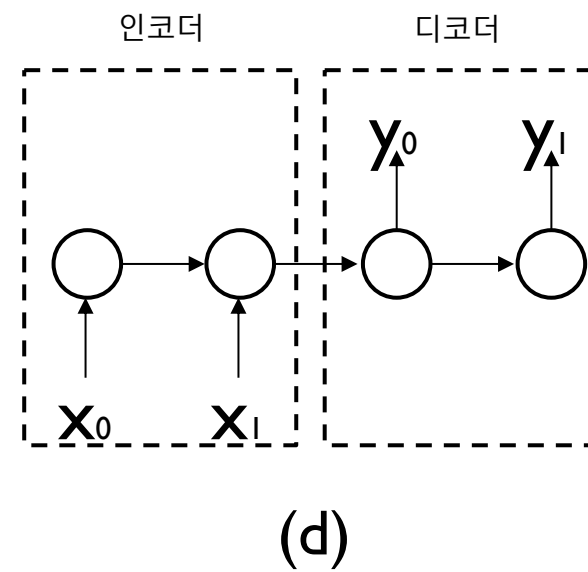
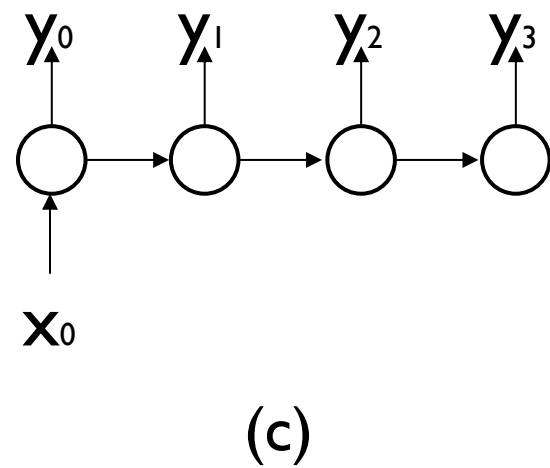
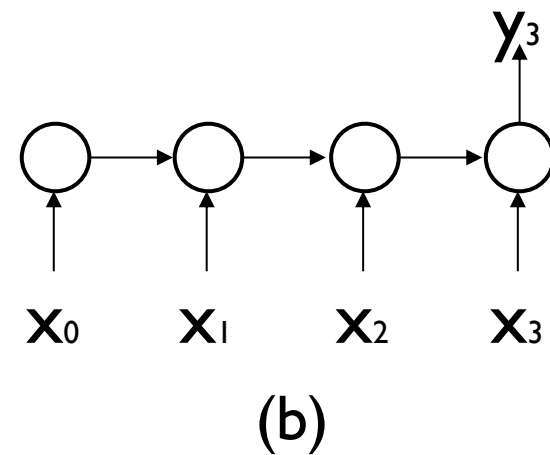
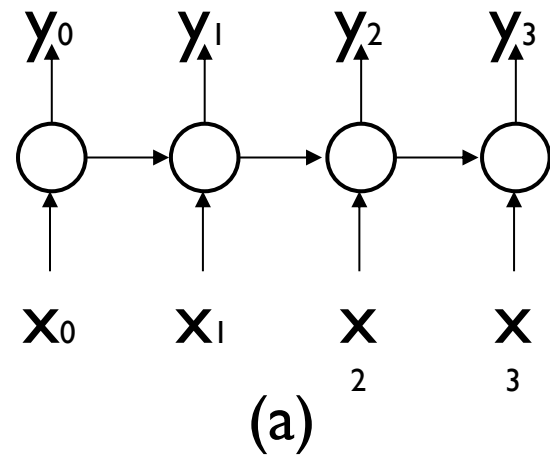
RNN의 구조



$$h_i = V_{i-1} + UX_i$$
$$y_i = Wh_i$$

- ▶ 위 표현에서 활성화 함수와 bias는 생략됨
- ▶ 상태값 h 를 다음 스텝에 사용한다
- ▶ U, V, W 는 i 의 함수가 아니라 모든 스텝에서 같은 모양을 갖는다

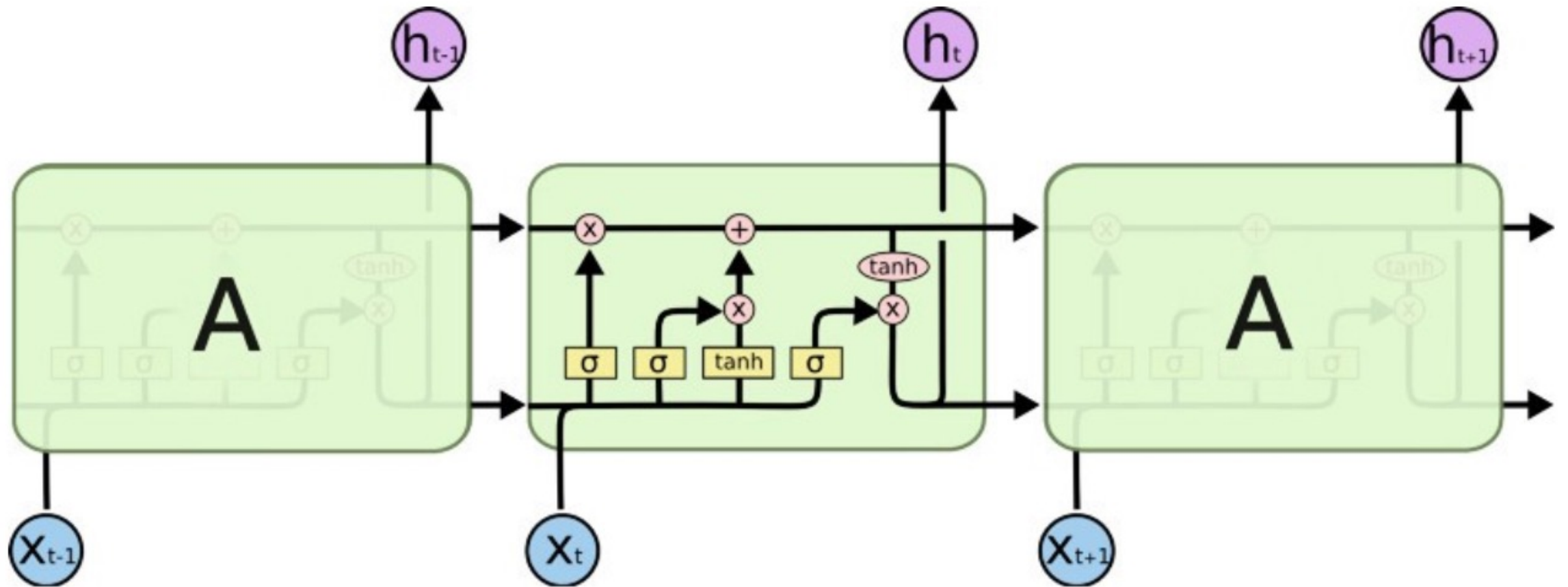
RNN 이용 형태



RNN의 개선

- ▶ RNN에서는 모든 입력(예, 단어)를 동일한 비중으로 기억한다
 - ▶ 오래된 정보를 모두 중요시 하면 정보가 너무 많이 축적되어 발산할 우려가 있고, 오래된 정보를 약하게 반영하면 중요한 정보를 놓치는 우려가 있다
- ▶ 이를 해결하기 위해서 LSTM(Long-Short Term Memory) 이 소개되었다
 - ▶ LSTM에서는 오랫동안 중요도를 유지할 정보와 그럴 필요 없이 망각해야 할 신호를 구분하여 따로 처리하여 성능을 개선했다

LSTM



- ▶ 망각 게이트: $f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$
 - ▶ 시그모이드를 적용하여 결과가 1이면 이전의 내부 상태 값을 모두 그대로 통과, 결과가 0이면 모두 망각을 수행 (중간 값은 통과 확률)

시계열 분석

시계열 분석의 종류

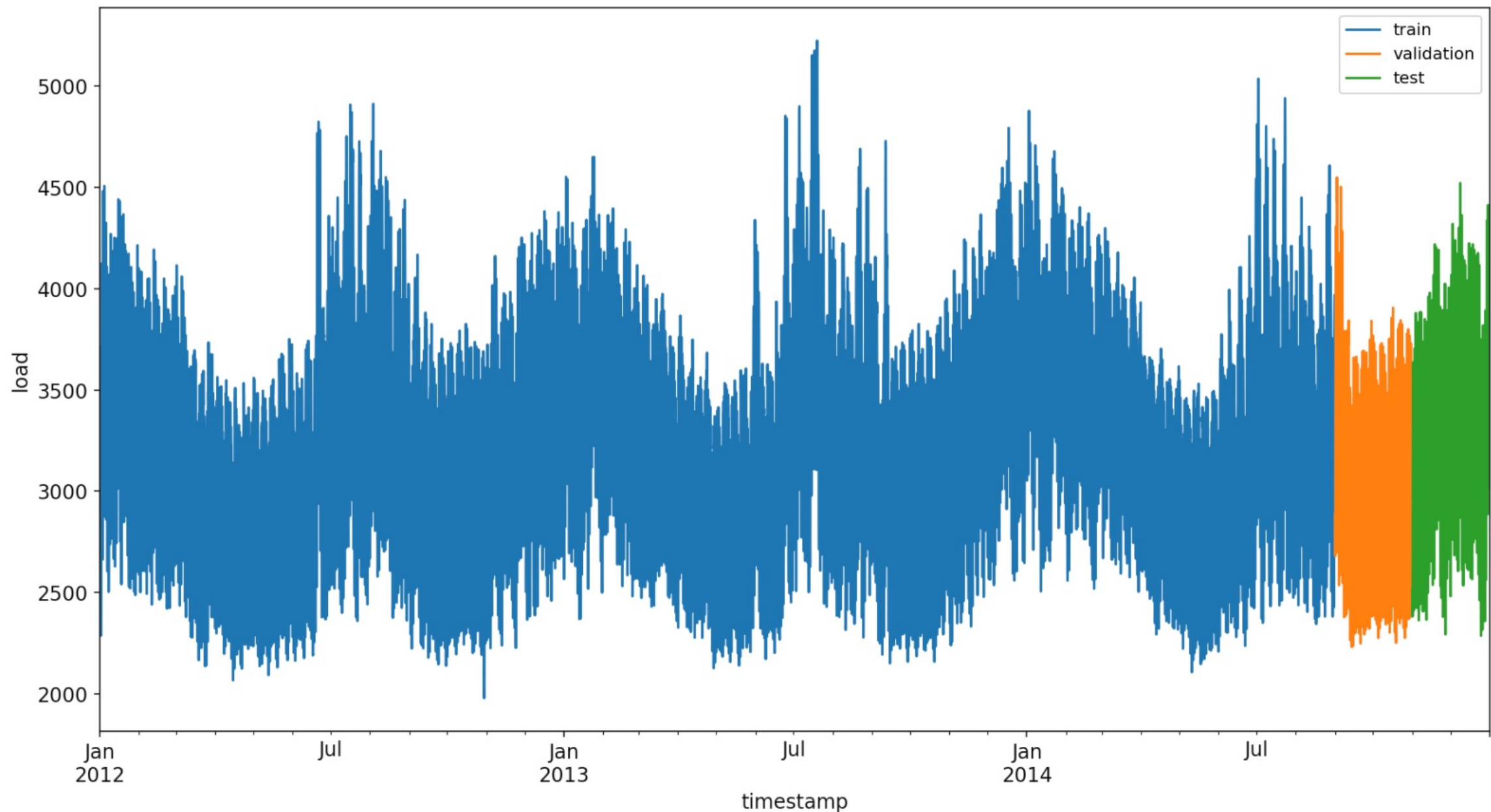
- ▶ 미래값 예측 (time forecasting)
 - ▶ 과거 데이터를 보고 미래값을 예측(forecasting)하는 것
 - ▶ 목적변수(y)의 과거값만 사용하는 경우와 다양한 설명변수(X)를 사용하는 방법으로 나눌 수 있다
 - ▶ 복잡한 시스템인 경우 RNN/LSTM 등 신경망을 사용한다
 - ▶ 단변수 예측에서는 prophet을 사용하면 계절 정보, 트렌드 등을 쉽게 추출할 수 있다
- ▶ 패턴 인식
 - ▶ 신호의 변화를 보고 패턴을 예측하는 모델
 - ▶ 기기의 동작 유형 판독
 - ▶ 고장의 유형 판독 등

시계열 예측 모델

- ▶ 선형 모델, 결정 트리, 랜덤 포레스트 등을 사용할 수 있다
 - ▶ 트리계열 모델을 다양한 입력 특성 (X)를 고려할 때, 특히 카테고리 변수가 많은 경우에 잘 동작한다
- ▶ 시계열 데이터 분석에서는 윈도우 크기를 먼저 정해야 한다
 - ▶ 분석에 사용하는 시간 구간을 윈도우라고 한다
 - ▶ 다양한 크기의 슬라이딩 윈도우를 적용할 수 있다
 - ▶ 입력 신호가 미래에 영향을 미치는 시간 지연(time lag)를 파악해야 하는 경우가 많다
- ▶ 선형모델이 머신러닝, 딥러닝 방법보다 성능이 우수한 경우도 있다

딥러닝 시계열 분석

- ▶ 3년간 전력사용량 시계열 데이터
 - ▶ <https://github.com/Azure/DeepLearningForTimeSeriesForecasting>

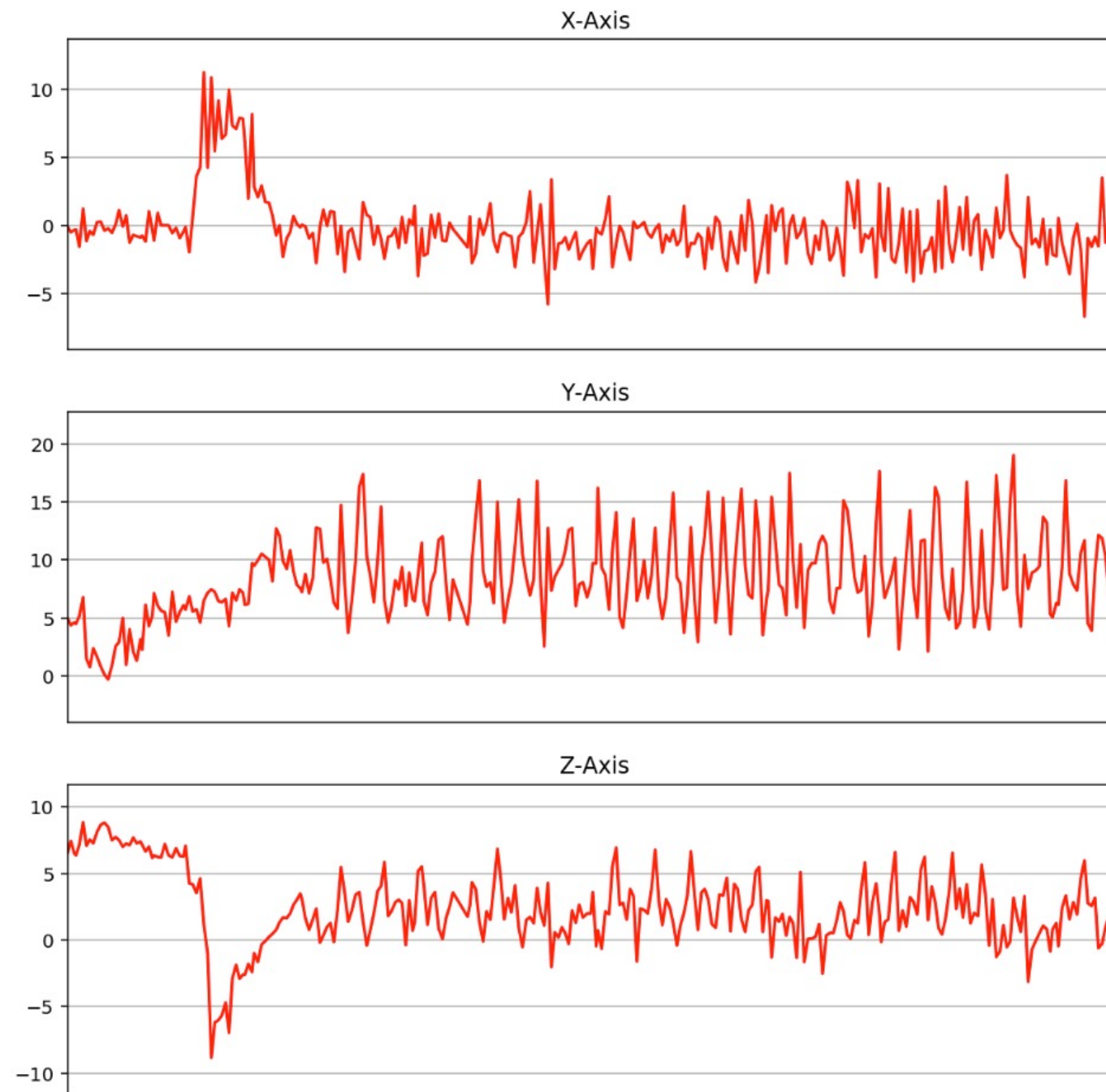


Prophet

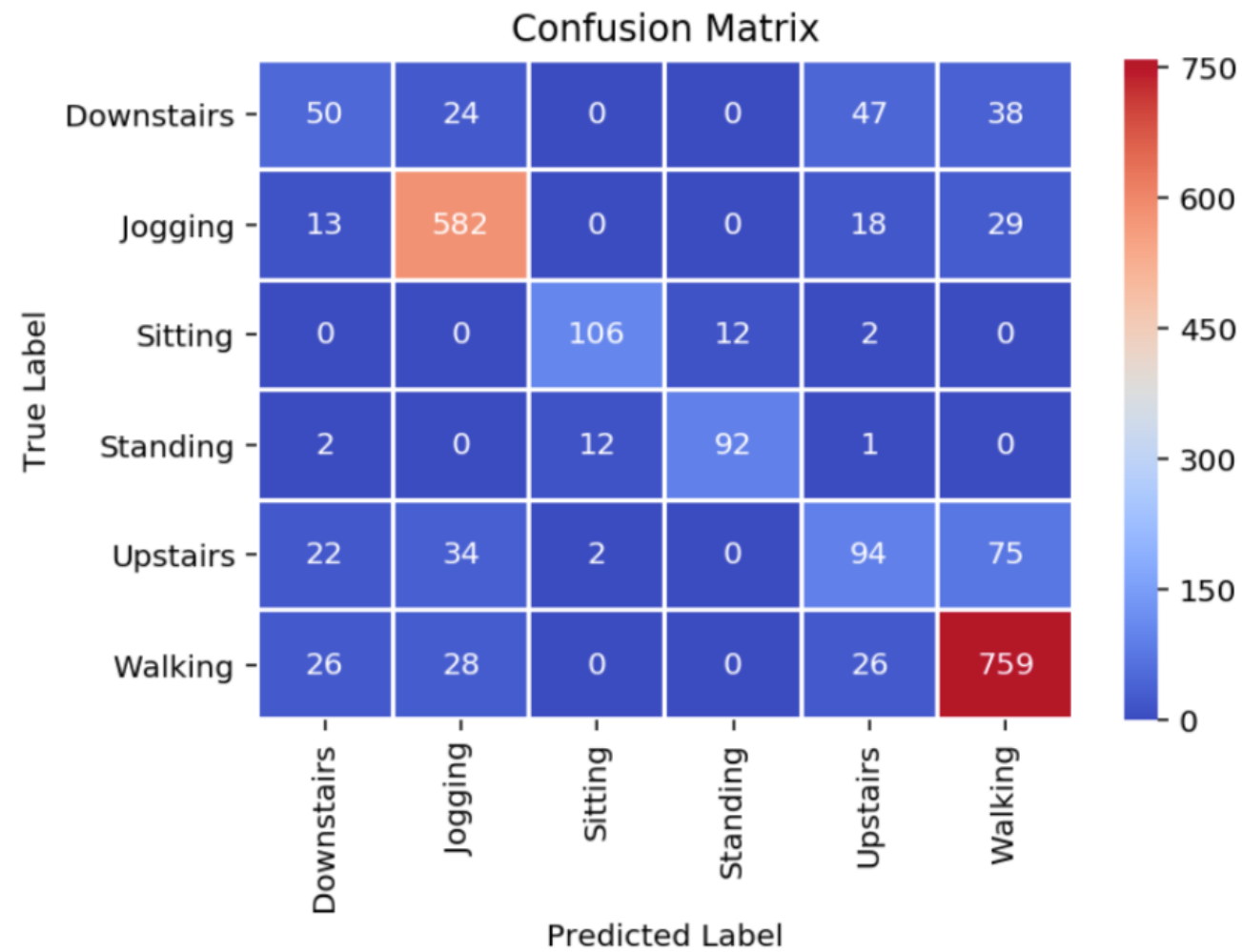
- ▶ 이벤트(Holiday) 정보를 반영하기가 쉽다
- ▶ 계절성 파악이 편리하다
 - ▶ 일간, 주간, 월간 계절성을 동시에 파악해준다
 - ▶ 트렌드 파악을 해준다
- ▶ **결측치가 있어도 잘 동작한다**
- ▶ 단변수 예측에서 편리하게 사용할 수 있다
- ▶ 참조

센서 데이터 패턴 분류

계단내려가기



분류 결과



	precision	recall	f1-score	support
0	0.44	0.31	0.37	159
1	0.87	0.91	0.89	642
2	0.88	0.88	0.88	120
3	0.88	0.86	0.87	107
4	0.50	0.41	0.45	227
5	0.84	0.90	0.87	839
avg / total	0.79	0.80	0.79	2094

예지 보수

예지 보수

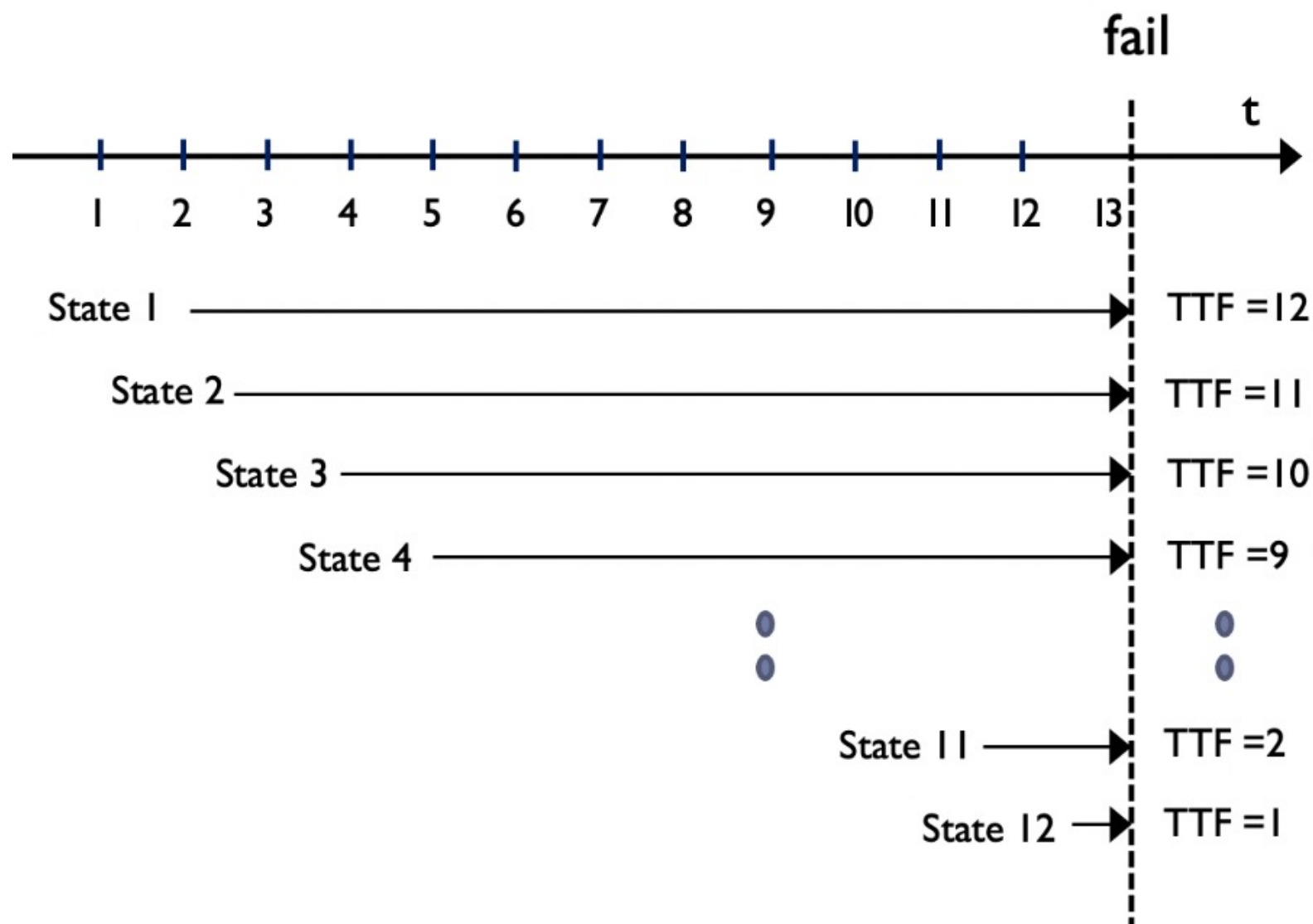
- ▶ Predictive Maintenance (PM)
 - ▶ 비용절감 효과
 - ▶ 장비 및 부품 고장 위험의 사전 예측
- ▶ 보수의 종류
 - ▶ 고장 후 보수: 기기가 고장이 나면 보수를 하는 것
 - ▶ 정기적인 보수: 일정한 시간 간격으로 보수를 하는 것
 - ▶ 예지 보수: 고장을 예측하여 적절한 시점에 보수를 하는 것

문제정의

- ▶ 고장예측의 종류
 - ▶ 고장 시점 예측, 고장 타입 분류, 고장 기간 예측
- ▶ 센서 값의 변화를 학습하는 기계학습 모델을 만든다
- ▶ 고장예측을 회귀 또는 분류 문제로 접근할 수 있다.
 - 회귀: 몇 번의 사이클 이후에 고장이 발생할 것인가?
Remaining Useful Life (RUL), Time to Failure (TTF) 예측
 - 분류: 특정한 사이클 수 이내에 고장이 발행한 것인가 (이진 분류)
여러 구간에서 고장이 발생할지를 예측 (다중 분류)

PM 회귀 모델

- ▶ TTF(time to fail): 고장까지 남은 시간을 예측

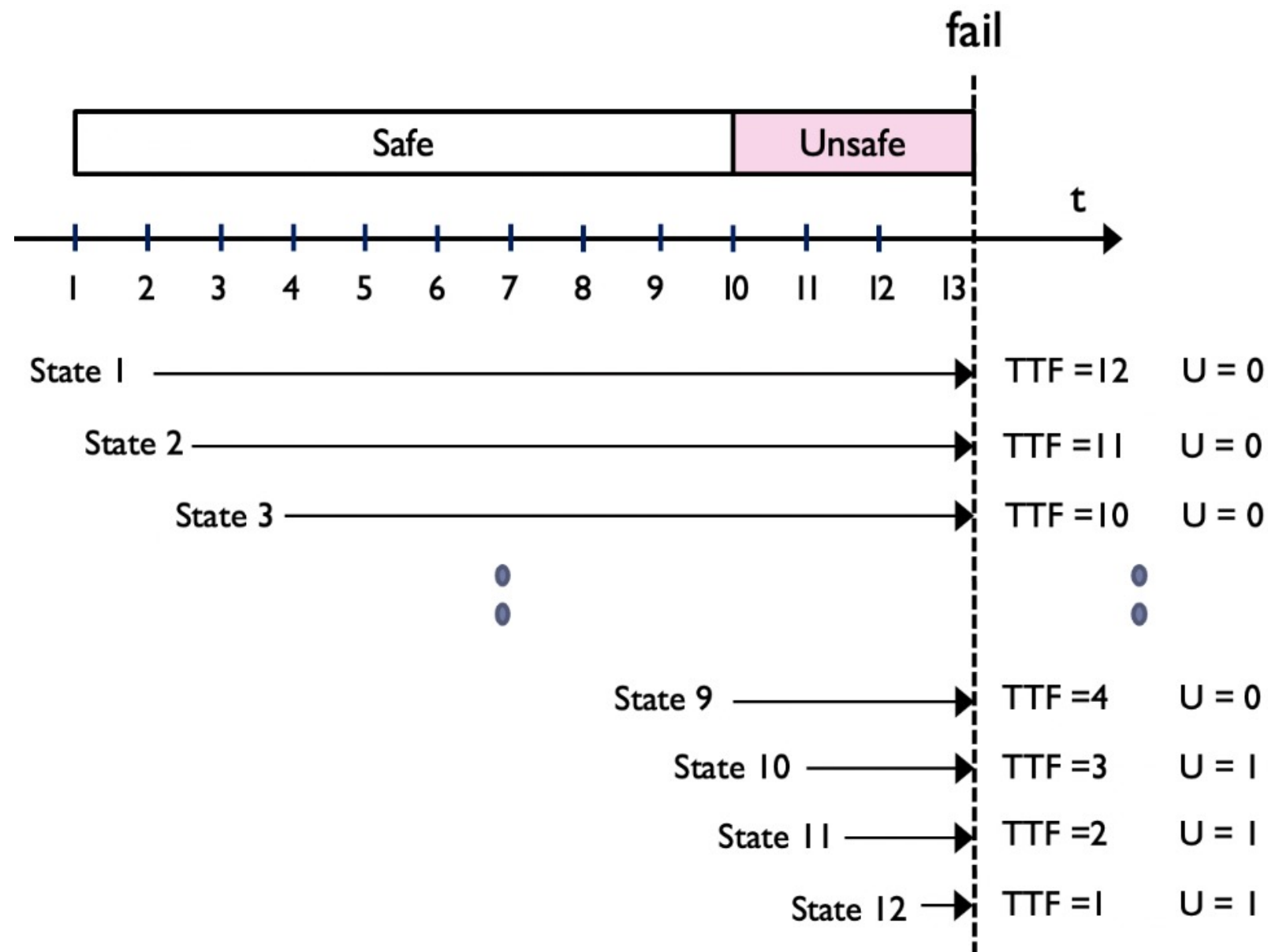


Regression

X: Features	<u>y:TTF</u>
State 1	12
State 2	11
State 3	10
State 4	9
.	.
.	.
State 11	2
State 12	1

PM 분류 모델

- ▶ TTF가 주어진 경계치 이하로 낮은 위험한 상태를 예측



Classification	
X: Features	U
State 1	0
State 2	0
State 3	0
.	.
.	.
State 9	0
State 10	1
State 11	1
State 12	1

Predictive Maintenance

- ▶ 운영중인 항공기 엔진이 언제 고장날지를 예측하는 문제

- <https://gallery.azure.ai/Collection/Predictive-Maintenance-Template-3>



사용 데이터

- ▶ 훈련 데이터
 - ▶ 21개의 센서 데이터를 사이클 단위로 측정
 - ▶ 같은 타입의 서로 다른 엔진에서 센서 데이터 측정
 - ▶ 정상상태에서 시작하며 훈련데이터의 경우 마지막 사이클이 고장난 시점이라고 간주함
- ▶ ground truth 데이터
 - ▶ 엔진별로 (총 100개) 고장 나기 전에 남아 있는 사이클 수를 표시
- ▶ 테스트 데이터에는 언제가 고장난 시점인지 알 수 없음
- ▶ 레이블링
 - ▶ 세가지 레이블을 사용
 - ▶ RUL, 이진분류, 다중분류

자연어 처리

NLP 목적

- ▶ NLP: Natural Language Processing
- ▶ 문서 분류
 - ▶ 스팸 필터링
 - ▶ 감성 분석
 - ▶ 이메일, 트위터, 블로그 등 문서의 타입 분류
 - ▶ 작문 (에세이)을 보고 잘 쓴 글인지 평가
 - ▶ 유사한 글이나 저자를 찾는 작업
 - ▶ 유사 논문이나 특허 찾기
 - ▶ 문서간의 연계성 찾기
- ▶ 주요 정보 추출
 - ▶ 회사의 주가변동, 지명, 고유명사 등을 추출
 - ▶ 문서 요약 - 키워드 추출, 주제 요약

NLP 예

- ▶ 챗봇
 - ▶ 사람과 대화하듯 동작
 - ▶ 룰 기반 방식 또는 신경망을 사용하여 자유롭게 대화하는 방식
- ▶ QA 시스템
 - ▶ 질문에 대답하는 서비스 (대한민국의 수도는? 오늘 날씨는? 등의 질문 적절한 답을 찾기)
- ▶ 문장 완성
 - ▶ 가장 자연스러운 다음 문장 완성
 - ▶ 맞춤법 검사, 동의어 제안, 검색
- ▶ 기계 번역

→ chatGPT의 출현으로 NLP 거의 모든 기능이 실용화되었다

텍스트 소스

- ▶ 디지털 문서 – 위키피디아, 책, 논문, 보고서, 이메일
- ▶ 웹 데이터 - 크롤링
- ▶ SNS 데이터 – 페이스북, 트위터, 블로그
- ▶ 고객 데이터 – 사용자 데이터, 클레임 데이터
- ▶ 내부 데이터 – 운영, 유지보수 로그 데이터

텍스트 표현법

- ▶ 컴퓨터가 인식할 수 있도록 토큰을 숫자로 표현하는 방법
 - ▶ BoW (Bag of words)
 - ▶ 문장 내에 등장한 단어 빈도 수 정보만 사용
 - ▶ 원핫 인코딩 사용 (각 토큰을 카테고리 변수로 취급)
 - ▶ Stop words 처리 (영어는 500여개, 한글은 677개를 정의)
 - ▶ TF-IDF
 - ▶ Term Frequency – Inverse Document Frequency
 - ▶ 문장내에 발생하는 토큰의 빈도수에 단어의 중요도를 반영
 - ▶ 워드 벡터 (임베딩)
 - ▶ 카테고리 인코딩이 아니라, 벡터 형태로 토큰을 표현
 - ▶ 벡터를 사용하여 토큰의 위치와 방향성을 파악할 수 있다
 - ▶ WordVector, Glove, fastText

원핫(one-hot) 인코딩

- ▶ 토큰에 고유 번호를 배정하여 사전을 만들고 모든 토큰의 고유번호 위치의 컬럼만 1로 표시하고, 나머지 컬럼은 모두 0인 벡터로 표시하는 방법이다.

- 텍스트: “어제 러시아에 갔다가 러시아 월드컵을 관람했다”
- 토큰 사전: {“어제”:0, “러시아”:1, “갔다”:2, “월드컵”:3, “관람”:4}
- 원핫 인코딩:

어제 = [1, 0, 0, 0, 0]

러시아 = [0, 1, 0, 0, 0]

갔다 = [0, 0, 1, 0, 0]

월드컵 = [0, 0, 0, 1, 0]

관람 = [0, 0, 0, 0, 1]

BOW(Bag of Word)

- ▶ “문장”을 벡터로 표현하는 방법
 - ▶ 아래는 임의의 6개의 문장을 BOW로 표현한 예이다. (5000 단어)
 - ▶ DTM (Document-Term Matrix)

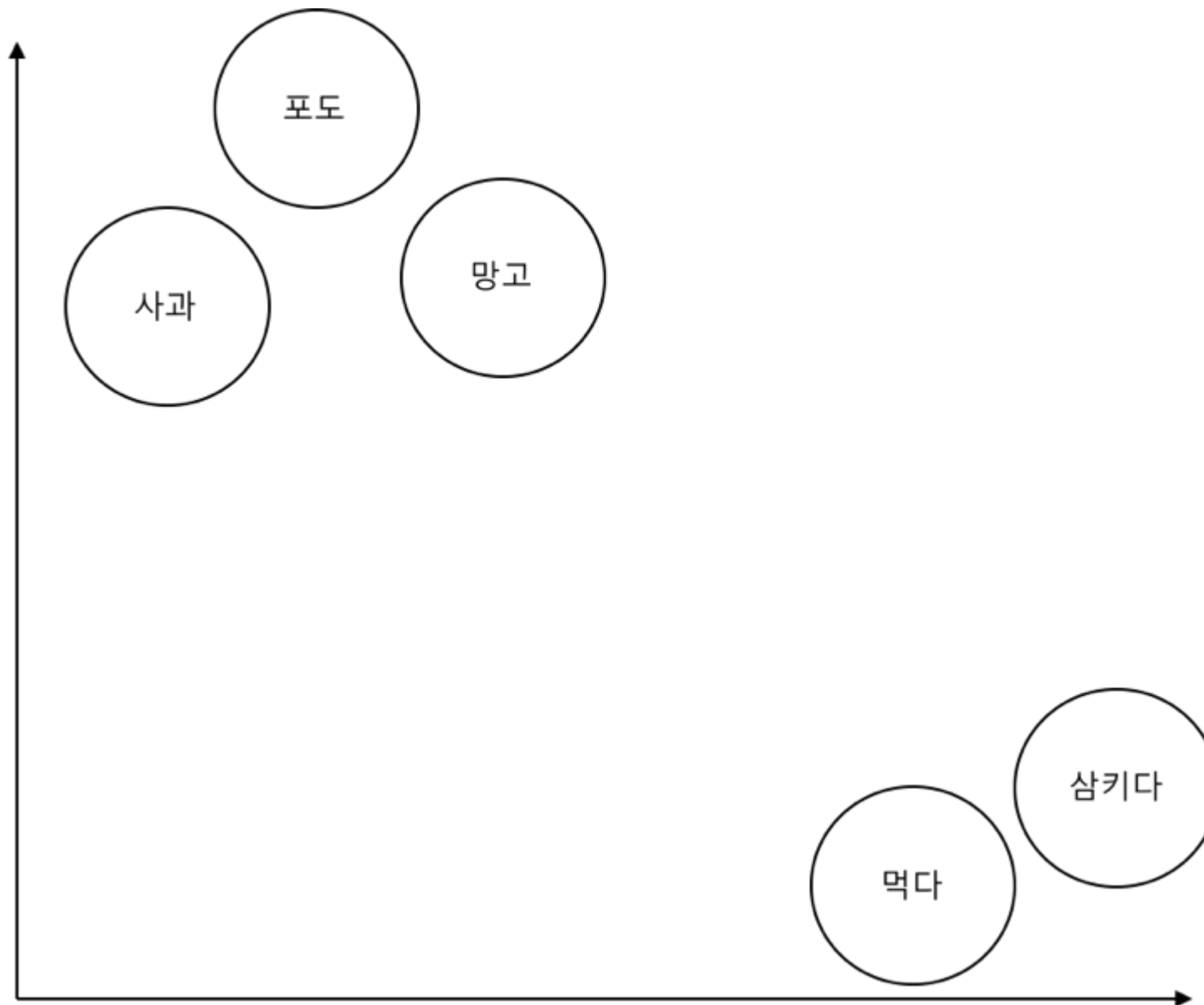
토큰번호	0	1	2	3	4	5	6	7	8	9	10	...	4998	4999
Text_1	1	0	0	1	1	0	1	0	1	0	0	0	0	0
Text_2	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Text_3	0	0	0	1	0	0	0	0	0	0	0	0	0	0
Text_4	0	0	0	0	0	0	0	1	0	0	0	0	1	0
...														
Text_6	0	0	0	0	0	0	1	0	0	0	0	0	0	0

단어 임베딩

- ▶ 단어를 고차원 공간 상의 벡터로 표현함으로써 단어간 거리(유사도)와 방향성(벡터)을 계산할 수 있게 되었다
 - ▶ 코사인 유사도를 사용한다
- ▶ 차원이 높을수록 정교한 의미 구분이 가능하며 50~1000개 정도의 차원을 사용한다.
 - ▶ 예를 들어 학교와 바다라는 단어를 단어 벡터로 표시하면 아래와 같이 보인다(아래 수치는 임의의 예시임)
 - 학교 = [0.23, 0.58, 0.97, ..., 0.87, 0.95]
 - 바다 = [0.45, 0.37, 0.81, ..., 0.22, 0.64]
- ▶ 단어 임베딩을 딥러닝의 입력으로 사용하여 모델 성능 개선

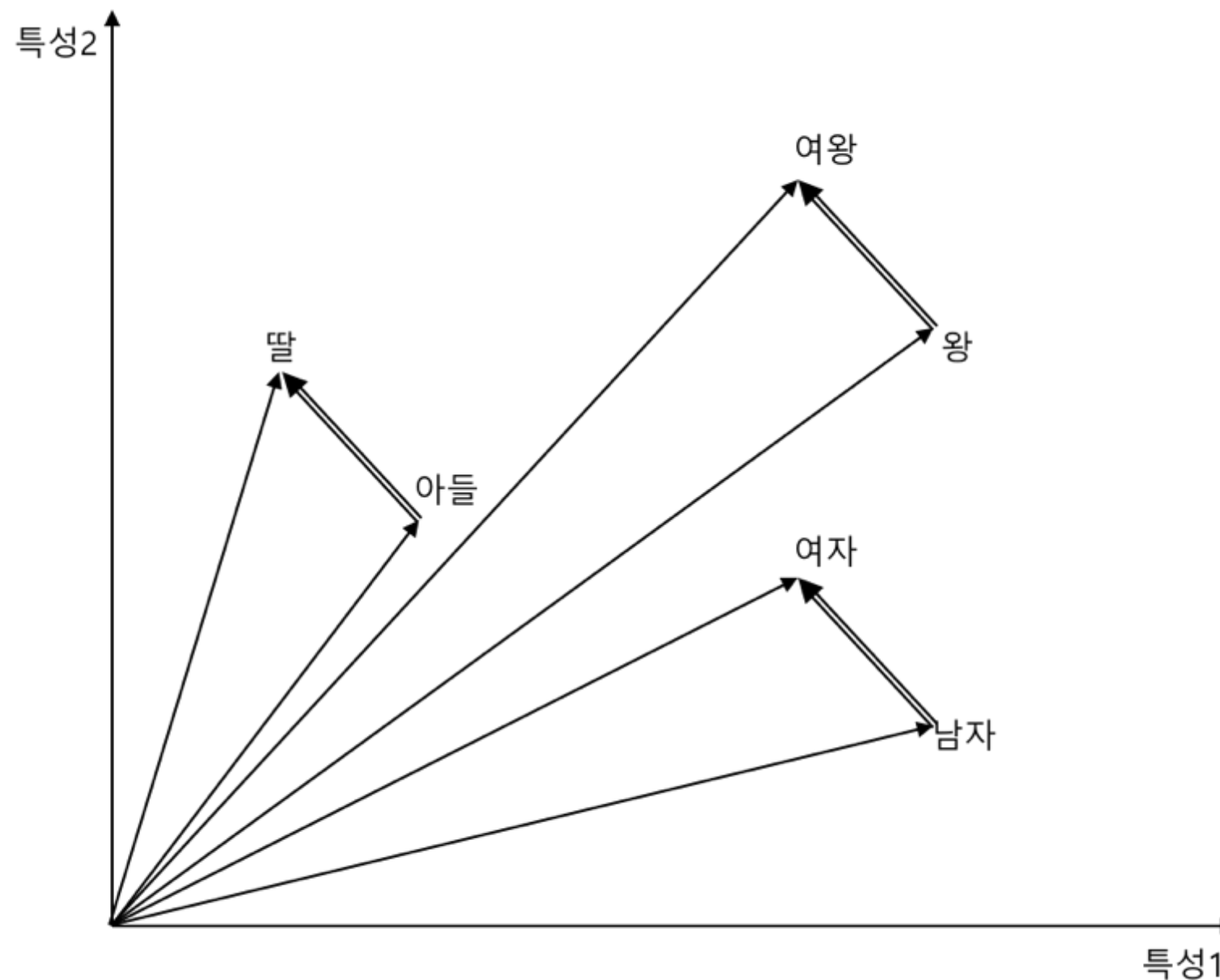
단어 임베딩

- ▶ 단어간의 거리와 방향을 계산할 수 있다



단어 임베딩

- ▶ 예를 들어 왕:여왕 = 아들: ? 에서 ? 부분이 딸이 될 것이다.

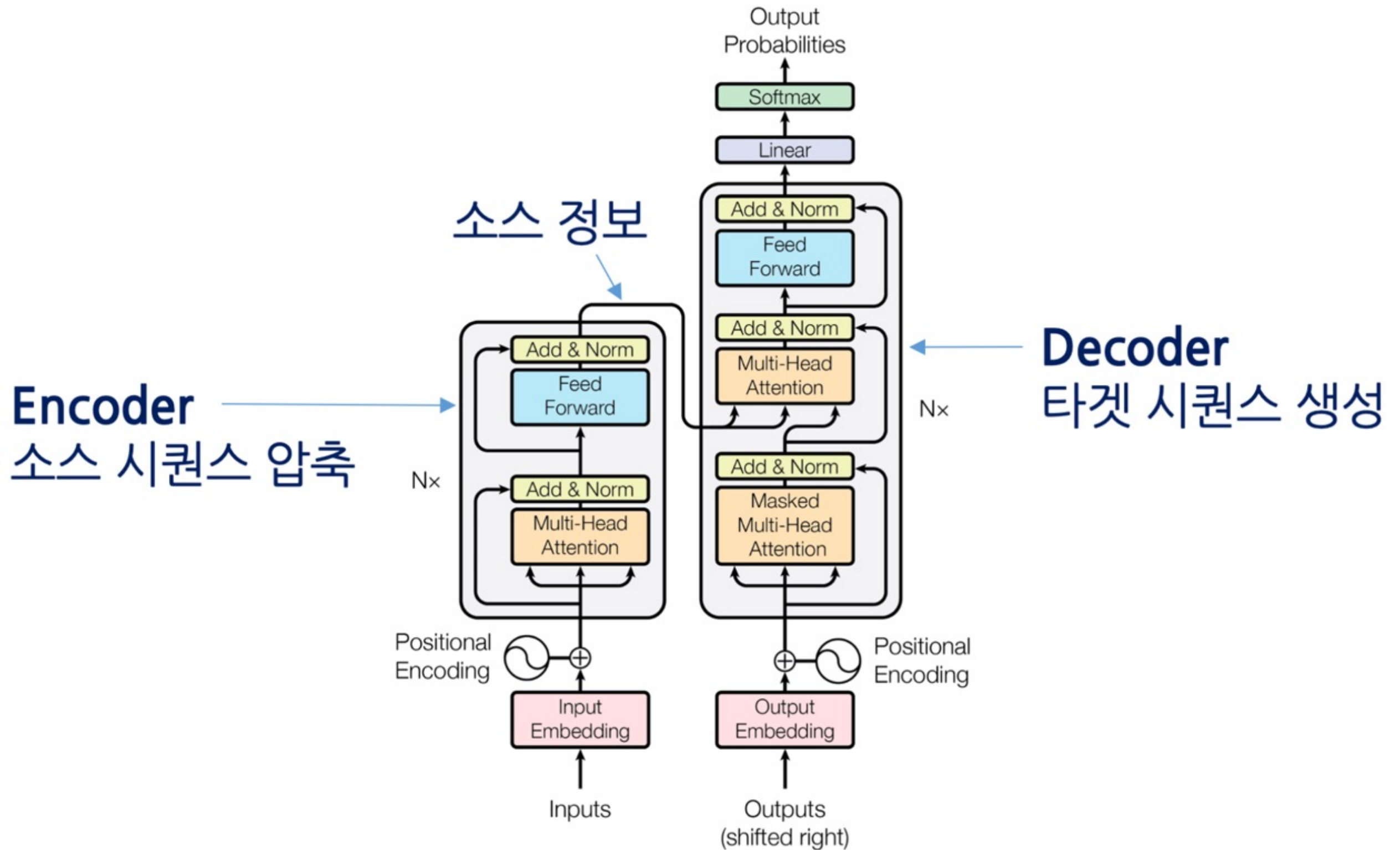


Transformer

언어 모델

- ▶ 언어 모델(Language Model)
 - ▶ 단어들이 주어졌을 때 다음에 나타날 가장 적절한 단어들의 확률을 계산하는 모델
 - ▶ 문장이 그럴듯한지를 확률로 나타내는 것
 - ▶ NLP에 도입하여 성능을 개선했다
- ▶ 초기에는 통계적 기반, HMM을 사용했다
- ▶ 신경망기반의 언어 모델
 - ▶ 2003년 벤지오 교수가 처음 제안
 - ▶ 공개된 대량의 텍스트를 사용하여 사전학습 시킨 모델 - BERT, GPT
 - ▶ 2017년 이후 트랜스포머가 언어 모델에 대부분 사용되고 있다
- ▶ 트랜스포머
 - ▶ 인코더-디코더를 개선한 모델로 어텐션 기법을 도입
 - ▶ 언어모델에 RNN은 사용되지 않는다

트랜스포머



어텐션

- ▶ 초기의 어텐션: café를 디코딩할 때 주의해서 볼 내용은?
 - ▶ RNN 구조에서 동작 seq2seq 모델 사용



- ▶ **셀프 어텐션**: 입력 전체 토큰에 대해 각각 수행하는 어텐션
 - ▶ CNN과 RNN의 단점을 개선



BERT와 GPT

- ▶ 2018 ~
- ▶ BERT(Bidirectional Encoder Representations from Transformers)
 - ▶ 마스크 언어모델(Masked Language Model)로 학습
 - ▶ 트랜스포머를 사용
 - ▶ 양방향(bidirectional) 학습을 한다
- ▶ GPT(Generative Pre-trained Transformer)
 - ▶ 트랜스포머를 사용하여 언어모델(Language Model)을 구현
 - ▶ 문장 시작부터 순차적으로 계산하는 일방향(unidirectional)

수고하셨습니다